INTERVIEWHUB · INTERVIEW PREP

Python for SRE & DevOps — InterviewHub

Platform, SRE & DevOps interview guide

Questions	**150 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/python_sre_interview_guide.html**
Printed	**2026-10-23**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What are mutable and immutable types in Python, and why do mutable default arguments cause bugs?

Core language

🎙 **Spoken answer · ~67 s**

Immutable objects such as ints, strings, tuples and frozensets cannot change after creation, while mutable ones such as lists, dicts and sets can be modified in place. Variables are references, so assigning one list to another name gives two names for the same object, and a change through one is visible through the other, which surprises people in scripts that pass configuration around. The classic bug is a mutable default argument: the default value is evaluated once when the function is defined, so a function with an empty list default shares that same list across calls, and results accumulate between invocations. In an automation script that might leak hosts from one run into the next. The fix is to use None as the default and create a new list inside the function. To avoid aliasing, copy with the copy method, slicing, or copy.deepcopy for nested structures. I also prefer immutable tuples or frozen dataclasses for configuration that must not change.

Likely follow-ups

- How do you copy a nested dictionary safely?
- Why are dictionary keys required to be hashable?

Immutable types never change; mutable ones change in place and are shared by reference. Default arguments are evaluated once, so use None and create the list inside the function. Copy or deepcopy to avoid aliasing.

```
def add_host(h, hosts=None):
    hosts = [] if hosts is None else hosts
    hosts.append(h)
    return hosts
```

```
import copy; cfg2 = copy.deepcopy(cfg)
```

Interview tip: Say default arguments are evaluated once at definition.

📚 **Official docs**

- Python Documentation

certs.eknathalabs.com

Q2 How do list, dict and set comprehensions and generator expressions differ, and when do you use each?

Core language

🎙 **Spoken answer · ~63 s**

A comprehension builds a collection in one expression: square brackets make a list, curly braces with key and value make a dictionary, and curly braces with a single expression make a set. They are concise and usually faster than an equivalent loop with append. A generator expression uses parentheses and produces items lazily, one at a time, without building the whole collection in memory. For automation this matters with large inputs: summing the sizes of a million log entries with a generator expression uses constant memory, while a list comprehension would allocate the whole list. I use comprehensions when the result is needed as a collection and the logic is simple, for example filtering running instances or building a lookup dictionary by instance ID. When a comprehension needs nested conditions or side effects, a plain loop is clearer. Generators also compose with functions like any, all, sum and join, short-circuiting where possible.

Likely follow-ups

- When does a generator expression save memory?
- Why avoid side effects inside comprehensions?

Comprehensions build lists, dicts or sets eagerly in one expression; generator expressions are lazy and use constant memory. Use comprehensions for simple filtering and mapping, generators for large streams, and loops when logic gets complex.

```
running = [i['id'] for i in instances if i['state'] == 'running']
```

```
total = sum(len(line) for line in open('big.log'))
```

Interview tip: Say lazy generators for large data, loops when logic is complex.

📚 **Start here (official docs)**

- Python documentation

certs.eknathalabs.com

Q3 What are generators and the yield keyword, and how do they help with log and data streaming?

Core language

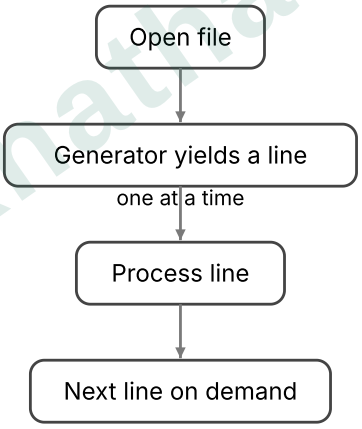
🎙 Spoken answer · ~67 s

A generator function contains yield and returns an iterator that produces values one at a time, pausing at each yield and resuming where it left off, keeping its local state between calls. This lets you process data as a stream instead of loading everything into memory. In operations work, a generator that reads a multi-gigabyte log line by line, filters errors and yields parsed records lets you chain stages, read, parse, filter and aggregate, each consuming the previous one lazily, similar to a Unix pipeline, with memory use staying flat. Generators are also useful for paginated APIs: a function that yields items page by page hides pagination from the caller. Related features include yield from to delegate to another generator, send to pass values in, and closing with try and finally for cleanup. A generator can be iterated only once, so if you need multiple passes you must recreate it or materialise the results. They make code readable and efficient together.

Likely follow-ups

- Why can a generator be consumed only once?
- How does yield from help with pagination?

🗺 Lazy log streaming



A generator uses yield to produce values lazily and keeps state between calls. Chain generators for streaming log processing with flat memory, and wrap paginated APIs so callers just iterate. They are single-use.

```
def errors(path):
    with open(path) as f:
        for line in f:
            if 'ERROR' in line:
                yield line

for e in errors('app.log'):
    print(e)
```

certs.eknathalabs.com

Interview tip: Say streaming pipelines keep memory flat.

📚 **Start here (official docs)**

- Python documentation

Q4 How do decorators work and what are practical uses in automation code? Core language

🎙 **Spoken answer · ~63 s**

A decorator is a function that takes a function and returns a new function, usually wrapping the original to add behaviour before or after the call. The at-sign syntax is shorthand for reassigning the name to the wrapped version. Inside the wrapper you call the original with its arguments and return the result, and you apply functools.wraps so the wrapped function keeps its name and docstring, which matters for logging and debugging. Decorators can take arguments by adding another outer function. Useful applications in operations scripts include retrying a flaky API call with backoff, timing how long a function takes, logging entry and exit, caching results, enforcing permissions, rate limiting, and registering functions as commands or handlers, as frameworks like click and FastAPI do. Class-based decorators are possible too. A caution is that heavy stacking of decorators can obscure control flow and stack traces, so I keep them focused and well tested.

Likely follow-ups

- Why use functools.wraps?
- How do you write a decorator that takes arguments?

A decorator wraps a function to add behaviour such as retries, timing, logging or caching. Use functools.wraps to preserve metadata, add an outer function for arguments, and keep decorators focused so control flow stays clear.

```
import functools, time
def retry(n=3, delay=1):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **kw):
            for i in range(n):
                try: return fn(*a, **kw)
                except Exception:
                    if i == n - 1: raise
                    time.sleep(delay * 2 ** i)
        return wrapper
    return deco
```

Interview tip: Say functools.wraps and a retry decorator as the example.

ocrts.eknathalabs.com

📚 **Start here (official docs)**

- Python documentation

Q5 What are context managers and how do you use them for resources such as files, locks and connections?

Core language

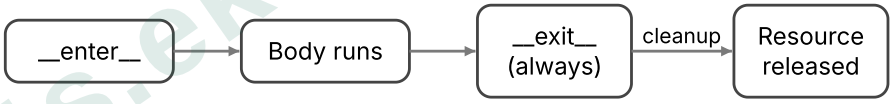
🎙 **Spoken answer · ~63 s**

A context manager defines setup and cleanup around a block, used with the with statement. Entering calls the enter method and leaving, even through an exception, calls exit, so resources are released reliably. Files, locks, database connections, temporary directories and subprocess handles all support it. In scripts this prevents leaks, such as leaving a file open or a lock held when an error occurs. You can write your own either as a class with enter and exit methods, or more simply with the contextlib.contextmanager decorator on a generator that yields once, with try and finally around the yield for cleanup. Practical examples: a context manager that takes a lock file, a timer that logs elapsed time, a temporary change of working directory or environment variable, or one that opens an SSH session and always closes it. contextlib also offers suppress for expected exceptions and ExitStack to manage a dynamic number of contexts.

Likely follow-ups

- How does contextlib.contextmanager work?
- When would you use ExitStack?

🗺 **with block**



A context manager guarantees cleanup via with, even on exceptions, for files, locks, connections and temp dirs. Write one with enter and exit, or with contextlib.contextmanager and try and finally around a single yield. ExitStack handles dynamic numbers of contexts.

```
from contextlib import contextmanager
@contextmanager
def timer(label):
    t = time.perf_counter()
    try: yield
    finally: print(label, time.perf_counter() - t)
```

```
with open('a.txt') as f, open('b.txt', 'w') as g:
    g.write(f.read())
```

Interview tip: Say with guarantees cleanup even when errors occur.

certs.eknathalabs.com

📚 **Official docs**

- Python: pathlib

Q6 How should you handle exceptions in operational scripts?

Core language

🎙 **Spoken answer · ~65 s**

The aim is to fail clearly, clean up and never hide real problems. I catch specific exceptions I can handle, such as a connection timeout or a missing file, and let unexpected ones propagate, since a bare except or catching Exception silently turns bugs into mysterious behaviour. When I catch, I either recover meaningfully, retry, or add context and re-raise, using raise from to keep the original cause. Use finally or context managers for cleanup, and the else clause for code that should run only when no exception happened. For scripts, the top level catches known error types, logs them with enough detail, including the traceback at debug level, and exits with a nonzero status so cron, CI and orchestrators notice. Custom exception classes express domain errors like a quota exceeded or a failed validation. I avoid using exceptions for normal control flow in hot paths, and I never swallow KeyboardInterrupt or SystemExit by accident.

Likely follow-ups

- Why avoid a bare except?
- What does raise from do?

Catch specific exceptions you can handle, let others propagate, add context with raise from, clean up with finally or context managers, use custom exceptions for domain errors, and exit nonzero at the top level after logging.

```
try:
    r = client.get(url, timeout=5)
    r.raise_for_status()
except requests.Timeout as e:
    raise RuntimeError(f'API timed out: {url}') from e
```

Interview tip: Say never swallow errors silently; exit nonzero.

📚 **Start here (official docs)**

- Python documentation

certs.eknathalabs.com

Q7 How do type hints, dataclasses and pydantic improve automation code? Core language

🎙 Spoken answer · ~66 s

Type hints annotate parameters, returns and variables with expected types. Python does not enforce them at run time, but tools such as mypy and editors check them, catching mistakes like passing a string where a list is expected, and they document intent for the next reader. Dataclasses reduce boilerplate for classes that mainly hold data, generating init, repr and equality, with options for immutability through frozen and for default factories. Pydantic goes further by validating and converting data at run time: you define a model with typed fields, and constructing it from a dictionary or JSON checks types, applies defaults and raises clear errors, which is ideal for configuration files, API responses and environment settings. In operational code that handles messy inputs, this catches problems at the boundary instead of deep inside a script. I use hints everywhere in shared code, dataclasses for internal records and pydantic for external input, and run mypy in CI to keep them accurate.

Likely follow-ups

- Does Python enforce type hints at run time?
- When would you choose pydantic over a dataclass?

Type hints document and let mypy catch errors; dataclasses remove boilerplate for data records; pydantic validates and converts external input such as config and API data at run time. Run mypy in CI.

```
from pydantic import BaseModel
class Settings(BaseModel):
    region: str
    retries: int = 3
Settings(**{'region': 'us-east-1'})
```

```
mypy src/
```

Interview tip: Say validate at the boundary with pydantic.

📚 Official docs

- Pydantic documentation

certs.eknathalabs.com

Q8 What is the GIL and how does it affect Python in infrastructure work? Core language

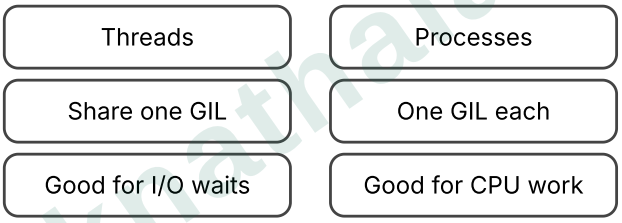
🎙 Spoken answer · ~63 s

The Global Interpreter Lock is a mutex in the standard CPython interpreter that lets only one thread execute Python bytecode at a time. As a result, threads do not give true parallelism for CPU-bound Python code, since they take turns, but they work well for I/O-bound work, because the lock is released while waiting on the network, disk or a subprocess. Most infrastructure scripts are I/O-bound, calling APIs, running SSH commands or reading files, so a thread pool or asyncio gives real speedups. For CPU-bound work, such as parsing large datasets or hashing many files, use multiprocessing or a process pool, which uses separate interpreters, or move hot loops to native extensions, as libraries like numpy and cryptography do. Recent Python versions are making the lock optional through experimental free-threaded builds, but code should not depend on that yet. So the rule is threads or asyncio for waiting, processes for computing.

Likely follow-ups

- Why do threads help for API calls but not for hashing?
- What is a free-threaded build?

🗺 Threads vs processes



The GIL lets one thread run Python bytecode at a time, so threads help I/O-bound work but not CPU-bound work. Use threads or asyncio for waiting on networks and processes or native extensions for heavy computation.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
```

```
python -c "import sys; print(sys._is_gil_enabled())"
```

Interview tip: Say threads for waiting, processes for computing.

📚 Official docs

- Python: concurrent.futures
- Python Documentation

certs.eknathalabs.com

Q9 Explain scope, closures and common pitfalls with late binding in loops.

Core language

🎙 **Spoken answer · ~68 s**

Python resolves names using the LEGB rule: local, enclosing, global, then built-in. Assigning to a name inside a function makes it local unless declared global or nonlocal, which is why modifying a module-level counter inside a function without the global keyword raises an error. A closure is a function that remembers variables from its enclosing scope even after that scope has finished, which is how decorators and factory functions work. A frequent pitfall is late binding: closures capture the variable, not its value at creation time, so functions created in a loop all see the final value of the loop variable. For example a list of callbacks built in a loop each using the loop variable will all use the last one. The fix is to bind the value with a default argument, use functools.partial, or create the function in a helper. Related pitfalls include shadowing built-in names like list or id, and unintentionally modifying global state, which makes scripts hard to test.

Likely follow-ups

- How do you fix late binding in a list of callbacks?
- What does nonlocal do?

Names resolve local, enclosing, global, built-in. Closures capture variables, not values, so loop-created functions share the last value; bind with default arguments or functools.partial. Use nonlocal or global deliberately and avoid shadowing built-ins.

```
callbacks = [lambda x=h: ping(x) for h in hosts]
```

```
from functools import partial
callbacks = [partial(ping, h) for h in hosts]
```

Interview tip: Say closures capture variables, not values.

📚 **Start here (official docs)**

- Python documentation

certs.eknathalabs.com

Q10 How do strings, bytes and encodings work, and what problems appear when handling logs and subprocess output?

Core language

🎙 Spoken answer · ~67 s

In Python 3 a str is a sequence of Unicode characters, while bytes is a sequence of raw bytes, and they are not interchangeable. Converting uses encode and decode with an encoding, usually UTF-8. Problems arise at boundaries: files opened in text mode use a default encoding that depends on the environment, so scripts behave differently in a container with a C locale, and I therefore pass encoding explicitly. Logs may contain invalid byte sequences, so decoding with the errors argument set to replace or backslashreplace avoids crashes, while binary files are read in binary mode. Subprocess output is bytes unless text mode or an encoding is requested. Network protocols and hashing functions need bytes, so encode before hashing. F-strings are the preferred way to format text, with format specifiers for widths and precision, and repr or the r conversion for debugging. For data handling I decode at the input boundary, work in str internally and encode at the output boundary.

Likely follow-ups

- How do you read a log file with invalid UTF-8?
- Why must text be encoded before hashing?

str is Unicode text and bytes is raw data. Decode at input, work in str, encode at output, pass encodings explicitly, and use errors=replace for dirty logs. Subprocess output needs text mode or decoding, and hashing needs bytes.

```
open('app.log', encoding='utf-8', errors='replace')
```

```
subprocess.run(cmd, capture_output=True, text=True, check=True)
```

```
hashlib.sha256('abc'.encode()).hexdigest()
```

Interview tip: Say specify the encoding explicitly.

📚 Official docs

- Python: subprocess

certs.eknathalabs.com

Q11 How do modules, packages, imports and the __main__ guard work in a script project?

Packaging & envs

🎙 Spoken answer · ~68 s

A module is a Python file and a package is a directory of modules, traditionally with an init file, imported by name through the module search path, which includes the script directory, installed site-packages and the PYTHONPATH. Importing runs the module's top-level code once and caches it. Relative imports inside a package use dots. The name dunder variable equals main when a file is run directly and the module name when imported, so the guard if name equals main protects code that should run only as a script, allowing the same file to be imported by tests without side effects. I structure projects with a package directory, thin entry points, a pyproject file and tests alongside, and install the project in editable mode for development so imports work the same as in production. Pitfalls are circular imports, which I fix by restructuring or importing inside functions, naming a file like a standard library module such as json.py, which shadows it, and modifying sys.path hacks.

Likely follow-ups

- Why does naming a file json.py break things?
- How do you avoid circular imports?

Modules are files and packages are directories imported through the search path. Guard script code with if __name__ == '__main__' so tests can import safely. Use a package layout with an editable install, avoid shadowing stdlib names and break circular imports by restructuring.

```
if __name__ == '__main__':
    raise SystemExit(main())
```

```
pip install -e .
```

Interview tip: Say the main guard keeps modules importable and testable.

📚 Official docs

- Python: venv
- Python Packaging User Guide

certs.eknathalabs.com

Q12 How do you process a very large log file efficiently in Python?

Files & data

🎙 Spoken answer · ~70 s

The key is to avoid loading it all into memory. Iterating over a file object reads it line by line with buffering, so a for loop over the file handles gigabytes with constant memory. I combine that with generators to filter and transform lazily, compile regular expressions once outside the loop, and keep per-line work cheap. For counting or aggregating, I use collections.Counter or defaultdict and update them as I stream. For compressed files, gzip.open in text mode streams the same way. When a file is too large for one core, I split it into byte ranges aligned to line boundaries and process chunks in a process pool, then merge results. To follow a live log, I read, then seek and poll like tail, and handle rotation by detecting inode changes. For extremely large analytic work I consider a proper tool, such as grep, awk, ripgrep or loading into DuckDB or a log platform, since a Python loop is rarely the fastest option for a simple search.

Likely follow-ups

- How do you tail a file that is rotated?
- When is awk or ripgrep better than Python?

Iterate the file object line by line with generators, precompile regexes, aggregate with Counter, stream gzip in text mode, split into chunks across processes if needed and handle rotation when tailing. Use grep, ripgrep or DuckDB when they fit better.

```
import re, collections, gzip
rx = re.compile(r' 5\d\d ')
c = collections.Counter()
with gzip.open('access.log.gz', 'rt') as f:
    for line in f:
        if rx.search(line): c[line.split()[0]] += 1
```

```
rg -c ' 5[0-9][0-9] ' access.log
```

Interview tip: Say stream with constant memory and precompile regexes.

📚 Official docs

- Python: pathlib
- Python Documentation

certs.eknathalabs.com

Q13 How do you read and write JSON, YAML and TOML safely?

Files & data

🎙 **Spoken answer · ~68 s**

For JSON the standard json module handles loads and dumps, and I pass indent and sort keys for readable, stable output that produces clean diffs in Git. For YAML I use PyYAML or ruamel.yaml, and the critical rule is to call safe_load, never load without a safe loader, because the full loader can construct arbitrary Python objects and execute code from untrusted input. Ruamel preserves comments and ordering, useful when editing files a human maintains. TOML is read with the standard tomllib module from Python 3.11, which is read-only, with a separate library such as tomli-w for writing. Whatever the format, I validate the structure after loading, with pydantic or jsonschema, so a missing key fails with a clear message. When writing configuration files that other processes read, I write to a temporary file and atomically rename, to avoid readers seeing a partial file. I also open files with an explicit encoding and avoid putting secrets into files that may be committed.

Likely follow-ups

- Why is yaml.load dangerous?
- Why write to a temp file then rename?

Use json with indent and sorted keys, yaml.safe_load never full load, tomllib for TOML, validate with pydantic or jsonschema, and write via a temp file and atomic rename. Preserve comments with ruamel when editing human-maintained YAML.

```
import yaml
cfg = yaml.safe_load(open('config.yml', encoding='utf-8'))
```

```
import tomllib
with open('pyproject.toml', 'rb') as f: d = tomllib.load(f)
```

```
json.dumps(data, indent=2, sort_keys=True)
```

Interview tip: Say safe_load only.

📚 **Start here (official docs)**

- Python documentation

certs.eknathalabs.com

Q14 How do you use regular expressions to parse logs, and what are the pitfalls? Files & data

🎙 Spoken answer · ~71 s

The re module compiles patterns, and I compile once and reuse. For structured parsing I use named groups, so the result is a dictionary of fields such as client IP, timestamp, method, path and status, which is much clearer than numeric group indexes. Use search to find a pattern anywhere, match to anchor at the start, fullmatch for the entire string and finditer for all occurrences, with raw strings to avoid backslash confusion. Pitfalls: catastrophic backtracking with nested quantifiers on adversarial or long inputs, which can freeze a script, so keep patterns specific and avoid overlapping wildcards; greedy matching that spans more than intended, fixed with non-greedy quantifiers or negated character classes; fragile regexes on formats with proper parsers, where JSON, XML or well-defined formats should use a real parser; and unhandled non-matching lines, where match returns None and attribute access fails. For common formats like Apache logs, a maintained parser or splitting on known delimiters can be simpler and faster. I test patterns with sample lines, including bad ones.

Likely follow-ups

- What is catastrophic backtracking?
- When is a regex the wrong tool?

Compile once, use named groups, choose search, match or fullmatch deliberately and use raw strings. Beware backtracking, greedy matches and None results; use real parsers for structured formats and test with bad lines.

```
rx = re.compile(r'(?P<ip>\S+) .* "(?P<method>[A-Z]+) (?P<path>\S+).*" (?P<status>\d{3})')
m = rx.match(line)
if m: rec = m.groupdict()
```

Interview tip: Say named groups and handle the None case.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q15 How do you use pathlib and shutil for file operations, and how do you write files atomically?

Files & data

🎙 Spoken answer · ~64 s

Pathlib models paths as objects: the slash operator joins paths, and methods such as exists, is_file, mkdir with parents and exist_ok, glob and rglob, read_text, write_text, stat and unlink handle common operations in a portable, readable way, replacing a mix of os.path calls. Shutil adds higher-level operations: copy and copy2, copytree, move, rmtree, disk_usage and which. For safe writes, I avoid truncating the destination first. The atomic pattern is to write the new content to a temporary file in the same directory, flush and fsync it, then use os.replace to rename over the destination, which is atomic on the same filesystem, so readers see either the old or the new file, never a partial one. I set permissions explicitly, especially for files containing secrets, with a restrictive mode from creation using os.open flags or tempfile. For deletion, I check paths carefully before rmtree, to avoid removing the wrong directory through an empty variable.

Likely follow-ups

- Why must the temp file be in the same directory?
- How do you guard against a dangerous rmtree?

Pathlib provides readable path handling and shutil gives copy, move and rmtree. Write atomically by creating a temp file in the same directory, fsync, then os.replace, set restrictive permissions and validate paths before deleting.

```
import os, tempfile, pathlib
d = pathlib.Path('/etc/app')
with tempfile.NamedTemporaryFile('w', dir=d, delete=False) as t:
    t.write(data); t.flush(); os.fsync(t.fileno())
os.replace(t.name, d / 'app.conf')
```

Interview tip: Say temp file then os.replace for atomic updates.

📚 Official docs

- Python: pathlib

certs.eknathalabs.com

Q16 How do you run external commands safely and robustly with subprocess?

Files & data

🎙 Spoken answer · ~71 s

I use subprocess.run with a list of arguments and without shell mode, which avoids shell injection and quoting bugs, because each argument is passed directly. I set check true so a nonzero exit raises CalledProcessError, capture_output and text to collect decoded output, a timeout so a hung command cannot block the script forever, and an explicit env and cwd when needed. If a shell feature is truly required, I use shlex.quote on any untrusted value, but usually I do the piping in Python. For long-running commands I stream output with Popen and iterate the stdout pipe, taking care of deadlocks when both stdout and stderr fill their buffers, avoided by merging stderr into stdout or using communicate. I log the command and exit code, but never secrets passed as arguments, since they appear in process listings; secrets go through environment variables or stdin. I handle FileNotFoundError when the binary is missing, and terminate the process tree on timeout where required. Whenever a Python library exists, it is better than shelling out.

Likely follow-ups

- Why is shell=True dangerous?
- How do you avoid a deadlock reading stdout and stderr?

🗺 Safe subprocess



Use subprocess.run with an argument list, no shell, check=True, capture_output and text, a timeout and explicit env. Avoid secrets in arguments, handle missing binaries, stream with Popen carefully and prefer libraries over shelling out.

```
r = subprocess.run(['kubectl', 'get', 'pods', '-o', 'json'], capture_output=True,
text=True, check=True, timeout=30)
pods = json.loads(r.stdout)
```

Interview tip: Say a list, not a shell string; always set a timeout.

📚 Official docs

- Python: subprocess

certs.eknathalabs.com

Q17 How do you work with archives and compressed files (tar, zip, gzip) in Python?

Files & data

🎙 Spoken answer · ~69 s

The standard library covers the common formats: the tarfile module reads and writes tar archives with compression modes for gzip, bz2 and xz, the zipfile module handles zip files, and gzip, bz2 and lzma handle single compressed streams. Shutil.make_archive and unpack_archive provide quick helpers. Security is the main concern when extracting archives from untrusted sources: a malicious tar or zip can contain paths with parent directory references or absolute paths, or symlinks, that write outside the target directory, known as path traversal or zip slip. Newer Python versions add an extraction filter parameter to tarfile, recommended as data, which rejects dangerous members, and for older versions I validate each member's resolved path against the target directory before extracting. Also limit the total size and number of files to resist decompression bombs. For backups I stream data into the archive rather than building it in memory, and add checksums so that restore can verify integrity. I record permissions and ownership only when the use case needs them.

Likely follow-ups

- What is a zip slip attack?
- What does the tarfile data filter do?

Use tarfile, zipfile, gzip and shutil helpers. When extracting untrusted archives, defend against path traversal and symlinks with the tarfile data filter or manual path checks, limit sizes, stream large archives and add checksums for integrity.

```
import tarfile
with tarfile.open('backup.tgz', 'w:gz') as t:
    t.add('/etc/app', arcname='app')
```

```
with tarfile.open('in.tgz') as t:
    t.extractall('/tmp/out', filter='data')
```

Interview tip: Say never extract untrusted archives without validation.

📚 Official docs

- Python: pathlib
- Python Documentation

certs.eknathalabs.com

Q18 How should a Python tool read configuration from files, environment variables and command-line flags?

Files & data

🎙 Spoken answer · ~65 s

I define a clear precedence and a single place that resolves it. A common order from lowest to highest is built-in defaults, a configuration file, environment variables, then command-line flags, so operators can override anything at run time without editing files. Environment variables follow the twelve-factor style, with a prefix for the tool, for example MYTOOL_REGION. I model the final configuration as a typed object, using pydantic settings or a dataclass, which validates types, applies defaults and gives clear errors for missing required values, and the rest of the code receives that object instead of reading os.environ everywhere, which keeps it testable. Secrets are read from environment variables, files mounted by the orchestrator or a secrets manager, never defaulted or logged, and printed configuration is redacted. I document each option, support a dry-run or print-config flag to show the effective values, and keep configuration files in a format such as YAML or TOML with safe loading.

Likely follow-ups

- Why have one place that resolves configuration?
- How do you redact secrets in logs?

Define precedence defaults, file, environment, flags; resolve once into a typed, validated settings object; read secrets from the environment or a secrets manager; redact when printing, and offer a print-config option.

```
from pydantic_settings import BaseSettings
class S(BaseSettings):
    region: str = 'us-east-1'
    token: str
    model_config = {'env_prefix': 'MYTOOL_'}
```

```
MYTOOL_REGION=eu-west-1 mytool run
```

Interview tip: Say one typed settings object, secrets never logged.

📚 Official docs

- Python: subprocess
- Python: argparse
- Python: pathlib

certs.eknathalabs.com

Q19 How do you work with temporary files and directories securely?

Files & data

🎙 **Spoken answer · ~67 s**

Predictable temporary file names in a shared directory like tmp are a security risk, since another user could create a symlink with that name and trick the script into overwriting something else, known as a symlink attack. The tempfile module avoids this: NamedTemporaryFile, TemporaryDirectory and mkstemp create uniquely named files with restrictive permissions, opened exclusively so they cannot be pre-created by someone else. TemporaryDirectory as a context manager removes the directory and its contents when the block exits, even on errors, which keeps scripts clean. When a file must be visible to another process, I use delete false and clean up explicitly, or create the temp file next to the destination for atomic replacement. For sensitive data, I avoid writing to disk at all, or use a directory on a memory-backed filesystem, and remove it securely when done. I never build paths by string concatenation with user input, instead resolving and checking that the result stays within an expected directory.

Likely follow-ups

- What is a symlink attack on a temp file?
- When would you use mkstemp?

Use tempfile for unique, securely created files and directories with restrictive permissions, TemporaryDirectory as a context manager for cleanup, same-directory temps for atomic replacement, and avoid predictable names or writing secrets to disk.

```
import tempfile
with tempfile.TemporaryDirectory() as d:
    work(d)
```

```
fd, path = tempfile.mkstemp(prefix='job-')
```

Interview tip: Say predictable temp names are a symlink-attack risk.

📚 **Official docs**

- Python: pathlib

certs.eknathalabs.com

Q20 How do you compute checksums and compare files or directories for changes?

Files & data

🎙 Spoken answer · ~67 s

The hashlib module provides algorithms such as SHA-256, and for files I read in chunks and update the hash incrementally, so memory use is constant even for very large files. Chunk reads of one megabyte or so are typical, and Python 3.11 adds hashlib.file_digest. For integrity verification of downloads I compare against a published checksum using hmac.compare_digest to avoid timing leaks when it matters. To compare directories, I walk both trees, build a mapping of relative path to hash, size and mode, and diff the dictionaries to find added, removed and changed files, or use filecmp.dircmp for a quick comparison. For change detection at scale, I first compare size and modification time, and compute hashes only when needed. Note that MD5 and SHA-1 are broken for security purposes, fine only for non-security checks such as cache keys, so I default to SHA-256. For configuration drift tasks, I normalise things like line endings and ordering before comparing, to avoid false positives.

Likely follow-ups

- Why read the file in chunks?
- Why not use MD5 for integrity of downloads?

Hash files in chunks with hashlib SHA-256, or file_digest on newer versions, and compare with a published checksum. For directories, map relative path to hash and diff the maps, checking size and mtime first. Avoid MD5 and SHA-1 for security.

```
import hashlib
h = hashlib.sha256()
with open(p, 'rb') as f:
    for chunk in iter(lambda: f.read(1 << 20), b''):
        h.update(chunk)
print(h.hexdigest())
```

Interview tip: Say chunked hashing, SHA-256, compare maps for directories.

📚 Official docs

- Python: pathlib
- py-spy

certs.eknathalabs.com

Q21 How do you choose between argparse, click and typer for command-line tools?

CLI & scripting

🎙 Spoken answer · ~65 s

Argparse is in the standard library, so it needs no dependency, which suits small scripts and environments where installing packages is hard. It supports subcommands, types, choices and help text but is verbose. Click is a widely used third-party library built on decorators, giving concise definitions, nested command groups, prompts, colours, progress bars, file handling and good testing support through its runner. Typer builds on click and uses Python type hints to define arguments and options, which makes the code very compact and produces help and shell completion automatically. For a quick script I use argparse; for a tool that will grow, with subcommands and users, I use click or typer. Whatever I choose, good practice is meaningful help text, sensible defaults, a version flag, a dry-run option for changes, nonzero exit codes on failure, environment variable fallbacks, and keeping the logic in importable functions so it can be tested without invoking the command line.

Likely follow-ups

- What does typer add over click?
- How do you test a click command?

Argparse needs no dependency and suits small scripts; click offers concise groups and testing support; typer uses type hints for compact definitions and completion. Choose by size and audience, and keep logic in testable functions with exit codes, help and dry-run.

```
import typer
app = typer.Typer()
@app.command()
def scale(name: str, replicas: int = 2, dry_run: bool = False): ...
```

```
if __name__ == '__main__': app()
```

Interview tip: Say argparse for small scripts, click or typer for growing tools.

📚 Official docs

- Python: argparse
- Python: subprocess

certs.eknathalabs.com

Q22 How do exit codes and signals work in Python scripts, and why do they matter?

CLI & scripting

🎙 Spoken answer · ~67 s

Exit codes are how scripts tell cron, CI systems, shells and orchestrators whether they succeeded: zero means success and any nonzero means failure, and conventions distinguish kinds of failure, for example one for general errors and two for usage errors. In Python, sys.exit with a number, or raising SystemExit, sets it, and an uncaught exception exits with one. If a script swallows errors and exits zero, automation continues as if nothing happened, which is a common and costly bug, so a top-level handler should map failures to nonzero codes. Signals notify a process asynchronously: SIGTERM requests a graceful stop, as sent by Kubernetes and systemd, SIGINT comes from Ctrl-C, and SIGKILL cannot be caught. The signal module lets a script register handlers to finish current work, flush output, release locks and exit cleanly within the grace period. I also make sure child processes are terminated, and that handlers do minimal work, usually setting a flag checked by the main loop.

Likely follow-ups

- Why can SIGKILL not be handled?
- What should a SIGTERM handler do?

Exit code zero means success and nonzero failure, so scripts must never hide errors. Handle SIGTERM and SIGINT with the signal module to stop gracefully, flush output, release locks and terminate children, usually by setting a flag the main loop checks.

```
import signal, sys
stop = False
def handler(sig, frm):
    global stop; stop = True
signal.signal(signal.SIGTERM, handler)
```

```
sys.exit(1)
```

Interview tip: Say orchestrators rely on exit codes and SIGTERM.

📚 Official docs

- Kubernetes: Determine the reason for Pod failure
- Python Documentation

certs.eknathalabs.com

Q23 How do you set up logging properly in a Python tool or service?

CLI & scripting

🎙 Spoken answer · ~66 s

I use the standard logging module rather than print. Each module gets a logger named with name dunder, so the hierarchy reflects the package, and libraries only create loggers and never configure handlers, leaving that to the application. The application configures logging once at startup, with level, format and handlers, through basicConfig, dictConfig or a file. A good format includes timestamp, level, logger name and message, and for services I use structured JSON logs so a central system can index fields, including request or job IDs through logging adapters or context variables. I log to standard error or standard output for containers and let the platform collect them, use rotating handlers only for traditional file logging, and choose levels carefully: debug for detail, info for milestones, warning for recoverable issues, error for failures. I pass arguments lazily with percent formatting rather than building strings, log exceptions with exc_info or the exception method, and never log secrets or personal data.

Likely follow-ups

- Why should libraries not configure logging?
- How do you add a request ID to every log line?

Use logging with per-module loggers, configure once in the application, include timestamp, level and name, emit JSON for services with context such as request IDs, write to stdout or stderr in containers, log exceptions with exc_info and never log secrets.

```
import logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(name)s %
(message)s')
log = logging.getLogger(__name__)
log.info('scaling %s to %d', name, n)
```

Interview tip: Say libraries create loggers, applications configure them.

📚 Official docs

- Python: logging
- Python Documentation

certs.eknathalabs.com

Q24 How do you make a Python script work well in Unix pipelines, reading stdin and writing clean stdout?

CLI & scripting

🎙 Spoken answer · ~70 s

Tools that compose well follow Unix conventions. Read from standard input when no file argument is given, using sys.stdin iterated line by line so it streams, or fileinput to accept files or stdin uniformly. Write results to standard output and keep diagnostics, progress and errors on standard error, so piping the output into another command is not polluted. Provide machine-friendly output, such as one record per line or JSON lines, with an option for a human-readable table, and handle a closed pipe gracefully: when the downstream command like head exits early, writing raises BrokenPipeError, which should be caught or the SIGPIPE default restored to exit quietly. Respect exit codes, and avoid interactive prompts when input is not a terminal, detected with isatty. Flush output when streaming so consumers see data promptly, and process input lazily rather than reading everything. Support a dash for stdin in file arguments. These details make a script feel like a native tool and make it easy to compose with grep, jq and xargs.

Likely follow-ups

- How do you avoid a BrokenPipeError traceback with head?
- Why send progress to stderr?

Stream stdin line by line, write data to stdout and diagnostics to stderr, offer JSON lines output, handle broken pipes, check isatty before prompting, flush when streaming and use exit codes properly.

```
import sys
for line in sys.stdin:
    print(line.strip().upper())
```

```
kubectl get pods -o json | python tool.py | head -5
```

Interview tip: Say data to stdout, messages to stderr.

📚 Official docs

- Python: venv
- Python Documentation

certs.eknathalabs.com

Q25 How do you design automation scripts to be idempotent and safe to re-run?

CLI & scripting

🎙 Spoken answer · ~68 s

A re-runnable script describes desired state and checks current state before acting. Instead of blindly creating a resource, it looks it up and creates it only if missing, or updates it if different, as in ensure-style functions. I use stable identifiers and names or tags, so a second run finds what the first created. For destructive or one-time actions, I record progress, using a state file or an external marker, so a script that failed halfway can resume without repeating completed steps, and I make each step independently idempotent. Deletions are guarded by checks, confirmations and a dry-run mode that prints intended changes, which I run first. I handle partial failures with cleanup in finally blocks, use locks to prevent concurrent runs, and make API calls with idempotency tokens where supported. Logging says whether each action was skipped or performed. Finally, I test by running twice and asserting that the second run makes no changes, the same principle used with configuration management tools.

Likely follow-ups

- How do idempotency tokens protect against duplicate creates?
- What goes in a dry-run output?

Check current state before acting, use stable names or tags, record progress for resumption, guard destructive steps with dry-run and confirmations, use locks and idempotency tokens, log skipped versus performed actions and test by running twice.

```
def ensure_bucket(s3, name):
    try:
        s3.head_bucket(Bucket=name); return False
    except ClientError:
        s3.create_bucket(Bucket=name); return True
```

```
python tool.py --dry-run
```

Interview tip: Say run it twice; the second run should do nothing.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)

certs.eknathalabs.com

Q26 How do you implement retries with exponential backoff and jitter correctly?

CLI & scripting

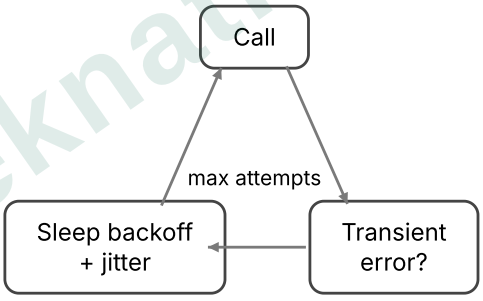
🎙 Spoken answer · ~68 s

Retries help with transient failures such as timeouts, connection resets, throttling responses and server errors, but they must be bounded and polite. The standard approach is exponential backoff: wait a base delay, double it after each failure up to a maximum, and add random jitter, so many clients that failed at the same moment do not retry in lockstep and create a thundering herd. I retry only on errors that are likely transient, such as timeouts, connection errors, HTTP 429 and 5xx, and never on client errors like 400, 401 or 404, which will not improve. Operations must be idempotent, or use idempotency keys, before retrying writes. Respect the Retry-After header when present, set an overall deadline and a maximum number of attempts, and log each retry. Libraries such as tenacity or urllib3's Retry and boto3's built-in retry modes implement this well, and are preferable to hand-written loops. For long-running consumers, add a circuit breaker so a failing dependency is not hammered.

Likely follow-ups

- Why add jitter to backoff?
- Which HTTP errors should not be retried?

🗺 Retry with backoff



Retry only transient errors with exponential backoff, a cap and random jitter, bound attempts and total time, honour Retry-After, make writes idempotent and log retries. Use tenacity, urllib3 Retry or boto3 retry modes rather than hand-rolled loops.

```
from tenacity import retry, stop_after_attempt, wait_exponential_jitter,
retry_if_exception_type
@retry(stop=stop_after_attempt(5), wait=wait_exponential_jitter(initial=1, max=30),
retry=retry_if_exception_type(requests.ConnectionError))
def call(): ...
```

Interview tip: Say bounded retries, jitter and idempotent operations.

certs.eknathalabs.com

📚 **Official docs**

- Microsoft: Cloud design pattern – Retry
- Tenacity

Q27 How do you add safety features such as dry-run, confirmation and progress reporting to operational scripts?

CLI & scripting

🎙 **Spoken answer · ~68 s**

Scripts that change production need guard rails. A dry-run flag runs all the discovery and decision logic but replaces each mutating call with a log line describing what would happen, and I make it the default for risky tools, requiring an explicit apply flag. Confirmation prompts show a summary, such as how many resources will be deleted and in which account and region, and require typing a specific word for destructive actions, with a flag to skip the prompt in CI. I print the target context prominently, because running against the wrong account is a classic mistake. Limits help too: a maximum number of items to change per run, with an override, protects against a bug deleting everything. Progress reporting, with counts and an estimated time through a library like tqdm or periodic log lines, helps operators trust long runs, and a final summary lists successes, failures and skipped items. Everything is logged with timestamps for audit, and exit codes reflect partial failure.

Likely follow-ups

- Why make dry-run the default for risky tools?
- What is a blast-radius limit?

Add dry-run, ideally default for risky tools, confirmation with a printed target context, a cap on items changed per run, progress reporting, a final summary, audit logging and exit codes that reflect partial failure.

```
if args.dry_run:
    log.info('would delete %s', vol_id)
else:
    ec2.delete_volume(VolumeId=vol_id)
```

```
python cleanup.py --max-delete 20 --apply
```

Interview tip: Say show the account and region before you act.

📚 **Start here (official docs)**

- Python documentation

certs.eknathalabs.com

Q28 How do you run commands on remote servers from Python, and what are the options?

CLI & scripting

🎙 Spoken answer · ~69 s

There are several layers. Paramiko is a pure Python SSH library that gives low-level control: connect, run commands, transfer files with SFTP and manage keys. Fabric builds on Paramiko with a friendlier API for running commands and transferring files across many hosts, with connection groups. Netmiko and similar libraries target network devices. Ansible is often the better tool for configuration at scale, and can be invoked through ansible-runner from Python when programmatic control is required. Simpler cases can call the system ssh through subprocess, which reuses the user's config, agent and ControlMaster. Practices I follow: use key-based authentication with an agent, verify host keys instead of auto-adding them, set timeouts, run commands with explicit error checking of exit status, collect stdout and stderr, parallelise with a thread pool for many hosts while limiting concurrency, and avoid putting secrets in command lines. I also decide whether Python is the right place for this logic, since an Ansible playbook may be more maintainable than a custom SSH script.

Likely follow-ups

- Why not auto-add unknown host keys?
- When would you use Ansible instead of Paramiko?

Use Paramiko for low-level SSH and SFTP, Fabric for friendlier multi-host commands, Netmiko for network devices, ssh via subprocess to reuse config, or ansible-runner. Verify host keys, set timeouts, check exit status, limit concurrency and avoid secrets in commands.

```
import paramiko
c = paramiko.SSHClient()
c.load_system_host_keys()
c.connect('10.0.0.5', username='ops', timeout=10)
_, out, err = c.exec_command('uptime')
print(out.read().decode(), out.channel.recv_exit_status())
```

Interview tip: Say verify host keys and always check the exit status.

📚 Official docs

- Python: subprocess
- Python Documentation

certs.eknathalabs.com

Q29 When should you write a tool in Python rather than Bash?

CLI & scripting

🎙 Spoken answer · ~71 s

I use Bash for short glue scripts that mostly chain existing commands, run in a known environment and are only a few dozen lines, since it is the lowest-friction way to invoke programs and pipes. I switch to Python when logic grows: when I need data structures beyond arrays, parsing of JSON, YAML or APIs, error handling beyond set -e, loops with real conditions, concurrency, unit tests, cross-platform behaviour, or the script will be maintained by a team. Bash has well known pitfalls: word splitting and quoting bugs, fragile handling of filenames with spaces, weak error handling, no real data types and limited testing. Python gives libraries, clear exception handling, type hints, packaging and tests, at the cost of needing an interpreter and dependencies, which can be a problem on minimal or locked-down hosts, where the standard library only approach or a single-file zipapp helps. A common rule of thumb is that when a Bash script passes about one hundred lines or needs a dictionary, it is time for Python.

Likely follow-ups

- What are common Bash pitfalls that Python avoids?
- How do you ship Python to minimal hosts?

Use Bash for short command glue; move to Python when you need data structures, API and JSON handling, error handling, concurrency, tests or team maintenance. Mind interpreter availability on minimal hosts.

```
set -euo pipefail
```

```
python3 -m zipapp mytool -o mytool.pyz -p '/usr/bin/env python3'
```

Interview tip: Say a hundred lines or a dictionary means Python.

📚 Official docs

- GNU Bash reference manual
- Python Documentation

certs.eknathalabs.com

Q30 How do you package a Python script as a command-line tool others can install?

CLI & scripting

🎙 Spoken answer · ~68 s

I turn the script into a small package with a pyproject file. The code lives in a package directory with a main function, and the pyproject declares the project name, version, dependencies and a scripts entry point mapping a command name to that function, so installing creates an executable on the user's path. Build with the standard build tool to create a wheel and source distribution, and install with pip, preferably pipx for isolated command-line tools, or uv tool install, which keeps dependencies separate from system Python. For internal distribution, publish to a private package index, or install straight from a Git tag. Include a version flag, a README, a changelog and tests, and pin or bound dependencies sensibly. For environments with no network or a fixed interpreter, a zipapp or a bundled executable made with PyInstaller can be copied as one file, although it is larger and platform specific. Releases are built and published by CI from tags, so installations are reproducible.

Likely follow-ups

- Why use pipx for command-line tools?
- What does a scripts entry point do?

Create a package with pyproject.toml declaring dependencies and a scripts entry point, build a wheel, install with pipx or uv tool, publish to a private index or Git tag, and use zipapp or PyInstaller for hosts without package access. Build from tags in CI.

```
[project.scripts]
mytool = "mytool.cli:main"
```

```
python -m build && pipx install dist/mytool-1.0.0-py3-none-any.whl
```

Interview tip: Say entry points, wheels and pipx for isolation.

📚 Official docs

- Python: argparse
- Python: subprocess
- Python: venv

certs.eknathalabs.com

Q31 How do you call HTTP APIs reliably in Python with requests or httpx?

APIs & cloud SDKs

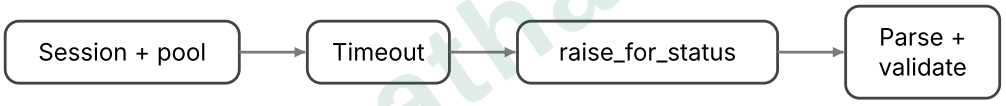
🎙 Spoken answer · ~69 s

I use a library rather than urllib. Requests is the classic, simple choice, and httpx offers a similar API with HTTP/2 and async support. The important habits are: always set a timeout, because by default requests waits forever and a hung call can freeze a whole automation run; reuse a Session or Client so connections are pooled and headers and authentication are set once; call raise_for_status so HTTP errors become exceptions; and configure retries with backoff for transient failures through a mounted adapter or the transport options. I verify TLS certificates, never disabling verification to silence an error, but pointing to the right CA bundle instead. For JSON I use the json method and check content types, and I handle pagination and rate limits. Secrets go into headers from environment or a secrets manager, never logged, and I log request IDs. I wrap the API in a small client class, which centralises the base URL, authentication, error mapping and makes it easy to mock in tests.

Likely follow-ups

- Why must you always set a timeout?
- How do you add retries to a requests session?

🗺 Reliable HTTP call



Use requests or httpx with explicit timeouts, a reused session, raise_for_status, retries with backoff, verified TLS, careful secrets handling and a small client class that centralises auth and error handling for easy mocking.

```
import requests
s = requests.Session()
s.headers['Authorization'] = f'Bearer {token}'
r = s.get(url, timeout=(3.05, 10))
r.raise_for_status()
data = r.json()
```

Interview tip: Say timeouts, sessions and raise_for_status.

📚 Official docs

- Python Documentation
- Requests documentation

certs.eknathalabs.com

Q32 How do you handle pagination, rate limits and large result sets from APIs?

APIs & cloud SDKs

🎙 Spoken answer · ~67 s

Most list APIs return results in pages, with a cursor, a next-page token, a page number or a link header, so a client must loop until no more pages. I wrap this in a generator that yields items, hiding pagination from the caller and keeping memory low, which works nicely with comprehension and processing pipelines. For rate limits, I read the response headers or the 429 status, honour Retry-After, and use backoff with jitter. For polite throughput I limit concurrency, use a token bucket or a simple sleep between calls, and cache stable data to reduce calls. When a collection changes during iteration, cursor-based pagination is safer than offset-based, which can skip or duplicate items. Where the API supports filters, fields selection or bulk endpoints, I use them to reduce calls, and where it supports webhooks or change feeds, I prefer them over polling. I log counts and page numbers for debugging, and fail clearly if a page is malformed.

Likely follow-ups

- Why is cursor pagination safer than offset?
- How do you implement a token bucket limiter?

Wrap pagination in a generator following cursors or links, honour 429 and Retry-After with backoff, limit concurrency, prefer cursor over offset pagination, use filters and bulk endpoints, and prefer webhooks to polling.

```
def items(url):
    while url:
        r = s.get(url, timeout=10); r.raise_for_status()
        j = r.json()
        yield from j['items']
        url = j.get('next')
```

Interview tip: Say a generator hides pagination from callers.

📚 Official docs

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling
- REST API pagination (Google AIP-158)

certs.eknathalabs.com

Q33 How do you use boto3 effectively: clients, resources, paginators and waiters?

APIs & cloud SDKs

🎙 Spoken answer · ~68 s

Boto3 is the AWS SDK for Python. A client maps one-to-one to service API calls and returns dictionaries, and is the recommended low-level interface, while resources offer an object-oriented layer that is in maintenance mode for many services. Many list calls return truncated results, so I use paginators, which iterate across pages automatically, rather than a single call that silently returns only the first page, a common bug. Waiters poll until a resource reaches a state, such as an instance running or a snapshot completed, with configurable delay and attempts, so I avoid hand-written sleep loops. Credentials come from the default chain: environment, shared config, SSO or an instance or workload role, never hard-coded, and I use assume-role sessions for cross-account work. Configure retries and timeouts with botocore Config, handle ClientError by examining the error code, reuse sessions and clients, create one per thread, and use filters server-side to reduce data. For testing, I use moto or stubber to avoid hitting real AWS.

Likely follow-ups

- Why use a paginator instead of a single call?
- How do you assume a role across accounts?

🗺 Paginated listing



Use clients with paginators for complete listings and waiters for state changes, credentials from the default chain or assumed roles, botocore Config for retries and timeouts, ClientError code handling, one client per thread and moto for tests.

```
import boto3
ec2 = boto3.client('ec2')
for page in ec2.get_paginator('describe_instances').paginate(Filters=
[{'Name':'tag:env','Values':['prod']}]):
    for r in page['Reservations']: ...
```

```
ec2.get_waiter('instance_running').wait(InstanceIds=[iid])
```

Interview tip: Say paginators and waiters instead of loops and sleeps.

📚 Official docs

- REST API pagination (Google AIP-158)
- Python: argparse
- Boto3 documentation

certs.eknathalabs.com

Q34 How do you use the Kubernetes Python client, and when would you use it instead of kubectl?

APIs & cloud SDKs

🎙 Spoken answer · ~66 s

The official kubernetes client library wraps the API with typed objects. You load configuration with load_kube_config for local use or load_incluster_config inside a pod, create an API object such as CoreV1Api or AppsV1Api, and call methods to list, create, patch and delete resources, or use the watch utility to stream events. It also supports dynamic clients for custom resources. I use it for programmatic tasks that go beyond a shell script: reporting across namespaces and clusters, cleaning up resources by rules, controllers and operators written with frameworks like kopf, or tools that need structured data and error handling. For simple one-off actions, kubectl is quicker and clearer. Practices: use service accounts with least-privilege RBAC when running in-cluster, set timeouts and handle ApiException status codes such as 404 and 409, page large lists with limit and continue tokens, avoid polling in favour of watches, and be careful with resourceVersion when updating, using patch instead of full replace to avoid conflicts.

Likely follow-ups

- How do you authenticate inside a pod?
- Why prefer patch to replace?

The kubernetes client loads kubeconfig or in-cluster config and exposes typed APIs, watches and dynamic clients. Use it for reports, cleanup, controllers and structured automation; kubectl for one-offs. Use least-privilege RBAC, handle ApiException, page lists, watch rather than poll and patch rather than replace.

```
from kubernetes import client, config
config.load_kube_config()
v1 = client.CoreV1Api()
for p in v1.list_pod_for_all_namespaces(limit=100).items:
    print(p.metadata.namespace, p.metadata.name, p.status.phase)
```

Interview tip: Say in-cluster config and least-privilege RBAC.

📚 Official docs

- Kubernetes Python client
- Python: argparse
- Python Documentation

certs.eknathalabs.com

Q35 How do you implement authentication for API clients, including tokens, OAuth and refresh?

APIs & cloud SDKs

🎙 Spoken answer · ~71 s

The method depends on the API. Simple services use a static API key or bearer token in a header, loaded from environment or a secrets manager. Many platforms use OAuth 2.0: for machine-to-machine automation the client credentials flow exchanges a client ID and secret for a short-lived access token, which the client caches and refreshes before it expires, typically by tracking the expiry time with a safety margin and refreshing on a 401 response as a fallback. Cloud providers use signed requests or short-lived credentials from roles, handled by their SDKs. In a client class I centralise this: a method that returns a valid token, thread-safe refresh so that concurrent callers do not all refresh at once, and an authenticated session that adds the header. Secrets are never logged or committed, and I scope tokens to minimum permissions. For mutual TLS, a client certificate is configured on the session. In tests, I mock the token endpoint, and I make token and secret rotation seamless by reading from the source on refresh.

Likely follow-ups

- Why refresh before expiry, not only on 401?
- How do you make token refresh thread-safe?

Use API keys from the environment or secrets manager, or OAuth client credentials with cached short-lived tokens refreshed before expiry, with a 401 fallback and a lock to avoid concurrent refreshes. Scope tokens narrowly and never log them.

```
r = requests.post(token_url, data={'grant_type':'client_credentials'}, auth=(cid, secret),
timeout=10)
token = r.json()['access_token']
expires = time.time() + r.json()['expires_in'] - 30
```

Interview tip: Say cache tokens and refresh with a margin.

📚 Official docs

- OAuth 2.0 / OIDC overview (OpenID)
- Python: argparse
- Requests documentation

certs.eknathalabs.com

Q36 How would you build a small webhook receiver or internal API with Flask or FastAPI?

APIs & cloud SDKs

🎙 Spoken answer · ~68 s

For a webhook receiver, such as one handling alerts or repository events, I use FastAPI or Flask to expose an endpoint that accepts a POST, verifies authenticity, validates the payload, acknowledges quickly and processes asynchronously. Verification means checking a signature header, an HMAC of the body with a shared secret compared in constant time, or a bearer token, and rejecting anything else, since the endpoint is exposed. FastAPI uses pydantic models to validate the body automatically and generates documentation, and supports async handlers. Long work is handed to a background task or a queue and a worker, returning a response within a few seconds so senders do not time out and retry. I make handlers idempotent using delivery IDs, because webhooks can be delivered more than once. Production concerns: run under a proper server such as uvicorn or gunicorn, add health and readiness endpoints, structured logging, metrics, request size limits, TLS at the ingress, rate limiting and tests with the framework's test client.

Likely follow-ups

- Why acknowledge the webhook quickly?
- How do you verify an HMAC signature safely?

Expose a POST endpoint, verify an HMAC signature in constant time, validate the body with pydantic, respond fast and process in the background or a queue, make handlers idempotent by delivery ID, and add health checks, logging, limits and tests.

```
import hmac, hashlib
ok = hmac.compare_digest(sig, 'sha256=' + hmac.new(secret, body,
hashlib.sha256).hexdigest())
```

```
uvicorn app:app --host 0.0.0.0 --port 8080
```

Interview tip: Say verify the signature, ack fast, process async, be idempotent.

📚 Official docs

- Requests documentation

certs.eknathalabs.com

Q37 How do you automate GitHub or GitLab with their APIs, for example for repository hygiene or release automation?

APIs & cloud SDKs

🎙 Spoken answer · ~65 s

Both platforms have REST and GraphQL APIs and official or community Python libraries such as PyGithub and python-gitlab, which wrap authentication, pagination and common objects. Typical automation includes enforcing branch protection and repository settings across an organisation, creating repositories from templates, opening pull requests for dependency or configuration updates across many repositories, auditing access and stale branches, labelling and triaging issues, and creating releases and tags from a pipeline. Authentication should use a GitHub App installation token or a fine-grained token with minimal permissions rather than a personal token, which is short-lived and auditable. I handle rate limits by reading the rate-limit headers and backing off, use conditional requests with ETags to save quota, and paginate everything. Changes are made through pull requests where possible so humans can review, and bulk operations run in dry-run first with a report. Webhooks trigger event-driven automation. Sensitive tokens live in the CI secret store, and results are logged for audit.

Likely follow-ups

- Why prefer a GitHub App over a personal token?
- How do you protect against hitting rate limits?

Use PyGithub or python-gitlab with GitHub App or fine-grained tokens, automate settings, pull requests, audits and releases, respect rate limits with ETags and backoff, paginate, run bulk changes in dry-run first and prefer pull requests for review.

```
from github import Github
g = Github(auth=auth)
for repo in g.get_organization('acme').get_repos():
    repo.get_branch('main').edit_protection(required_approving_review_count=1)
```

Interview tip: Say scoped app tokens and dry-run bulk changes.

📚 Official docs

- Requests documentation

certs.eknathalabs.com

Q38 How do you integrate Python automation with Slack, PagerDuty or ticketing systems?

APIs & cloud SDKs

🎙 Spoken answer · ~68 s

Most of these offer REST APIs and webhooks. For notifications, a Slack incoming webhook or the Web API posts messages to channels, and I format them with blocks, include links to runbooks, dashboards and the job that produced them, and keep them concise to avoid noise. For incidents, the PagerDuty Events API triggers, acknowledges and resolves alerts using a deduplication key, so repeated events update a single incident instead of creating many. For tickets, Jira, ServiceNow and others have APIs to create and update issues, which automation can use to record findings or track remediation. Interactive flows use Slack slash commands or buttons, calling back to a service that must verify the Slack signature and respond within a few seconds. Good practice: store tokens in a secrets manager, handle failures of the notification system without breaking the main job, rate limit messages, redact sensitive data in messages, use threads to group related updates, and make the integration idempotent so retries do not duplicate tickets.

Likely follow-ups

- What is a dedup key in PagerDuty?
- How do you verify a Slack request?

Use Slack webhooks or the Web API for concise linked messages, the PagerDuty Events API with a dedup key, ticket APIs for findings and Slack signature verification for interactive commands. Keep tokens secret, tolerate notification failures and avoid duplicates.

```
requests.post(slack_webhook, json={'text': f':warning: backup failed on {host}
<{runbook}|runbook>'}, timeout=10)
```

```
requests.post('https://events.pagerduty.com/v2/enqueue', json={'routing_key': key,
'event_action': 'trigger', 'dedup_key': f'backup-{host}', 'payload': {...}})
```

Interview tip: Say dedup keys prevent duplicate incidents.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q39 How do you validate data from external APIs and files using pydantic or JSON Schema?

APIs & cloud SDKs

🎙 Spoken answer · ~68 s

Data from outside the process cannot be trusted to match what the code expects, so I validate at the boundary. With pydantic, I declare a model describing the expected structure and types, and construct it from the raw dictionary, which converts compatible types, applies defaults, enforces constraints such as ranges, lengths and patterns, and raises a ValidationError listing every problem with its location. Custom validators handle cross-field rules, such as an end time after a start time. With JSON Schema, a language-neutral description validated by the jsonschema library, I can share the same schema with other systems and documentation, and it is the natural choice for configuration files and OpenAPI-defined payloads. Either way, I fail early with clear errors, log what failed, and decide per case whether to reject the whole input or skip invalid records and report them. Strict modes avoid surprising coercion. Models also serve as documentation and give editors completion, and I keep them versioned along with the API they represent.

Likely follow-ups

- When would you choose JSON Schema over pydantic?
- How do you handle partially invalid records?

Validate at the boundary with pydantic models or JSON Schema: types, constraints, defaults and cross-field rules, failing early with clear errors. Decide whether to reject all input or skip and report bad records, and prefer strict modes to avoid surprising coercion.

```
from pydantic import BaseModel, Field
class Instance(BaseModel):
    id: str
    cpu: int = Field(ge=1, le=128)
Instance.model_validate(raw)
```

```
jsonschema.validate(instance=cfg, schema=schema)
```

Interview tip: Say validate once at the edge so the rest of the code can trust types.

📚 Official docs

- Python: pathlib
- Requests documentation
- Pydantic documentation

certs.eknathalabs.com

Q40 How do you choose between threading, multiprocessing and asyncio?

Concurrency & speed

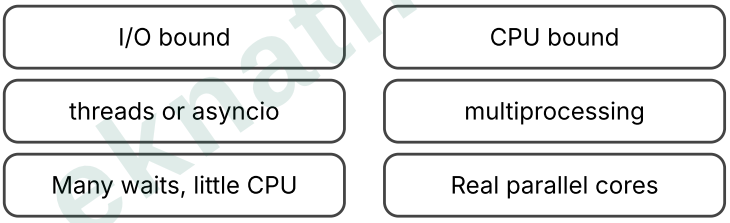
🎙 Spoken answer · ~65 s

I match the tool to the bottleneck. If work is I/O-bound, waiting on networks, APIs, SSH or disks, then threads or asyncio give concurrency, since the GIL is released during waiting. Threads are simplest to adopt with existing blocking libraries, using a ThreadPoolExecutor, and handle moderate concurrency of tens to hundreds of tasks. Asyncio uses a single thread with cooperative scheduling and scales to thousands of concurrent connections with low overhead, but needs async-compatible libraries such as aiohttp or httpx, and any blocking call stalls the event loop. If work is CPU-bound, parsing, compressing or computing, then multiprocessing or ProcessPoolExecutor uses multiple cores, at the cost of process start-up, serialisation of arguments and results, and memory. I also consider simplicity: for a script that calls fifty APIs, a thread pool is usually enough. Whichever I use, I bound concurrency with a limit, handle errors per task, and avoid shared mutable state without locks or queues.

Likely follow-ups

- What blocks the asyncio event loop?
- Why is process communication expensive?

🗺 Choosing concurrency



Threads or asyncio for I/O-bound work, multiprocessing for CPU-bound work. Threads suit existing blocking libraries, asyncio scales to thousands of connections but needs async libraries. Bound concurrency, handle per-task errors and avoid shared mutable state.

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=20) as ex:
    results = list(ex.map(check_host, hosts))
```

Interview tip: Say identify whether it is waiting or computing first.

📚 Official docs

- Python: asyncio
- Python: concurrent.futures

corts.eknathalabs.com

Q41 How does asyncio work, and how would you use it to call many APIs concurrently?

Concurrency & speed

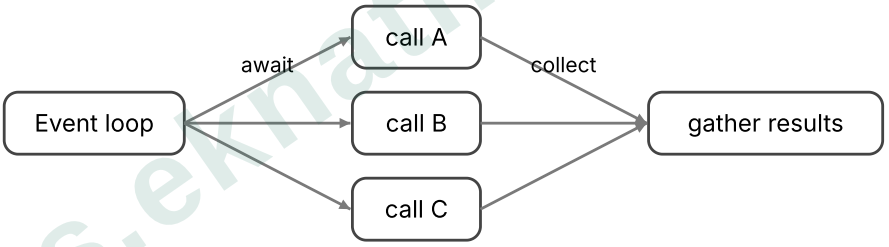
🎙 Spoken answer · ~63 s

Asyncio runs an event loop in one thread. Functions defined with async def are coroutines, and the await keyword hands control back to the loop while waiting for an operation, such as a network response, so other coroutines can progress. To run many calls concurrently, I create tasks, or use asyncio.gather or a TaskGroup, and use an async HTTP client like httpx or aiohttp with a shared client session. Concurrency must be bounded with a semaphore, since unlimited tasks can exhaust sockets, memory or rate limits. Timeouts are applied with asyncio.timeout or wait_for, errors are handled per task, and gather with return_exceptions collects failures without cancelling everything, whereas TaskGroup cancels siblings on error. The main pitfalls are calling blocking functions inside coroutines, which freezes the loop, solved by moving them to a thread with asyncio.to_thread, forgetting to await a coroutine, and mixing sync and async code carelessly. The entry point is asyncio.run.

Likely follow-ups

- How do you limit concurrent requests?
- What happens if you call time.sleep in a coroutine?

🗺 Concurrent API calls



Asyncio uses one event loop with coroutines that await I/O. Create tasks with gather or TaskGroup, use an async client, bound concurrency with a semaphore, apply timeouts, collect errors per task and move blocking calls to threads with to_thread.

```
import asyncio, httpx
async def fetch(c, u, sem):
    async with sem:
        return (await c.get(u, timeout=10)).status_code
async def main():
    sem = asyncio.Semaphore(20)
    async with httpx.AsyncClient() as c:
        print(await asyncio.gather(*(fetch(c, u, sem) for u in urls)))
asyncio.run(main())
```

Interview tip: Say a semaphore bounds concurrency; never block the loop.

certs.eknathalabs.com

📚 **Official docs**

- Python: asyncio
- Python: concurrent.futures
- Requests documentation

Q42 **How do you use ThreadPoolExecutor to run tasks in parallel safely, for example checks across many hosts?**

Concurrency & speed

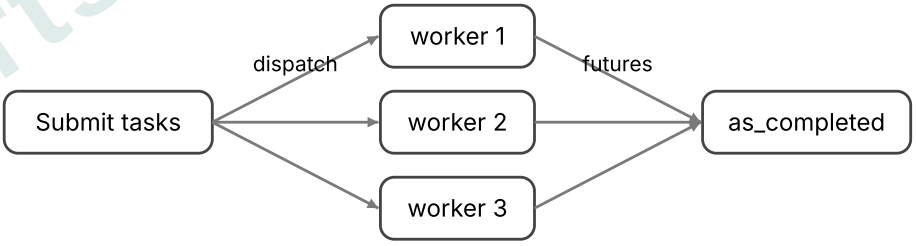
🎙 **Spoken answer · ~70 s**

The concurrent.futures module makes this simple. I create a ThreadPoolExecutor with a limited number of workers, submit one task per host or use map, and collect results as they complete with as_completed, which lets me report progress and handle failures individually. Each future raises its exception when result is called, so I wrap the call and record failures without stopping the batch. Choosing the worker count matters: enough to overlap waiting, but low enough not to overwhelm the targets, a bastion or an API's rate limit. Tasks should be independent and return data rather than mutating shared structures, and if shared state is unavoidable, protect it with a lock or use queue-safe structures. Per-task timeouts must be implemented inside the task, for example on socket or HTTP calls, because a thread cannot be killed from outside. Using the executor as a context manager waits for completion and cleans up. I log which host each result belongs to and return a summary with an exit code reflecting failures.

Likely follow-ups

- Why can a thread not be force-killed?
- How do you report progress as tasks finish?

🗺 **Thread pool**



Create a bounded ThreadPoolExecutor, submit one task per host, collect with as_completed, record exceptions per future, keep tasks independent, protect shared state, implement timeouts inside tasks and summarise results with a meaningful exit code.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
with ThreadPoolExecutor(max_workers=16) as ex:
    futs = {ex.submit(check, h): h for h in hosts}
    for f in as_completed(futs):
        try: print(futs[f], f.result())
        except Exception as e: print(futs[f], 'FAILED', e)
```

certs.eknathalabs.com

Interview tip: Say bound the workers and handle each future's exception.

📚 **Official docs**

- Python: concurrent.futures

Q43 How do you profile a slow Python script and find the bottleneck?

Concurrency & speed

🎙 **Spoken answer · ~66 s**

I measure before changing anything. For a quick view, cProfile with the sort option shows time per function, and visualisers such as snakeviz or flame graphs make it easier to read. For line-level detail, line_profiler shows time per line in a chosen function. Timing blocks with time.perf_counter or a context manager helps isolate phases like network, parsing and writing. For running processes, py-spy attaches without modifying code and produces flame graphs or dumps of what each thread is doing, which is ideal in production. Memory is checked with tracemalloc or memray. Typical findings are accidental quadratic algorithms such as repeated membership checks on lists, which become sets, work repeated inside loops, per-item network calls that could be batched, excessive logging, regex recompilation and serial calls that could be concurrent. After optimising, I rerun the profile to verify the improvement and add a benchmark if the performance is important. Premature optimisation is avoided: focus on the largest contributor first.

Likely follow-ups

- What does py-spy let you do in production?
- Why do membership tests on a list become slow?

Profile first with cProfile and snakeviz, line_profiler and perf_counter timing, and use py-spy on running processes. Look for quadratic algorithms, repeated work, per-item calls and serial I/O, optimise the biggest contributor and re-measure.

```
python -m cProfile -o out.prof tool.py && snakeviz out.prof
```

```
py-spy top --pid 4321
```

```
py-spy record -o flame.svg --pid 4321 --duration 30
```

Interview tip: Say measure first, fix the biggest contributor.

📚 **Official docs**

- Python: pathlib
- py-spy
- Python Documentation

certs.eknathalabs.com

Q44 How do you diagnose and fix memory problems in Python scripts and services? Concurrency & speed

🎙 Spoken answer · ~70 s

Symptoms are growing resident memory, out-of-memory kills or slow garbage collection. First, determine whether memory grows steadily, indicating a leak or unbounded cache, or spikes from loading a large dataset. Tools: tracemalloc snapshots compare allocations between two points and show which lines allocated the growth, memray gives detailed profiles including native allocations, and objgraph helps find reference chains that keep objects alive. Typical causes: reading whole files or API result sets into lists instead of streaming, unbounded caches or dictionaries keyed by ever-growing values, global lists collecting results, references held by closures, logging handlers or callbacks, and circular references with finalisers. Fixes: stream with generators and chunks, use bounded caches such as functools.lru_cache with maxsize or a TTL cache, process in batches, use compact types such as arrays, slots in classes, or numpy and pandas dtypes, and release references. In containers, I set memory limits and watch the working set, and for long-running services, I monitor memory metrics and restart policies as a safety net, not as the fix.

Likely follow-ups

- How do you compare two tracemalloc snapshots?
- Why are unbounded caches dangerous?

Distinguish steady growth from spikes, then use tracemalloc, memray and objgraph to find allocations and retained references. Common causes are loading everything into memory, unbounded caches and held references; fix with streaming, bounded caches, batching and compact types.

```
import tracemalloc
tracemalloc.start()
# ... run work ...
for s in tracemalloc.take_snapshot().statistics('lineno')[:10]: print(s)
```

```
python -m memray run tool.py
```

Interview tip: Say find what is retaining the objects, not just who allocated them.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q45 How do you design a robust Python client library for an internal API, and how do you make it testable?

APIs & cloud SDKs

🎙 Spoken answer · ~67 s

I wrap the API in a small class so callers do not deal with URLs, headers and parsing. The constructor takes the base URL, credentials and a session, which makes it easy to inject a fake in tests. Methods map to API operations with clear names and typed arguments and return typed objects, such as dataclasses or pydantic models, not raw dictionaries, so errors appear at the boundary. Common concerns are centralised: authentication, timeouts, retries with backoff, pagination helpers, and translation of HTTP failures into a small hierarchy of exceptions, for example NotFound, RateLimited and ServerError, so callers can handle cases without inspecting status codes. Logging includes request IDs but excludes secrets. For testing, I use the responses or respx library to mock HTTP at the transport level, or a fake server fixture, and contract tests against a staging endpoint. I version the library alongside the API, document with docstrings and examples, and keep a changelog, so consumers can upgrade safely.

Likely follow-ups

- Why return typed objects instead of dictionaries?
- How do you mock HTTP calls in tests?

Wrap the API in a class with injected session and credentials, typed methods and models, centralised auth, retries, pagination and a small exception hierarchy. Test with responses or respx and contract tests, and version and document the library.

```
class Client:
    def __init__(self, base, token, session=None):
        self.s = session or requests.Session()
        self.s.headers['Authorization'] = f'Bearer {token}'
        self.base = base
```

```
import responses
@responses.activate
def test_get(): ...
```

Interview tip: Say one exception hierarchy and an injectable session.

📚 Official docs

- Python: argparse
- Python Documentation
- pytest documentation

certs.eknathalabs.com

Q46 How do you use queues and worker patterns for producer-consumer workloads in Python?

Concurrency & speed

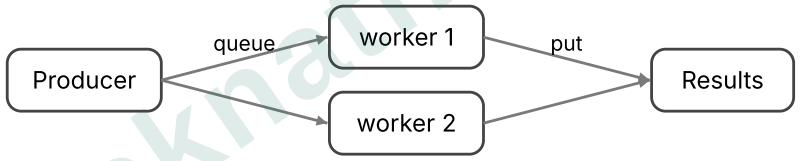
🎙 Spoken answer · ~68 s

When work arrives continuously, or a stage produces items faster than the next can consume, a queue decouples them. The standard library queue module provides thread-safe queues, and multiprocessing and asyncio have their own versions. A producer puts items on the queue, several workers take items, process them and mark them done, and a sentinel value or a join call signals completion. A bounded queue size applies back-pressure: when full, the producer blocks, preventing memory growth if consumers are slow. Workers should catch exceptions per item so one failure does not kill the worker, and failed items can go to a retry or dead-letter queue. Graceful shutdown stops accepting new work, drains the queue and joins workers, and handles SIGTERM. For durable, distributed workloads across machines, an external broker with Celery, RQ or a cloud queue such as SQS is better, since in-process queues vanish on a crash. I add metrics on queue depth and processing time, which reveal bottlenecks and sizing needs.

Likely follow-ups

- What does a bounded queue give you?
- When do you need an external broker?

🗺 Producer / consumer



Use a queue between producers and a pool of workers, bound its size for back-pressure, handle errors per item with retry or dead-letter paths, shut down gracefully and use an external broker such as Celery, RQ or SQS when work must be durable or distributed.

```
import queue, threading
q = queue.Queue(maxsize=100)
def worker():
    while True:
        item = q.get()
        try: process(item)
        finally: q.task_done()
```

```
for _ in range(8): threading.Thread(target=worker, daemon=True).start()
```

Interview tip: Say bound the queue to get back-pressure.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q47 How do you avoid race conditions and deadlocks in concurrent Python code?

Concurrency & speed

🎙 Spoken answer · ~71 s

A race condition occurs when the result depends on the timing of threads accessing shared data, for example two threads incrementing a counter, because read-modify-write sequences are not atomic even with the GIL. Prevention starts with design: avoid shared mutable state, pass data through queues, and have each worker return results that the main thread combines. When sharing is needed, protect critical sections with a lock used as a context manager, keep them small, and use thread-safe structures. Deadlocks happen when threads wait for each other's locks, typically by acquiring several locks in different orders; the remedy is a consistent global lock ordering, using timeouts on acquire, or reducing to a single lock. Related problems are lock contention, which hurts performance, and forgetting to release locks on exceptions, avoided by with. For processes, shared memory needs explicit synchronisation, and file-based locks protect external resources. I also test concurrent code under load with randomised delays and use tools like thread dumps with faulthandler or py-spy to see where threads are stuck.

Likely follow-ups

- Why is counter += 1 not thread-safe?
- How do you find where a deadlocked process is stuck?

Avoid shared mutable state by using queues and returning results, protect unavoidable sharing with small lock-guarded sections, use consistent lock ordering or timeouts to avoid deadlocks, and inspect stuck processes with faulthandler or py-spy dump.

```
lock = threading.Lock()
with lock:
    counter += 1
```

```
py-spy dump --pid 4321
```

Interview tip: Say prefer queues over shared state.

📚 Official docs

- Python: concurrent.futures
- Python Documentation

certs.eknathalabs.com

Q48 How does caching with functools.lru_cache and TTL caches speed up scripts, and what are the pitfalls?

Concurrency & speed

🎙 Spoken answer · ~67 s

Caching stores the results of expensive calls so repeated calls with the same input return immediately. The functools.lru_cache decorator memoises a function by its arguments, which must be hashable, with a maxsize that bounds memory by evicting the least recently used entries, and cache_info shows hits and misses. The cache decorator is an unbounded variant. For data that goes stale, such as API results, a time-to-live is needed, available in libraries like cachetools or a manual timestamp, and for sharing across processes or runs, a disk or external cache like Redis. Pitfalls: caching functions with side effects or results that depend on hidden state, so stale or wrong values are returned; unbounded caches growing without limit; caching mutable return values that callers then modify, corrupting the cached object; lru_cache on instance methods, which keeps the instance alive through the cache; and forgetting invalidation. Cache only pure, expensive calls, set sizes and lifetimes deliberately, and expose a way to clear it.

Likely follow-ups

- Why must arguments be hashable?
- Why is lru_cache on a method risky?

lru_cache memoises pure functions by hashable arguments with a bounded size; use TTL caches for changing data and shared caches across processes. Beware side effects, unbounded growth, mutable results, caching methods and missing invalidation.

```
from functools import lru_cache
@lru_cache(maxsize=256)
def account_name(account_id): ...
```

```
account_name.cache_info()
```

```
from cachetools import TTLCache; cache = TTLCache(maxsize=100, ttl=300)
```

Interview tip: Say bounded size and an expiry for changing data.

📚 Official docs

- AWS Builders' Library: Caching challenges and strategies

certs.eknathalabs.com

Q49 How would you speed up processing of a large dataset in Python without rewriting everything?

Concurrency & speed

🎙 Spoken answer · ~67 s

I look for the biggest wins first. Algorithmic improvements usually dominate: replace repeated list searches with sets or dictionaries, avoid nested loops over large collections, sort and merge instead of comparing everything pairwise, and move invariant work out of loops. Use built-in functions and the standard library, which are implemented in C, such as sum, sorted, collections.Counter and itertools. For tabular numeric data, vectorised operations in pandas or numpy, or an analytical engine such as DuckDB or Polars, can be orders of magnitude faster than Python loops and can process files larger than memory with lazy evaluation. For CPU-bound parts that must stay in Python, split into chunks and use a process pool, being careful about serialisation costs. For hot numerical loops, Numba, Cython or rewriting in a compiled language through an extension can help. I/O matters too: read in large chunks, use efficient formats such as Parquet instead of CSV, and avoid per-row network calls. Measure after each change.

Likely follow-ups

- When would DuckDB beat pandas?
- What costs appear when using multiprocessing?

First fix algorithms with sets, dictionaries and avoiding nested loops, then use C-backed built-ins, vectorised pandas, numpy, Polars or DuckDB, use process pools for CPU-bound chunks, choose efficient formats like Parquet and measure after each change.

```
import duckdb
duckdb.sql("select status, count(*) from 'access.parquet' group by 1").show()
```

```
pd.read_csv('big.csv', chunksize=100_000)
```

Interview tip: Say algorithm first, then vectorise, then parallelise.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q50 How do you structure tests with pytest, and what are fixtures and parametrisation?

Testing & quality

🎙 Spoken answer · ~68 s

Pytest discovers functions named test_ in files named test_, and uses plain assert statements with detailed failure output. Fixtures provide reusable setup and teardown: a function decorated with pytest.fixture returns or yields a resource, such as a temporary directory, a fake client or a database, and tests request it by naming it as an argument. Scope controls how often it is created: per function, module or session. Fixtures can depend on other fixtures, and conftest files share them across a directory. Parametrisation runs the same test with many inputs through the parametrize marker, giving each case its own test result, which is ideal for edge cases such as parsing different log formats. Other useful features: tmp_path for temporary files, monkeypatch to set environment variables or replace functions, capsys and caplog to check output and logs, markers to group slow or integration tests, and plugins like pytest-cov and pytest-xdist. I keep tests small, deterministic and independent, naming them after the behaviour they verify.

Likely follow-ups

- What does fixture scope control?
- How does monkeypatch differ from mock?

🗺 pytest flow



Pytest finds test_ functions and uses plain asserts. Fixtures provide reusable setup and cleanup at chosen scopes, parametrize runs cases from data, and tmp_path, monkeypatch, capsys and caplog help with files, environment and output. Keep tests small and deterministic.

```
import pytest
@pytest.mark.parametrize('line,status', [('GET / 200', 200), ('GET /x 500', 500)])
def test_status(line, status):
    assert parse(line).status == status
```

```
pytest -q -x --maxfail=3
```

Interview tip: Say fixtures for setup, parametrize for cases.

📚 Official docs

- pytest documentation
- pytest documentation

certs.eknathalabs.com

Q51 How do you mock cloud and API calls in tests, for example with unittest.mock, moto or responses?

Testing & quality

🎙 Spoken answer · ~65 s

Tests should not call real cloud accounts or APIs: they are slow, flaky, cost money and can change real resources. I choose the mocking level to fit. For AWS with boto3, the moto library simulates many services in memory, so code can create buckets, instances and queues against a fake backend, and assertions check the resulting state, which tests behaviour rather than call sequences. For HTTP APIs, responses or respx intercept requests at the transport level and return canned responses, including errors and rate limits, so I can test retry logic. For arbitrary dependencies, unittest.mock's patch replaces objects where they are used, and MagicMock records calls so I can assert on arguments, but over-mocking couples tests to implementation details. I design code with dependency injection, passing the client in, which makes substitution simple. I also keep a small set of integration tests against a real sandbox account, run less frequently, to catch differences between mocks and reality.

Likely follow-ups

- Where should you patch: where defined or where used?
- Why add a few real integration tests?

🗺 Mocking AWS



Use moto for AWS, responses or respx for HTTP, and unittest.mock for other dependencies, patching where used. Inject clients for easy substitution, assert on resulting state rather than call sequences, and keep a small set of integration tests against a sandbox.

```
from moto import mock_aws
@mock_aws
def test_bucket():
    s3 = boto3.client('s3', region_name='us-east-1')
    s3.create_bucket(Bucket='b')
    assert ensure_bucket(s3, 'b') is False
```

Interview tip: Say test behaviour against a fake backend, not call counts.

📚 Official docs

- pytest documentation
- Requests documentation

certs.eknathalabs.com

Q52 How do you test command-line tools and scripts?

Testing & quality

🎙 **Spoken answer · ~70 s**

I separate logic from the command line layer: the main behaviour lives in importable functions that take parameters and return results, tested directly, while the CLI module only parses arguments and calls them. For the CLI layer, frameworks give test helpers: click and typer provide a CliRunner that invokes a command in-process and captures output and exit code, so I assert on both. With argparse, I call main with a list of arguments. For scripts that run subprocesses or touch the file system, I use tmp_path and monkeypatch to redirect paths and environment variables, and mock the subprocess boundary or use a fake executable placed on PATH. I test the unhappy paths: invalid input, missing files, permission errors, and that the exit code is nonzero with a helpful message. Dry-run modes are tested to verify nothing is changed. A small number of end-to-end tests run the installed command in a clean environment, which also proves packaging works. Output formats such as JSON are checked by parsing, not string matching.

Likely follow-ups

- Why separate logic from argument parsing?
- How do you assert exit codes in tests?

Keep logic in importable functions and test them directly, test the CLI layer with CliRunner or main(argv) checking output and exit codes, use tmp_path and monkeypatch for files and environment, test failure paths and dry-run, and add a few end-to-end installs.

```
from click.testing import CliRunner
def test_cli():
    r = CliRunner().invoke(app, ['scale', 'web', '--dry-run'])
    assert r.exit_code == 0 and 'would scale' in r.output
```

Interview tip: Say test the exit code and the failure paths.

📚 **Official docs**

- Python: argparse
- Python: subprocess
- pytest documentation

certs.eknathalabs.com

Q53 What do you measure with code coverage, and how do you use it sensibly in CI?

Testing & quality

🎙 Spoken answer · ~69 s

Coverage measures which lines or branches executed during tests, using coverage.py through pytest-cov. It shows untested code, but not whether tests assert anything meaningful, so a high number can be misleading, and I treat it as a guide for finding gaps, not as a goal in itself. Branch coverage is more informative than line coverage, since it checks both outcomes of conditions. In CI I generate a report and enforce a reasonable minimum threshold, such as the current level, preventing regression, with a rule that new code should be well covered, using diff coverage tools to focus on changes. I exclude generated code and unreachable defensive lines with comments, but sparingly. The report is published as an artifact and a pull request comment. For critical paths such as deletion logic and parsing, I aim for thorough tests including error cases, and consider mutation testing to check that tests catch changes. The aim is confidence, not a percentage, so I review uncovered lines in code review.

Likely follow-ups

- Why is branch coverage better than line coverage?
- Can 100 percent coverage still miss bugs?

Coverage shows which code ran, not whether tests assert meaningful behaviour. Use branch coverage, a ratcheting threshold and diff coverage for new code, review uncovered lines, test critical paths thoroughly and avoid treating the percentage as the goal.

```
pytest --cov=mytool --cov-branch --cov-report=term-missing --cov-fail-under=85
```

```
diff-cover coverage.xml --compare-branch=origin/main
```

Interview tip: Say coverage finds gaps; it does not prove quality.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q54 How do you use linters, formatters and type checkers such as ruff, black and mypy in a Python project?

Testing & quality

🎙 Spoken answer · ~66 s

They automate consistency and catch bugs before runtime. A formatter such as black or ruff format applies one style, ending debates and keeping diffs small. A linter such as ruff, which replaces many older tools like flake8, isort and pyupgrade, flags unused imports and variables, undefined names, risky patterns such as bare except, mutable defaults, subprocess with shell and insecure functions through its security rules, and can auto-fix many issues quickly. A type checker such as mypy or pyright analyses annotations to find mismatched types, missing returns and None handling errors, which are very common in scripts dealing with optional API fields. I configure everything in pyproject, run them in a pre-commit hook for instant feedback and again in CI as required checks, so nothing depends on a developer's setup. Adoption in an existing code base is gradual: enable a basic rule set, fix or baseline existing issues, then tighten. Security-focused additions include bandit rules and dependency auditing.

Likely follow-ups

- Why run the same checks in pre-commit and CI?
- How do you adopt mypy in an untyped code base?

Use a formatter for consistent style, ruff for linting and security rules with auto-fixes, and mypy or pyright for type errors. Configure in pyproject, run in pre-commit and CI, and adopt gradually in existing code by baselining and tightening.

```
ruff check --fix . && ruff format .
```

```
mypy --strict src/
```

```
pre-commit run --all-files
```

Interview tip: Say pre-commit for speed, CI for enforcement.

📚 Official docs

- Python Documentation
- Ruff documentation

certs.eknathalabs.com

Q55 How do you test code that depends on time, randomness or external state?

Testing & quality

🎙 Spoken answer · ~68 s

Such code is hard to test because results change between runs, so I make the dependencies explicit and controllable. For time, I avoid calling datetime.now deep inside functions; instead, I pass a clock or the current time as a parameter, or wrap it in a small function that tests can replace, or use a library such as freezegun or time-machine to freeze and advance time, which lets me test expiry, scheduling and retry delays instantly, without sleeping. For randomness, I seed the random generator or inject it, and for jitter I test properties such as bounds rather than exact values. For external state such as environment variables, the file system and the network, I use monkeypatch, temporary directories and fakes, and reset state between tests so they are independent and can run in any order or in parallel. For tests of retry and backoff logic, I replace sleep with a recorder, asserting on the delays requested. Flaky tests are fixed or quarantined promptly.

Likely follow-ups

- How do you test a backoff schedule without waiting?
- Why inject a clock?

Make time, randomness and external state explicit: inject clocks or use freezegun, seed or inject random generators, use monkeypatch, tmp_path and fakes, replace sleep with a recorder and keep tests independent. Fix flaky tests quickly.

```
from freezegun import freeze_time
@freeze_time('2026-01-01 12:00')
def test_token_expired(): ...
```

```
monkeypatch.setattr(time, 'sleep', lambda s: delays.append(s))
```

Interview tip: Say inject the clock and replace sleep.

📚 Official docs

- pytest documentation

certs.eknathalabs.com

Q56 What is a good testing strategy for infrastructure automation code in Python?

Testing & quality

🎙 Spoken answer · ~67 s

I use a pyramid. At the base, fast unit tests cover pure logic: parsing, decision rules, formatting and validation, with no cloud access. Above that, component tests exercise modules against fakes such as moto for AWS and responses for HTTP, verifying behaviour including error and retry paths. Then integration tests run against a real but disposable sandbox account, project or cluster, created for the pipeline and destroyed afterwards, to confirm permissions, API quirks and eventual consistency issues that mocks hide, kept few and focused. For destructive tools, tests prove that dry-run changes nothing and that safeguards, such as limits and tag checks, prevent deleting the wrong things, since the worst bugs are destruction of production data. Tests assert idempotency by running operations twice. Contract tests compare mocks with real API responses periodically. In CI, unit and component tests run on every commit, integration tests on merge or nightly, with cost and cleanup controls and unique resource prefixes to avoid collisions.

Likely follow-ups

- Which tests prove a delete tool is safe?
- How do you prevent leftover resources from integration tests?

Use a pyramid: fast unit tests for logic, component tests with moto and responses, a few integration tests in disposable sandboxes, tests that dry-run changes nothing and safeguards hold, idempotency tests and periodic contract checks, with cleanup and unique prefixes.

```
pytest tests/unit -q && pytest tests/integration -m sandbox --maxfail=1
```

```
aws resourcegroupstaggingapi get-resources --tag-filters Key=test-run,Values=$RUN_ID
```

Interview tip: Say prove the destructive paths are guarded.

📚 Official docs

- Python Documentation
- pytest documentation

certs.eknathalabs.com

Q57 How do you set up pre-commit hooks and CI checks that keep a Python automation repository healthy?

Testing & quality

🎙 Spoken answer · ~67 s

Pre-commit runs fast checks locally before each commit, defined in a configuration file that lists hooks from repositories pinned by version. Typical hooks include ruff for lint and format, mypy, detection of private keys and secrets, trailing whitespace and end-of-file fixers, YAML and TOML validation, and a check against large files. Because hooks are versioned in the repository, every developer gets the same setup after installing once. They are quick, so they do not become an obstacle. CI repeats the same checks, since local hooks can be skipped, and adds slower ones: the full test suite with coverage, dependency vulnerability scanning with pip-audit, a build of the package, container image scanning if relevant, and integration tests on merge. Branch protection requires the checks to pass, and automated dependency updates through Renovate or Dependabot propose upgrades with the tests as the safety net. A README documents how to run the checks locally with one command, so contributors can reproduce CI failures.

Likely follow-ups

- Why must CI repeat pre-commit checks?
- How does pip-audit help?

🗺 Quality gates



Pre-commit runs fast pinned hooks locally for lint, format, types and secret detection; CI repeats them plus the full tests, coverage, pip-audit and build, enforced by branch protection. Automated dependency updates use the tests as a safety net.

```
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks: [{id: ruff}, {id: ruff-format}]
```

```
pip-audit -r requirements.txt
```

Interview tip: Say the same checks locally and in CI.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q58 How do you test infrastructure state from Python, for example with Testinfra?

Testing & quality

🎙 Spoken answer · ~66 s

Testinfra is a pytest plugin that lets you write tests about the state of a server: that a package is installed, a service is running and enabled, a port is listening, a file exists with given permissions and content, or a command returns a certain output. It connects through local execution, SSH, Docker or Ansible inventory, so the same tests can validate a container, a VM built by Packer, or hosts provisioned by Ansible. Tests use a host fixture with objects such as host.package, host.service, host.socket and host.file, which makes them readable. I use it to verify images before publishing them, to check that a configuration run achieved the intended state, and as smoke tests after deployments. It complements, but does not replace, application-level tests. Similar tools include Goss and InSpec. Tests should check observable outcomes, not implementation details, run quickly and fail with clear messages, and be part of the pipeline that builds and releases the infrastructure.

Likely follow-ups

- Which connection backends does Testinfra support?
- How is this different from application tests?

Testinfra is a pytest plugin that asserts server state such as packages, services, ports and files over local, SSH, Docker or Ansible connections. Use it to verify images, configuration runs and deployments, alongside application tests.

```
def test_nginx(host):
    assert host.package('nginx').is_installed
    assert host.service('nginx').is_running
    assert host.socket('tcp://0.0.0.0:80').is_listening
```

```
pytest --hosts=ssh://web01 tests/
```

Interview tip: Say check observable state, not implementation.

📚 Official docs

- Python Documentation
- pytest documentation

certs.eknathalabs.com

Q59 How do virtual environments work, and how do venv, virtualenv, pipx and uv differ?

Packaging & envs

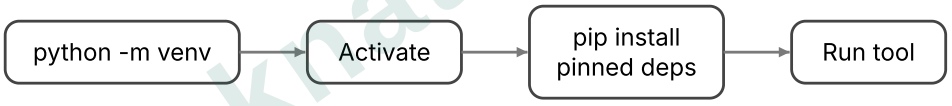
🎙 Spoken answer · ~70 s

A virtual environment is an isolated directory with its own Python interpreter link and site-packages, so each project installs its own dependency versions without affecting the system Python or other projects. The venv module is built in, virtualenv is a faster third-party tool with extra features, and tools like uv create environments extremely quickly. Activating an environment adjusts the path so python and pip refer to it, though calling the interpreter by its full path works without activation, which is how cron jobs and systemd units should run them. Pipx installs command-line tools each in its own isolated environment and exposes the commands, ideal for tools like black or awscli-like utilities. Uv is a modern, fast package and project manager that can create environments, resolve and lock dependencies, manage Python versions and run tools, replacing pip, pip-tools, virtualenv and pipx in many workflows. On servers, I never install project dependencies into the system Python, since operating system tools may depend on it and upgrades can break them.

Likely follow-ups

- Why avoid installing into the system Python?
- How do you run a venv script from cron?

🗺 Isolated environment



A virtual environment isolates a project's interpreter and packages. venv is built in, virtualenv and uv are faster, pipx isolates CLI tools, and uv can also manage dependencies and Python versions. Call the environment's interpreter directly in cron or systemd and never pollute the system Python.

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
```

```
*/5 * * * * /opt/tool/.venv/bin/python /opt/tool/run.py
```

```
pipx install ruff
```

Interview tip: Say never install into the system Python.

📚 Official docs

- Python: venv

certs.eknathalabs.com

Q60 Compare pip, pip-tools, Poetry and uv for dependency management and lock files.

Packaging & envs

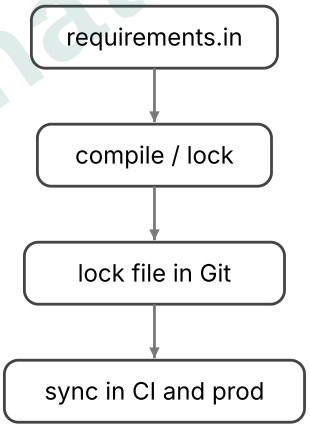
🎙 Spoken answer · ~70 s

Pip installs packages from an index, and a requirements file lists them, but on its own it does not lock transitive dependencies, so two installs on different days can differ. Pip-tools compiles a loose requirements input file into a fully pinned requirements file with hashes, giving reproducible installs with a simple workflow. Poetry manages a project with pyproject, resolves dependencies, creates a lock file, builds and publishes packages, and handles virtual environments, with a more opinionated workflow. Uv is a newer, very fast tool that provides a pip-compatible interface and a project workflow with a cross-platform lock file, plus Python installation and tool running. The principle is the same: declare direct dependencies with sensible version ranges in pyproject, generate a lock file that pins everything, including hashes, install from the lock in CI and production, and update deliberately through a pull request, ideally automated. The choice depends on team preference, speed needs and tooling support, but consistency within the organisation matters more than which tool is chosen.

Likely follow-ups

- Why include hashes in a lock file?
- What is the difference between direct and transitive dependencies?

🗺 Lock workflow



Pip alone does not lock transitive dependencies. pip-tools compiles pinned files, Poetry and uv manage projects with lock files, and uv is very fast. Declare ranges in pyproject, lock everything with hashes, install from the lock and update through reviewed pull requests.

```
pip-compile --generate-hashes requirements.in -o requirements.txt
```

```
uv lock && uv sync --frozen
```

Interview tip: Say lock files make installs reproducible.

certs.eknathalabs.com

📚 **Official docs**

- Terraform: Dependency lock file
- Python: pathlib
- Python: venv

Q61 What goes in pyproject.toml, and how do you structure a modern Python project?

Packaging & envs

🎙 **Spoken answer · ~63 s**

The pyproject file is the standard place for project metadata and tool configuration. The build-system table names the build backend, such as hatchling, setuptools or flit, and its requirements. The project table holds the name, version, description, required Python version, dependencies, optional dependency groups for extras like dev or docs, and scripts that define command-line entry points. Tool tables configure ruff, mypy, pytest and coverage in one file. A common layout uses a src directory containing the package, which prevents accidentally importing the local folder instead of the installed package and catches packaging mistakes, with a tests directory beside it, a README, a licence, a changelog and CI configuration. Version can be static or derived from Git tags. The project is installed in editable mode for development, and built into a wheel for release. Keeping configuration in one standard file reduces clutter and makes the project understandable to new contributors and tooling.

Likely follow-ups

- What does the src layout protect against?
- What are optional dependencies?

pyproject.toml holds build system, project metadata, dependencies, optional groups and script entry points plus tool configuration for ruff, mypy and pytest. Use a src layout with tests beside it, editable installs for development and wheels for release.

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
[project]
name = "mytool"
version = "1.0.0"
requires-python = ">=3.10"
dependencies = ["requests>=2.31"]
```

```
pip install -e '.[dev]'
```

Interview tip: Say one standard file and a src layout.

certs.eknathalabs.com

📚 **Official docs**

- Python Documentation
- Python Packaging User Guide

Q62 How do you secure the Python dependency supply chain?

Packaging & envs

🎙 **Spoken answer · ~67 s**

Third-party packages run with the privileges of the script, so the supply chain is an attack surface. I start by minimising dependencies and choosing well-maintained ones. I pin exact versions with hashes in a lock file and install with hash checking, so a tampered or substituted package fails verification. I prefer an internal package index or proxy that mirrors approved packages, so builds do not depend on the public index and typosquatted or dependency-confusion names, where a public package shadows an internal name, are blocked by configuring index priority and reserving internal names. Vulnerability scanning with pip-audit, Dependabot or similar runs in CI and on a schedule, and automated update pull requests keep versions current. I generate a software bill of materials for releases, use trusted publishing and signing for packages I publish, and run builds in clean environments with least privilege and no long-lived secrets. Reviewing the changelog of major updates and watching for suspicious maintainer changes completes the picture.

Likely follow-ups

- What is dependency confusion?
- What does hash-checking mode do?

🗺 **Supply chain controls**



Minimise dependencies, pin versions with hashes and install in hash-checking mode, use an internal index or proxy with reserved internal names, scan with pip-audit or Dependabot, produce SBOMs, sign releases and build in clean low-privilege environments.

```
pip install --require-hashes -r requirements.txt
```

```
pip-audit -r requirements.txt
```

```
pip config set global.index-url https://pypi.internal.example.com/simple
```

Interview tip: Say hashes plus an internal mirror stop tampering and confusion.

certs.eknathalabs.com

📚 **Official docs**

- Python: venv
- Python Documentation

Q63 How do you containerise a Python application well?

Packaging & envs

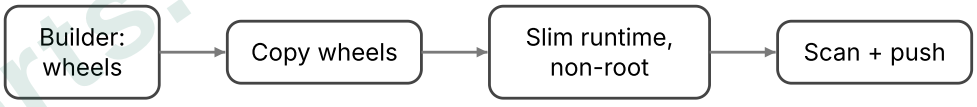
🎙 **Spoken answer · ~69 s**

I start from a slim official base image matching the Python version, or a distroless image for the final stage, and use a multi-stage build: the first stage installs build tools and compiles dependencies into wheels or a virtual environment, and the final stage copies only the environment and application code, keeping the image small and free of compilers. Dependencies are installed from the lock file before copying the source, so Docker's layer cache is reused when only code changes. I run as a non-root user, set environment variables like PYTHONUNBUFFERED so logs appear immediately and PYTHONDONTWRITEBYTECODE to avoid clutter, use a read-only filesystem where possible, and define a health check. Secrets are never baked into the image or build arguments, but supplied at run time. The process handles SIGTERM, runs as PID one directly through an exec-form command, or under a minimal init, so shutdown is clean. I pin the base image by digest, scan the image for vulnerabilities and rebuild regularly for patches.

Likely follow-ups

- Why copy the lock file before the source code?
- Why use the exec form of CMD?

🗺 **Multi-stage build**



Use a slim base and a multi-stage build, install locked dependencies before source for caching, run as non-root with unbuffered output, use exec-form CMD and handle SIGTERM, keep secrets out of the image, pin base digests and scan and rebuild regularly.

```
FROM python:3.12-slim AS build
COPY requirements.txt .
RUN pip wheel -r requirements.txt -w /wheels
FROM python:3.12-slim
COPY --from=build /wheels /wheels
RUN pip install --no-index /wheels/*
USER 10001
CMD ["python", "-m", "mytool"]
```

Interview tip: Say multi-stage, non-root, exec form, no secrets baked in.

certs.eknathalabs.com

📚 **Official docs**

- Python Documentation

Q64 How do you manage multiple Python versions across developer machines and servers?

Packaging & envs

🎙 **Spoken answer · ~69 s**

The project declares the supported Python versions in pyproject with requires-python, and CI tests across them with a matrix, so compatibility is verified. For developers, version managers such as pyenv or uv install and switch interpreters per project, with a version file in the repository like python-version, so everyone uses the same one. On servers, I avoid relying on the system Python, which belongs to the operating system and changes with upgrades, and instead use containers with a pinned Python, or a dedicated interpreter installed under an application directory or by a configuration management role, with a virtual environment built on it. When an operating system upgrade removes or changes a Python version, applications that depend on it break, which is why dedicated interpreters and containers are safer. I track end-of-life dates, since versions stop receiving security patches after about five years, and plan upgrades, testing with the new version in CI early and fixing deprecation warnings, which can be turned into errors in tests.

Likely follow-ups

- Why not rely on the system Python?
- How do you catch deprecations before an upgrade?

Declare requires-python, test a version matrix in CI, use pyenv or uv with a version file for developers, and on servers use containers or dedicated interpreters instead of the system Python. Track end-of-life dates and treat deprecation warnings as errors in tests.

```
pyenv install 3.12.4 && pyenv local 3.12.4
```

```
uv python install 3.12
```

```
pytest -W error::DeprecationWarning
```

Interview tip: Say containers or a dedicated interpreter, never system Python.

📚 **Official docs**

- Python Documentation

certs.eknathalabs.com

Q65 How do you distribute Python tools to hosts that have no internet access?

Packaging & envs

🎙 **Spoken answer · ~66 s**

There are several approaches depending on constraints. A wheelhouse is a directory of wheels for the application and all its dependencies, built on a matching platform and Python version with pip download or pip wheel, copied to the target and installed with no-index and find-links, which works offline and reproducibly. A zipapp, created with the zipapp module, bundles pure Python code and dependencies into one runnable file, simple when dependencies are pure Python. PyInstaller or similar tools freeze the application and an interpreter into a single executable, which avoids needing Python on the target but is larger and platform specific. A container image delivered through an internal registry or tarball is another option when a container runtime exists. Finally, an internal package mirror such as devpi, Artifactory or Nexus lets hosts install normally from inside the network. Whichever I use, I sign or checksum artefacts, version them, and ensure the build platform matches the target's architecture and libc.

Likely follow-ups

- What must match between build and target platforms?
- When would you choose PyInstaller?

Use a wheelhouse installed with --no-index, a zipapp for pure Python, PyInstaller for a single executable, a container image, or an internal mirror like devpi or Artifactory. Match architecture, libc and Python version, and checksum or sign artefacts.

```
pip download -r requirements.txt -d wheelhouse/ --platform manylinux2014_x86_64 --only-binary=:all:
```

```
pip install --no-index --find-links wheelhouse/ -r requirements.txt
```

Interview tip: Say a wheelhouse built for the exact target platform.

📚 **Official docs**

- Python Documentation

certs.eknathalabs.com

Q66 How do you version, release and publish an internal Python package?

Packaging & envs

🎙 Spoken answer · ~67 s

I use semantic versioning: patch for bug fixes, minor for backward compatible features and major for breaking changes, with the version derived from a Git tag by the build backend so there is a single source of truth. A changelog records user-visible changes. The release flow is automated: when a tag is pushed, CI runs the test suite, builds the sdist and wheel with the standard build tool, verifies the package with twine check, and uploads to the internal index, such as Artifactory, Nexus, a cloud artifact registry or a private PyPI, using short-lived credentials or trusted publishing, not a long-lived token stored in plain text. Consumers pin version ranges, and I keep old releases available, yanking bad ones rather than deleting them. For breaking changes I provide deprecation warnings one release ahead, and document the migration. Documentation is published alongside, and the package includes metadata, a licence and a typed marker file so that type checkers can use the annotations.

Likely follow-ups

- What is the difference between yanking and deleting a release?
- What is a py.typed marker?

Use semantic versions derived from Git tags, keep a changelog and automate releases in CI: test, build sdist and wheel, twine check and upload to the internal index with short-lived credentials. Yank bad releases, announce deprecations first and ship a py.typed marker.

```
python -m build && twine check dist/*
```

```
twine upload --repository-url https://pypi.internal.example.com/ dist/*
```

Interview tip: Say tag, build in CI, publish with short-lived credentials.

📚 Official docs

- Python: venv
- Python Documentation
- Python Packaging User Guide

peers.eknathalabs.com

Q67 How do you use multiprocessing and process pools correctly for CPU-bound work?

Concurrency & speed

🎙 **Spoken answer · ~70 s**

Multiprocessing runs separate interpreter processes, so CPU-bound Python code can use several cores. The ProcessPoolExecutor or multiprocessing.Pool distributes tasks to worker processes. Arguments and results are pickled and sent between processes, so they must be picklable, and large objects are expensive to transfer, which means chunking work and returning small results is important, or sharing data through memory-mapped files or shared memory. Functions must be importable at module level, and scripts need the main guard, because on some platforms new processes import the main module again, and without the guard they spawn recursively. The start method matters: fork is fast but unsafe with threads and some libraries, while spawn is safer and the default on macOS and Windows. Choose the worker count near the number of cores, leaving headroom, and consider memory since each process has its own copy. Handle worker exceptions by checking future results, set timeouts and make sure the pool shuts down cleanly. For many small tasks, overhead can exceed the gain, so measure before adopting.

Likely follow-ups

- Why does the main guard matter with multiprocessing?
- What must be true of arguments passed to workers?

Use ProcessPoolExecutor or Pool for CPU-bound tasks; arguments and results must be picklable and chunked to limit transfer cost. Use the main guard, choose spawn or fork carefully, size workers to cores and memory, and handle worker errors. Measure, since small tasks may not benefit.

```
from concurrent.futures import ProcessPoolExecutor
if __name__ == '__main__':
    with ProcessPoolExecutor(max_workers=os.cpu_count()) as ex:
        out = list(ex.map(hash_file, files, chunksize=16))
```

Interview tip: Say chunk the work and keep results small.

📚 **Official docs**

- Python: concurrent.futures

certs.eknathalabs.com

Q68 What is property-based testing and where does it help in operations code?

Testing & quality

🎙 Spoken answer · ~68 s

Property-based testing, with libraries such as Hypothesis, generates many inputs automatically and checks that stated properties hold for all of them, instead of testing a few hand-picked examples. You describe the kinds of input, such as strings, integers in a range or structured records, and a property, such as parsing then formatting a value gives the original back, a sorted list is ordered, or a function never raises for valid input. When a failure is found, the tool shrinks the input to a minimal failing example, which makes the bug easy to understand. It is valuable for parsers of logs and configuration formats, CIDR and version-string handling, serialisation, rate limiters and idempotency, where edge cases such as empty strings, unusual Unicode, huge numbers or boundaries are easy to overlook. It complements example-based tests, which remain better for documenting specific scenarios. Practical points are keeping properties meaningful, seeding for reproducibility in CI, controlling runtime with settings, and saving found failures as regression tests.

Likely follow-ups

- What does shrinking do?
- Which properties are good for a parser?

Property-based testing with Hypothesis generates many inputs and checks properties such as round trips and ordering, shrinking failures to minimal examples. It suits parsers, CIDR and version handling and idempotency, alongside example tests.

```
from hypothesis import given, strategies as st
@given(st.lists(st.integers()))
def test_sorted(xs):
    assert sorted(xs) == sorted(sorted(xs))
```

```
pytest --hypothesis-seed=42
```

Interview tip: Say round-trip properties find edge cases you did not think of.

📚 Official docs

- pytest documentation
- Hypothesis documentation

certs.eknathalabs.com

Q69 How do you deploy and run a Python service or script on servers with systemd?

Packaging & envs

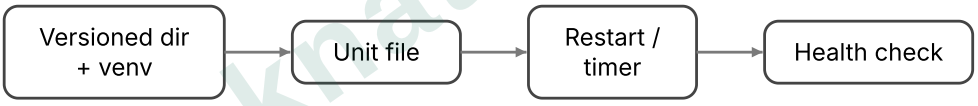
🎙 Spoken answer · ~65 s

I install the application into a dedicated directory with its own virtual environment built from the lock file, owned by a service account, rather than into the system Python. A systemd unit then runs it: ExecStart points at the environment's interpreter or console script, User and Group set the unprivileged account, WorkingDirectory and Environment or EnvironmentFile supply configuration, and Restart on failure with a limited restart rate handles crashes. Hardening options such as NoNewPrivileges, ProtectSystem, PrivateTmp, ReadOnlyPaths and memory or CPU limits reduce risk. Logs go to standard output and are collected by the journal, with structured fields. For periodic jobs, a systemd timer pairs with a oneshot service and gives better logging, dependency handling, randomised delay and missed-run catch-up than cron. Deployments create a new versioned directory and switch a symlink, so rollback is quick, with configuration management or a pipeline doing the work, and a health check after restart. Graceful shutdown handles SIGTERM, within TimeoutStopSec.

Likely follow-ups

- Why use a systemd timer instead of cron?
- Which systemd options harden a service?

🗺 systemd deployment



Install into a dedicated directory with its own virtual environment and service account, run with a systemd unit using Restart, EnvironmentFile and hardening options, use timers for periodic jobs, switch versioned releases by symlink and handle SIGTERM within the stop timeout.

```
[Service]
User=svc-tool
ExecStart=/opt/tool/current/.venv/bin/tool run
Restart=on-failure
NoNewPrivileges=true
ProtectSystem=strict
```

```
systemctl status tool && journalctl -u tool -n 50
```

Interview tip: Say a dedicated venv and user, hardened unit, timers over cron.

📚 Official docs

- systemd.timer
- Python Documentation

certs.eknathalabs.com

Q70 How do you generate reports from CSV, JSON or cloud data using the csv module or pandas?

Files & data

🎙 Spoken answer · ~70 s

For simple tasks, the csv module reads and writes rows as lists or dictionaries with DictReader and DictWriter, streams large files and needs no dependency. For analysis, pandas loads data into DataFrames where I can filter, group, join and pivot with concise operations, and read and write CSV, JSON, Excel and Parquet. Typical reports are cost by tag from billing exports, inventory of instances by region, certificate expiry lists or vulnerability summaries. I specify dtypes and date parsing explicitly, use chunksize or usecols for big files, handle missing values deliberately and avoid iterating rows in Python, preferring vectorised operations. For very large data, DuckDB or Polars query files directly with SQL or lazy plans, and are fast and memory efficient. Output should be reproducible and reviewable: sorted rows, stable column order, timestamps in UTC, and clear headers. Reports can be written to CSV or Excel, rendered as HTML or Markdown tables and sent by email or Slack, with sensitive columns removed and the generating code and parameters recorded.

Likely follow-ups

- Why avoid iterating rows in pandas?
- When would you use DuckDB instead of pandas?

Use csv DictReader and DictWriter for simple streaming, pandas for grouping, joins and pivots with explicit dtypes and chunking, and DuckDB or Polars for large files. Keep output reproducible with stable ordering and UTC timestamps, and strip sensitive columns.

```
import pandas as pd
df = pd.read_csv('billing.csv', usecols=['tag_team', 'cost'], dtype={'tag_team': 'string'})
print(df.groupby('tag_team')['cost'].sum().sort_values(ascending=False))
```

Interview tip: Say vectorise, never loop over rows.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q71 How would you write a robust health-check or monitoring script that runs from cron or a timer? Ops automation

🎙 Spoken answer · ~68 s

The script should be simple, fast and honest about failure. It checks one or a few well-defined things, such as an HTTP endpoint returning the expected status and content within a time limit, a TCP port, disk usage on a mount, certificate expiry or a queue depth, with explicit timeouts on every network call so it cannot hang and pile up. It reports results through the exit code, a metric or a push to the monitoring system, and a structured log line, and distinguishes check failed from check could not run. To avoid alert storms, it applies thresholds with a few consecutive failures before alerting, and recovery notifications. A lock file or flock prevents overlapping runs if one is slow. It runs as an unprivileged user with minimal access, and credentials come from the environment or a secrets store. I prefer exposing metrics to Prometheus, through a textfile collector or an exporter, and alerting centrally, rather than sending ad-hoc emails from each script.

Likely follow-ups

- How do you prevent overlapping cron runs?
- How do you distinguish a failing check from a broken check?

🗺 Health check script



Check specific things with explicit timeouts, report through exit codes, metrics and structured logs, distinguish failure from inability to check, require consecutive failures before alerting, prevent overlap with a lock and run unprivileged. Prefer Prometheus metrics to ad-hoc emails.

```
r = requests.get(url, timeout=5)
ok = r.status_code == 200 and 'ok' in r.text
sys.exit(0 if ok else 2)
```

```
flock -n /var/lock/check.lock python check.py
```

Interview tip: Say timeouts everywhere and a lock against overlap.

📚 Official docs

- systemd.timer

certs.eknathalabs.com

Q72 How would you write a script to find and clean up unused cloud resources safely, such as unattached volumes or old snapshots?

Ops automation

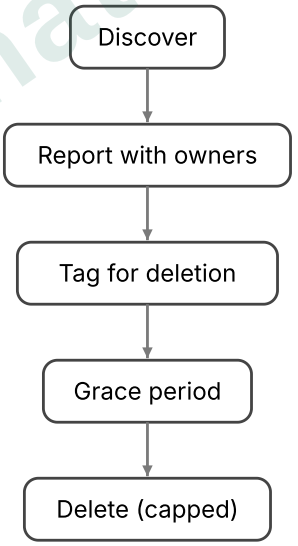
🎙 Spoken answer · ~69 s

Cleanup scripts are dangerous because mistakes delete real data, so safety dominates the design. I begin with discovery only: list candidates, such as unattached volumes older than a threshold, snapshots without an associated AMI or retention tag, or idle load balancers, and output a report with owner tags, ages and estimated cost. The deletion step is separate, requires an explicit apply flag and defaults to dry-run. Multiple guards apply: skip resources tagged with a do-not-delete or retention marker, require a minimum age, restrict to specific accounts and regions printed before running, cap the number of deletions per run, and snapshot or back up before deleting where appropriate. A grace period approach first tags a resource with a planned deletion date and notifies the owner, deleting later only if no one objects. I use paginators for complete listing, handle dependencies and errors per resource without stopping, log every action with identifiers for audit, and make the job idempotent. Testing runs in a sandbox account with decoy resources.

Likely follow-ups

- What is a grace period or tag-then-delete approach?
- Which guards limit the blast radius?

🗺 Safe cleanup



Separate discovery from deletion, default to dry-run, honour protection tags, enforce minimum age, print account and region, cap deletions, tag-then-delete with owner notification, paginate fully, log every action and test in a sandbox with decoys.

certs.eknathalabs.com

```
for page in ec2.get_paginator('describe_volumes').paginate(Filters=
[{'Name':'status','Values':['available']}]):
    for v in page['Volumes']:
        if age_days(v['CreateTime']) > 30 and not protected(v):
candidates.append(v['VolumeId'])
```

```
python cleanup.py --account prod --region eu-west-1 --max-delete 25 --apply
```

Interview tip: Say report first, then tag, then delete.

📚 **Official docs**

- boto3 documentation

Q73 How do you monitor TLS certificate expiry with Python?

Ops automation

🎙 **Spoken answer · ~70 s**

Expired certificates cause avoidable outages, so a scheduled check should inventory endpoints and warn well ahead. For each host and port, the script opens a TLS connection with the ssl module and a timeout, uses the proper server name for SNI, retrieves the peer certificate and reads the notAfter date, computing days remaining. Libraries such as cryptography parse certificates in more detail, including issuer, subject alternative names and chain, useful for finding wrong or incomplete chains. Sources of endpoints include a static list, DNS records, load balancers, Kubernetes ingress resources and certificate manager inventories, plus certificate files on disk for internal services. Thresholds create tiered alerts, for example warning at thirty days and critical at seven, and the check emits a Prometheus metric with the expiry timestamp so alert rules and dashboards work centrally. It must verify the certificate properly for trust as well, and distinguish connection failures from expiry. Where possible, automation with ACME or a certificate manager renews certificates, and the check confirms renewal actually happened.

Likely follow-ups

- Why must you set SNI when connecting?
- What other certificate problems besides expiry should you detect?

Connect with ssl and SNI under a timeout, read notAfter, compute days left, parse chains with cryptography, source endpoints from inventories and Kubernetes, alert in tiers and export an expiry metric. Also detect trust and chain errors and confirm automatic renewal worked.

```
import ssl, socket, datetime
ctx = ssl.create_default_context()
with socket.create_connection((host, 443), timeout=5) as s, ctx.wrap_socket(s,
server_hostname=host) as t:
    exp = datetime.datetime.strptime(t.getpeercert()['notAfter'], '%b %d %H:%M:%S %Y %Z')
print((exp - datetime.datetime.utcnow()).days)
```

certs.eknathlabs.com

Interview tip: Say export the expiry as a metric and alert in tiers.

📚 **Official docs**

- Python Documentation

Q74 How would you automate database backups and restore verification with Python?

Ops automation

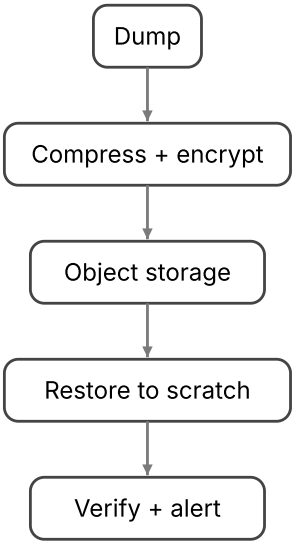
🎙 **Spoken answer · ~70 s**

A backup script orchestrates native tools rather than reimplementing them. It runs pg_dump or a physical backup tool, or triggers cloud snapshots through the SDK, with credentials from the environment or a secrets manager, uses compression and encryption, streams output directly to object storage to avoid large temporary files, and writes a checksum and a manifest with the timestamp, version, size and source. It applies a retention policy, deleting old backups according to rules, with protection for the most recent good backups, and sends metrics and alerts if a run fails or is late. Crucially, backups are only real if they restore, so a verification job periodically restores the latest backup to an isolated instance, runs sanity queries or row count comparisons, measures restore time against the recovery objective, and reports the result. Locking prevents overlapping runs, errors from the subprocess are checked through exit codes and not ignored, and logs omit secrets. Access to backup storage is restricted and immutable or versioned storage protects against ransomware and mistakes.

Likely follow-ups

- Why must a backup be tested by restoring?
- How do you protect backups from deletion?

🗺 **Backup and restore test**



certs.eknathalabs.com

Orchestrate native dump or snapshot tools, stream compressed encrypted output to object storage with checksums and a manifest, apply retention, alert on failure or lateness, and regularly restore into an isolated instance to verify content and measure restore time. Use immutable storage.

```
subprocess.run(f'pg_dump -Fc {db} | gzip | aws s3 cp - s3://backups/{db}/{ts}.dump.gz',
shell=True, check=True)
```

```
pg_restore -d verify_db latest.dump && psql verify_db -c 'select count(*) from orders'
```

Interview tip: Say the restore test is the real backup test.

📚 **Official docs**

- Python Documentation

Q75 How would you build an inventory or compliance report across many cloud accounts and regions?

Ops automation

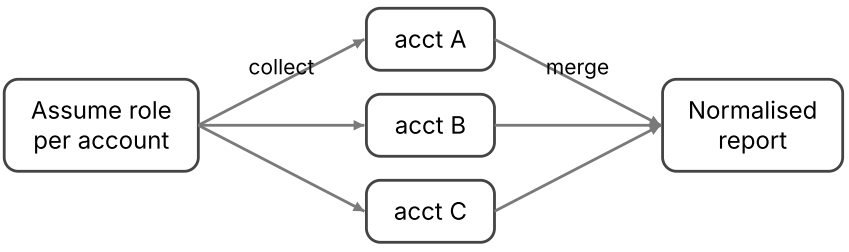
🎙 **Spoken answer · ~73 s**

I design it as a pipeline of collection, normalisation and reporting. Collection iterates through accounts, using an assumed role in each, and through enabled regions, calling the relevant describe or list APIs with paginators, in parallel with a bounded thread pool and error handling per account and region, so one failure does not abort the run. Retries and backoff handle throttling. The raw results are normalised into a common schema with fields like account, region, resource type, identifier, tags, creation time and key settings, which makes querying uniform across services. The data is stored, for example as Parquet or in a database, with a timestamp for each run, so I can see trends and changes. Rules then evaluate compliance, such as missing owner tags, unencrypted volumes, public buckets or outdated runtimes, producing findings with severity and owners. Reports are generated as dashboards or files and delivered to teams. Native services such as AWS Config or Resource Explorer can do part of this, so I use them when they fit, and write custom code for gaps.

Likely follow-ups

- Why normalise into a common schema?
- When would you use AWS Config instead of custom code?

🗺 **Multi-account inventory**



certs.eknathalabs.com

Collect with assumed roles across accounts and regions using paginators and bounded parallelism, normalise to a common schema, store timestamped results, evaluate compliance rules into findings with owners and deliver reports. Use native services like AWS Config where they fit.

```
sts = boto3.client('sts')
creds = sts.assume_role(RoleArn=f'arn:aws:iam::{acct}:role/audit', RoleSessionName='inv')
['Credentials']
ec2 = boto3.client('ec2', region_name=r, aws_access_key_id=creds['AccessKeyId'],
aws_secret_access_key=creds['SecretAccessKey'], aws_session_token=creds['SessionToken'])
```

Interview tip: Say tolerate per-account failures and store results over time.

📚 **Official docs**

- boto3 documentation

Q76 **How would you write a Kubernetes operator or controller in Python, for example with kopf?** Ops automation

🎙 **Spoken answer · ~70 s**

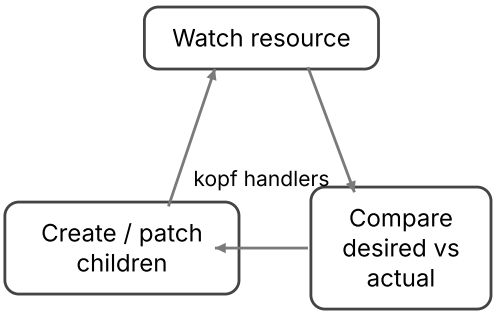
A controller watches resources and reconciles actual state toward desired state. With kopf, a Python framework, I define handlers with decorators for events such as creation, update, deletion and timers on a custom resource, and the framework handles watching, retries, status patches, finalizers and leader election. First I define the custom resource definition with a validated schema, then write handlers that read the spec, create or patch child resources such as deployments or secrets, set owner references so Kubernetes garbage-collects children, and update the status with conditions that users can inspect. Handlers must be idempotent, since events can be repeated or replayed, and should not block for long. RBAC is least privilege for exactly the resources the operator manages, and it runs as a deployment with health probes and metrics. I test handlers with unit tests on the logic and an integration test in a kind cluster. For serious or widely used operators, Go with controller-runtime is the common choice, but Python is productive for internal automation.

Likely follow-ups

- Why must handlers be idempotent?
- What are owner references for?

certs.eknathalabs.com

🗺 **Reconcile loop**



Use kopf handlers for create, update, delete and timers on a custom resource: reconcile child resources with owner references, update status conditions, keep handlers idempotent and non-blocking, apply least-privilege RBAC and test in a kind cluster. Go with controller-runtime suits larger operators.

```
import kopf
@kopf.on.create('example.com', 'v1', 'databases')
def create(spec, name, namespace, logger, **kw):
    logger.info(f'creating {name}')
    return {'phase': 'Ready'}
```

```
kopf run operator.py --namespace=default
```

Interview tip: Say idempotent reconcile with owner references and status.

📚 **Official docs**

- Kubernetes: Operator pattern
- Kubernetes Python client
- Python Documentation

certificeknathalabs.com

Q77 How would you build a ChatOps bot or command that lets engineers run safe operations from Slack?

Ops automation

🎙 Spoken answer · ~70 s

The bot is a thin, audited front end over well-tested automation. Engineers invoke commands such as a status check or a scale operation through a slash command or button, which Slack sends to a service as a signed HTTP request. The service verifies the Slack signature and timestamp, acknowledges within three seconds and processes asynchronously, posting results back into the thread. Authorisation is essential: map the Slack user to roles, check permissions per command, and require approvals for risky actions with a second person confirming through an interactive message. Commands are an allow list, each backed by a function with validated arguments, not free-form shell execution. The bot uses its own scoped credentials, never the user's, and logs who ran what, when and with which parameters, to an audit trail, linking to the run output. Rate limits, per-channel restrictions and a kill switch limit damage. Idempotency keys prevent double execution on retries, and long operations report progress. Secrets stay out of messages, and outputs are redacted before posting.

Likely follow-ups

- Why verify the Slack signature?
- How do approvals work for risky commands?

Receive signed slash-command requests, verify them, ack fast and process async; use an allow list of validated commands, map users to roles, require second-person approval for risky actions, use scoped bot credentials, log an audit trail, add idempotency and redact outputs.

```
from slack_bolt import App
app = App(token=bot_token, signing_secret=secret)
@app.command('/scale')
def scale(ack, command, respond):
    ack()
    respond(f"Scaling {command['text']}")
```

Interview tip: Say allow-listed commands, role checks and an audit log.

📚 Official docs

- Python: subprocess

certs.eknathalabs.com

Q78 Compare cron, systemd timers, Kubernetes CronJobs and workflow schedulers such as Airflow for running scheduled Python jobs.

Ops automation

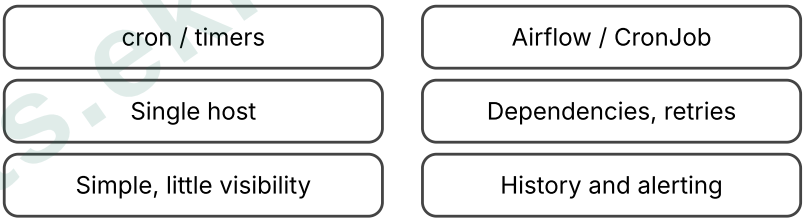
🎙 Spoken answer · ~72 s

Cron is simple and ubiquitous: a schedule line runs a command, but it has minimal environment, no dependency handling, little visibility, mail-based output and no built-in protection from overlapping runs or missed schedules when the machine is down. Systemd timers improve on this with journald logging, resource controls, dependencies, randomised delay, persistent catch-up after downtime and easy inspection with systemctl. Kubernetes CronJobs run containers on a schedule with concurrency policies, history limits, retries through the Job controller and the cluster's logging, good when the workload is containerised, though time zones, missed-start deadlines and idempotency need care. Workflow schedulers such as Airflow, Prefect or Dagster model multi-step pipelines with dependencies, retries, backfills, parameters, dashboards, alerting and per-task logs, suited to data pipelines and complex workflows, with more infrastructure to run. I choose the simplest tool that meets the need: timers for single-host jobs, CronJobs for cluster tasks, and a workflow engine when dependencies, backfills and visibility matter. Jobs should always be idempotent, lock against overlap and alert when they fail or do not run.

Likely follow-ups

- What does a CronJob concurrencyPolicy do?
- How do you detect a job that silently did not run?

🗺 Schedulers



Cron is simple but blind; systemd timers add logging, catch-up and controls; Kubernetes CronJobs suit containerised tasks with concurrency policies; Airflow-style schedulers handle multi-step dependencies and backfills. Pick the simplest that fits, make jobs idempotent, lock against overlap and alert on missed runs.

```
kubectl create cronjob backup --image=registry/backup:1.2 --schedule='0 2 * * *' -- python backup.py
```

```
systemctl list-timers --all
```

Interview tip: Say simplest tool that fits, plus dead-man's-switch alerting.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: CronJob
- systemd.timer(5)
- Python Documentation

Q79 How do you handle scheduled jobs that must not overlap or must run exactly once across several machines?

Ops automation

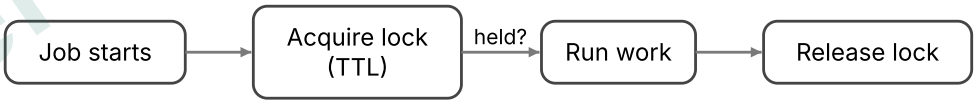
🎙 **Spoken answer · ~73 s**

Overlap protection on one host uses a lock file with flock, or the scheduler's own setting such as a Kubernetes concurrencyPolicy of Forbid, or systemd, which does not start a unit that is already running. Across machines, a distributed lock is needed: options include a database row or advisory lock, a lease in Redis with an expiring key, a compare-and-swap in etcd or Consul, or leader election, where one instance is chosen to run the scheduled work while others stand by. Locks should have a time-to-live and be renewed while the job runs, so a crashed holder does not block forever, and the job should tolerate the lock expiring by being idempotent, since clock pauses and network partitions can cause two holders. True exactly-once is hard, so I aim for at-least-once execution with idempotent effects, using idempotency keys or state records. Fencing tokens protect against a stale holder writing after losing the lock. Simpler alternatives avoid the problem: run the scheduler as a single instance with failover, or use a managed scheduler that guarantees single invocation.

Likely follow-ups

- Why do distributed locks need a time-to-live?
- What is a fencing token?

🗺 **Locking**



Use flock or scheduler concurrency settings on one host, and distributed locks or leader election across hosts with a TTL and renewal. Because exactly-once is hard, aim for at-least-once with idempotent effects and fencing tokens, or use a single scheduler with failover.

```
flock -n /var/lock/job.lock python job.py
```

```
r.set('lock:backup', host, nx=True, ex=600)
```

Interview tip: Say at-least-once with idempotent effects beats chasing exactly-once.

certs.eknathalabs.com

📚 **Official docs**

- systemd.timer
- Apache Kafka documentation

Q80 How would you monitor disk space and clean up logs automatically, safely?

Observability & tooling

🎙 **Spoken answer · ~72 s**

I start with detection: use psutil or shutil.disk_usage to read usage per mount, and report both percent and absolute free space, including inode usage, which can run out separately. Thresholds alert in tiers and export metrics so trends and time-to-full forecasts are visible. For cleanup, the safest approach is configured rotation, logrotate or journald limits, and fixing the source of excess logging, instead of a script deleting files. If an automated cleaner is needed, it works from an explicit allow list of directories and patterns, only removes files older than a retention period, never follows symlinks outside its scope, skips files that are open, which I can check with lsof or psutil, and logs everything it removes, with a dry-run mode. Truncating an active log is preferable to deleting it, since a deleted file held open by a process continues consuming space until the process closes it. After cleaning, it verifies free space increased and escalates if not. Testing uses a temporary directory with fake files before it touches a real system.

Likely follow-ups

- Why does deleting an open file not free space?
- Why monitor inodes as well as bytes?

Measure percent, bytes and inodes with psutil or shutil, alert in tiers and export metrics. Prefer logrotate and fixing noisy logging; if a cleaner is needed, use an allow list, retention age, symlink safety, skip open files, dry-run and logging, and verify the outcome.

```
import shutil
u = shutil.disk_usage('/var')
print(u.free / u.total)
```

```
lsof +L1 | head
```

```
find /var/log/app -type f -mtime +14 -name '*.gz' -print
```

Interview tip: Say deleted-but-open files still hold space.

📚 **Start here (official docs)**

- Python documentation

certs.eknathalabs.com

Q81 How would you automate a rolling restart or deployment across servers from Python while keeping the service available?

Ops automation

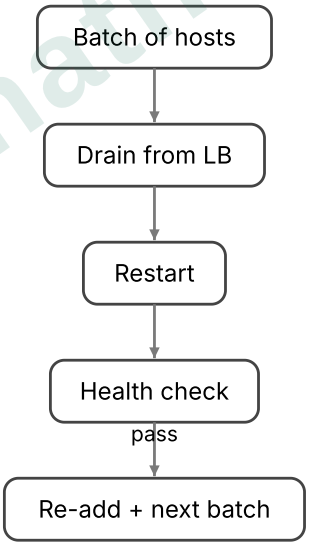
🎙 Spoken answer · ~72 s

I model it as an ordered, observable procedure. First, build the host list from a trusted source and divide it into small batches, never touching more than a safe fraction of capacity at once. For each batch: remove hosts from the load balancer or mark them draining and wait for connections to complete, perform the action, such as deploying a release and restarting the service, run health checks that must pass within a timeout, then re-add the hosts and watch error rate and latency from monitoring before moving on. If checks fail, the rollout stops automatically and optionally rolls back the affected batch, with a clear report. The first batch is a canary. Operations use idempotent steps, timeouts and retries, a lock to avoid two rollouts at once, and a dry-run showing the plan. Parallelism is bounded within a batch. Logs record each step per host. For standard cases, I would use existing tools, Ansible, Kubernetes rolling updates or a deployment platform, and only write custom code for logic those tools cannot express.

Likely follow-ups

- What stops the rollout automatically?
- When would you not write this in Python?

🗺 Rolling restart



Build the host list, batch it within capacity limits, drain from the load balancer, deploy and restart, require health checks, watch metrics, then continue; stop and optionally roll back on failure. Use a canary, idempotent steps, locks, dry-run and logs, and prefer Ansible or Kubernetes when they fit.

```
for batch in chunks(hosts, size=3):
    drain(batch); deploy(batch)
    if not healthy(batch, timeout=120): rollback(batch); raise SystemExit('batch failed')
    undrain(batch); time.sleep(30)
```

certs.eknathalabs.com

Interview tip: Say batch, drain, deploy, health check, stop on failure.

📚 **Official docs**

- Python Documentation

Q82 **How do you expose custom metrics from a Python service or script to Prometheus?** Observability & tooling

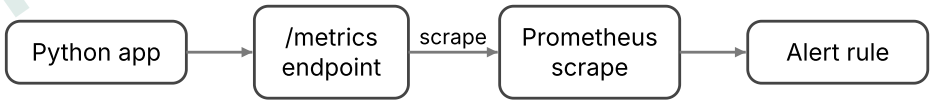
🎙 **Spoken answer · ~73 s**

With the prometheus_client library I define metrics and expose them. A counter tracks events that only go up, such as requests or errors, a gauge tracks values that move up and down like queue depth, a histogram records distributions such as request latency in buckets, which allows percentile estimates in queries, and a summary is less often used. Labels add dimensions, but each unique label combination creates a new time series, so I avoid high-cardinality values such as user ids, URLs with ids or timestamps. The library can start an HTTP server on a port serving the metrics endpoint, or integrate with the web framework through middleware. Long-running services are scraped by Prometheus. Short-lived batch jobs cannot be scraped reliably, so they either write to the node exporter textfile collector or push to a Pushgateway, used only for such jobs, with a last-success timestamp metric. Naming follows conventions: base units such as seconds and bytes, a suffix of total for counters, and a clear help string. I then add alert rules and dashboards for the metrics.

Likely follow-ups

- Why is label cardinality a problem?
- How do batch jobs export metrics?

🗺 **Metrics pipeline**



Use prometheus_client counters, gauges and histograms with low-cardinality labels, expose an HTTP endpoint for long-running services, and use the textfile collector or Pushgateway for batch jobs with a last-success timestamp. Follow naming conventions.

```
from prometheus_client import Counter, Histogram, start_http_server
REQS = Counter('jobs_total', 'Jobs processed', ['status'])
LAT = Histogram('job_seconds', 'Job duration')
start_http_server(9100)
```

Interview tip: Say avoid high-cardinality labels.

certs.eknathalabs.com

📚 **Official docs**

- Prometheus: Instrumentation best practices
- prometheus_client
- Prometheus: Overview

Q83 How do you implement structured logging in Python?

Observability & tooling

🎙 **Spoken answer · ~65 s**

Structured logging emits each event as machine-readable fields, usually JSON, so a log platform can filter and aggregate by field rather than parsing text. With the standard logging module, I configure a formatter that outputs JSON, using python-json-logger or structlog, and add context such as timestamp in UTC, level, logger name, message, service, version, environment and a request or correlation id. Context variables or logging adapters attach the correlation id automatically across a request, including in async code. Exceptions are logged with a traceback in a field, using logger.exception. I log to standard output in containers, letting the platform collect them, and configure levels through the environment so debug can be enabled temporarily. Good practice includes consistent field names, avoiding secrets and personal data through redaction filters, not logging inside tight loops, sampling noisy events and keeping messages stable and descriptive with variable data in fields. Libraries should not configure handlers; the application does, at startup.

Likely follow-ups

- Why a correlation id?
- Where should handlers be configured?

Emit JSON log lines with consistent fields, UTC timestamps, service and version, and a correlation id carried by context variables. Use logger.exception for tracebacks, log to stdout, redact secrets and configure handlers only in the application.

```
import structlog
log = structlog.get_logger().bind(service='billing')
log.info('invoice_created', invoice_id=42, amount_cents=1999)
```

Interview tip: Say fields, not strings, plus a correlation id.

📚 **Official docs**

- Python: logging
- Python Documentation

certs.eknathalabs.com

Q84 How do you add distributed tracing to a Python service with OpenTelemetry?

Observability & tooling

🎙 Spoken answer · ~69 s

OpenTelemetry provides APIs and SDKs for traces, metrics and logs with vendor-neutral export. For tracing, I install the SDK and instrumentation packages for the libraries in use, such as Flask, FastAPI, requests, SQLAlchemy and boto3, and either run the zero-code opentelemetry-instrument wrapper or initialise a tracer provider in code. A resource describes the service name, version and environment. Spans are created automatically for incoming requests and outgoing calls, and I add manual spans around important business operations with attributes such as customer tier or batch size, avoiding sensitive data. Context propagation passes the trace id through HTTP headers using the W3C trace context standard, so spans from different services join into a single trace. Spans are exported over OTLP to a collector, which batches, samples, filters and forwards to a backend such as Tempo or Jaeger. Sampling controls volume and cost, for example head sampling by ratio or tail sampling to keep errors and slow traces. Trace ids also go into logs, linking logs with traces.

Likely follow-ups

- How does context propagation work between services?
- What is the collector for?

🗺 OpenTelemetry



Install the SDK and instrumentation, set a resource with service name, create manual spans with safe attributes, propagate W3C trace context, export OTLP to a collector with sampling, and put trace ids in logs.

```
opentelemetry-instrument --service_name billing python app.py
```

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)
with tracer.start_as_current_span('charge') as span:
    span.set_attribute('batch.size', n)
```

Interview tip: Say propagate context and sample at the collector.

📚 Official docs

- OpenTelemetry: Signals (traces, metrics, logs)
- OpenTelemetry Python
- OpenTelemetry documentation

certs.eknathalabs.com

Q85 How do you design health and readiness endpoints for a Python web service?

Observability & tooling

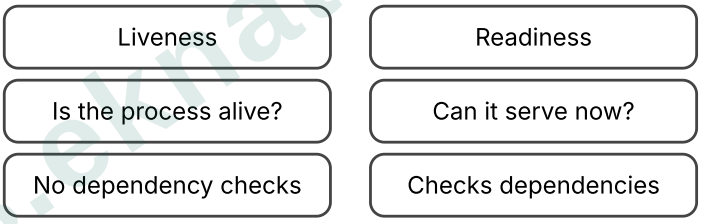
🎙 Spoken answer · ~70 s

A liveness endpoint answers whether the process is running and not stuck; it should be cheap and must not check dependencies, otherwise a database outage makes the orchestrator restart every healthy instance and worsen the incident. A readiness endpoint answers whether this instance can serve traffic now, so it can check that startup finished, caches are warm and critical dependencies are reachable, with short timeouts, and it returns a failing status so the load balancer stops routing to it without killing it. A startup probe protects slow-starting applications. During shutdown, readiness should fail first so traffic drains while in-flight requests finish. Responses use proper status codes, with a small body listing component status for humans, but no sensitive details, and endpoints are not exposed publicly. Checks should not be expensive or trigger heavy load, so I cache results for a few seconds. In Kubernetes, I configure the probes with sensible periods, thresholds and timeouts. Deeper dependency checks belong in monitoring and synthetic tests more than in liveness.

Likely follow-ups

- Why should liveness not check the database?
- What happens to readiness during shutdown?

🗺 Liveness vs readiness



Keep liveness cheap and dependency-free, use readiness to check that the instance can serve with short timeouts, add a startup probe for slow starts, fail readiness first on shutdown, return proper codes without sensitive detail and cache check results.

```
@app.get('/healthz')
def live(): return {'status': 'ok'}
@app.get('/readyz')
def ready():
    if not db_ok(timeout=1): raise HTTPException(503)
    return {'status': 'ready'}
```

Interview tip: Say liveness is cheap; readiness checks dependencies.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Liveness, readiness and startup probes
- Python Documentation

Q86 How would you build a webhook receiver for Alertmanager or similar alert sources in Python?

Observability & tooling

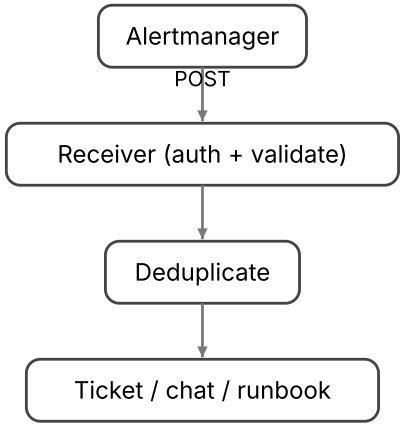
🎙 **Spoken answer · ~71 s**

A small web service, using FastAPI or Flask, receives the JSON payload that Alertmanager posts. It authenticates the sender through a shared secret, basic authentication, mutual TLS or network restrictions, validates the payload against a schema, and responds quickly with a success code, moving slow work to a background task or queue so Alertmanager does not time out and retry unnecessarily. The payload groups alerts with status firing or resolved, labels and annotations, so the handler maps labels like service and severity to actions: create or update a ticket, post to chat, trigger a runbook automation, or open an incident. Deduplication uses the alert fingerprint or group key so repeated notifications update the same item rather than creating duplicates, and resolved alerts close or annotate it. Any automated remediation is limited to safe, idempotent actions with rate limits and a kill switch, and all actions are logged. The receiver itself is monitored, because a silent broken alert path is dangerous, with a dead-man's-switch alert confirming the path works end to end.

Likely follow-ups

- How do you avoid duplicate tickets?
- What is a dead-man's switch alert?

🗺 **Alert webhook**



Receive and validate the payload with authentication, reply fast and process asynchronously, map labels to actions, deduplicate by fingerprint or group key, close on resolved, limit automated remediation and monitor the receiver with a dead-man's switch.

certs.eknathalabs.com

```
@app.post('/alerts')
async def alerts(payload: AlertGroup, bg: BackgroundTasks):
    bg.add_task(handle, payload)
    return {'ok': True}
```

Interview tip: Say dedupe by fingerprint and keep a dead-man's switch.

📚 **Official docs**

- Prometheus: Alertmanager
- Python Documentation

Q87 How do you analyse large log files in Python, including with pandas? Observability & tooling

🎙 **Spoken answer · ~69 s**

I choose the approach by size. For streaming, I read the file line by line, or use generators, parse each line with a compiled regular expression or a JSON parser, and aggregate counts in dictionaries or collections.Counter, which uses constant memory and handles gigabytes. For richer analysis, I parse into a pandas DataFrame, converting timestamps to datetime, setting dtypes, and using categorical types for repeated strings, then resample by time, group by status, endpoint or host, compute percentiles and find top errors. Large files are read in chunks with chunksize, and compressed files are read directly. For bigger data, DuckDB or Polars run SQL or lazy queries over files quickly. I handle malformed lines deliberately by counting and logging them rather than failing. Typical outputs are error rates over time, slowest endpoints, spikes after deployments and top client addresses, shown as tables or charts. For ongoing needs, a log platform such as Loki or Elasticsearch is better than ad-hoc scripts, and scripts are suited to investigation.

Likely follow-ups

- How do you keep memory low on a huge file?
- When is a log platform better than a script?

Stream line by line with compiled regexes and Counters for constant memory, use pandas with explicit dtypes and chunks for richer analysis, use DuckDB or Polars for very large files, count malformed lines and use a log platform for ongoing needs.

```
import re, collections
pat = re.compile(r'" (\d{3}) ')
c = collections.Counter(m.group(1) for line in open('access.log') if (m :=
pat.search(line)))
print(c.most_common())
```

Interview tip: Say stream first, load into pandas only when needed.

certs.eknathalabs.com

📚 **Official docs**

- Python: pathlib
- Python Documentation

Q88 How do you calculate SLIs and SLO burn rates from metrics using Python?

Observability & tooling

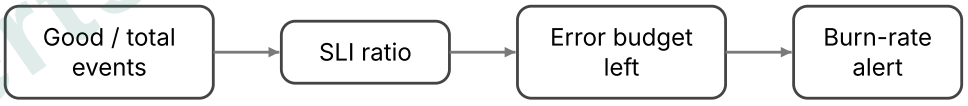
🎙 **Spoken answer · ~78 s**

An SLI is a measured ratio such as successful requests divided by total requests, or the fraction of requests faster than a latency threshold, and the SLO is the target for it over a window, for example 99.9 percent over thirty days. The error budget is the allowed failure, here 0.1 percent. To compute it, I query the metrics backend, for example the Prometheus HTTP API, for good and total event counts over the window, handle gaps and counter resets, and calculate the ratio and the remaining budget. Burn rate is the speed of budget consumption relative to the allowed rate: a burn rate of one uses exactly the budget over the window, while fourteen would exhaust a month in about two days. Multi-window, multi-burn-rate alerts combine a long and a short window, for example one hour and five minutes, to catch fast burns quickly and avoid noisy alerts. A script can produce weekly reports showing attainment, budget remaining and top contributors to errors, used to guide decisions about feature work versus reliability work. In practice, recording rules and tools such as Sloth generate this in Prometheus.

Likely follow-ups

- What does a burn rate of 14 mean?
- Why use two windows in an alert?

🗺 **SLO maths**



Compute good over total events from the metrics API for the SLO window, derive remaining error budget, and calculate burn rate as consumption relative to the allowed rate. Use multi-window burn-rate alerts and weekly reports; recording rules or Sloth can generate it natively.

```
r = requests.get(PROM+'/api/v1/query', params={'query':
'sum(rate(http_requests_total{code!~"5.."}
[30d]))/sum(rate(http_requests_total[30d]))'}).json()
sli = float(r['data']['result'][0]['value'][1])
```

Interview tip: Say burn rate relates consumption to the allowed rate.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Alerting on SLOs
- prometheus_client

Q89 How do you debug a slow or hanging Python process in production? Observability & tooling

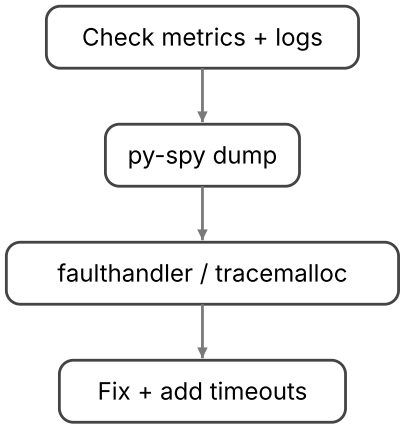
🎙 **Spoken answer · ~74 s**

I begin non-invasively. Check system metrics for CPU, memory, file descriptors and I/O wait, and look at logs and traces for what changed. To see what the process is doing right now, py-spy dumps the stack of every thread of a running process without restarting it or modifying code, and records a flame graph of where time goes, with low overhead. The faulthandler module, enabled at startup, prints tracebacks on a fatal signal or on a timer or a user signal, which is valuable for hangs and deadlocks. For memory growth, tracemalloc compares snapshots to find allocation sites, and objgraph helps find reference cycles or leaks. For async programs, the event loop debug mode and task dumps show blocked coroutines, commonly a blocking call inside async code. Typical causes are missing network timeouts, lock contention, a thread waiting on a queue, unbounded caches and large payloads. Once identified, I reproduce in a test, fix, and add a metric or alert and timeouts so it is visible next time. Attaching a debugger to production is a last resort.

Likely follow-ups

- What does py-spy do without restarting?
- Which tool finds memory allocation sites?

🗺 **Production debugging**



Check system metrics and logs first, use py-spy to dump stacks and flame graphs from a live process, enable faulthandler for hangs, use tracemalloc for memory growth, check for blocking calls in async code and missing timeouts, then reproduce, fix and add alerting.

```
py-spy dump --pid 1234
```

certs.eknathalabs.com

```
py-spy record -o profile.svg --pid 1234 --duration 30
```

```
import faulthandler, signal
faulthandler.register(signal.SIGUSR1)
```

Interview tip: Say py-spy attaches without a restart.

📚 **Official docs**

- py-spy
- Python Documentation

Q90 How do you write a custom exporter or textfile metrics for data that existing exporters do not cover?

Observability & tooling

🎙 **Spoken answer · ~73 s**

When no exporter provides the data, I write a small one. For a long-running exporter, I use prometheus_client with a custom collector class whose collect method queries the source on each scrape and yields metric families, so values are always fresh and not stale from a background loop. It must be fast, apply timeouts to the upstream call, and report a scrape success gauge and duration, so failure to reach the source is visible and not silently showing old numbers. Metric names follow conventions, labels are low cardinality, and units are base units. For batch or cron-driven data, a script writes a file in Prometheus text format to the node exporter textfile directory, written atomically through a temporary file and rename, with a last-run timestamp so alerts can detect staleness. The exporter runs as an unprivileged service, listens on a limited interface and has its own alerts. Before building it, I check whether a community exporter or the blackbox exporter already covers the need. Documentation and tests of the output format help future maintainers.

Likely follow-ups

- Why write the textfile atomically?
- Why report a scrape success metric?

Use a prometheus_client custom collector that queries on scrape with timeouts and exposes scrape success and duration, or write atomically to the node exporter textfile directory with a last-run timestamp. Follow naming and cardinality rules, run unprivileged and check for existing exporters first.

```
from prometheus_client.core import GaugeMetricFamily, REGISTRY
class C:
    def collect(self):
        g = GaugeMetricFamily('queue_depth', 'Depth', labels=['queue'])
        g.add_metric(['jobs'], fetch_depth())
        yield g
REGISTRY.register(C())
```

certs.eknathalabs.com

Interview tip: Say collect on scrape and expose scrape success.

📚 Official docs

- Prometheus: Instrumentation best practices
- Python: pathlib
- prometheus_client

Q91 How do you implement retries with exponential backoff and jitter, and when should you not retry?

Patterns & scenarios

🎙 Spoken answer · ~69 s

Retries recover from transient failures such as timeouts, connection resets, throttling and server errors. The delay between attempts grows exponentially, for example one, two, four, eight seconds, up to a maximum, and jitter adds randomness so many clients do not retry in lockstep and overload a recovering service. I limit total attempts and total elapsed time, honour a Retry-After header, and retry only errors known to be transient: network errors, status 429 and 5xx. I do not retry client errors such as bad requests or authorisation failures, or operations that are not safe to repeat. Operations should be idempotent, or carry an idempotency key, so a retried request that actually succeeded the first time does not duplicate effects. Libraries such as tenacity, backoff, urllib3 Retry and the boto3 retry modes provide this, so I avoid hand-rolled loops. Each retry is logged or counted in a metric, since silent retries hide a degrading dependency, and a circuit breaker stops retries when a dependency is clearly down.

Likely follow-ups

- Why add jitter?
- Which errors should not be retried?

Retry only transient errors with exponential backoff and jitter, capped attempts and elapsed time, respecting Retry-After. Keep operations idempotent, use libraries like tenacity, count retries and combine with a circuit breaker.

```
from tenacity import retry, stop_after_attempt, wait_random_exponential,
retry_if_exception_type
@retry(stop=stop_after_attempt(5), wait=wait_random_exponential(multiplier=1, max=30),
retry=retry_if_exception_type(TransientError))
def call(): ...
```

Interview tip: Say jitter, caps and idempotency.

📚 Official docs

- Microsoft: Cloud design pattern – Retry
- Tenacity
- Requests documentation

certs.eknathalabs.com

Q92 What is a circuit breaker and how would you apply it in Python automation?

Patterns & scenarios

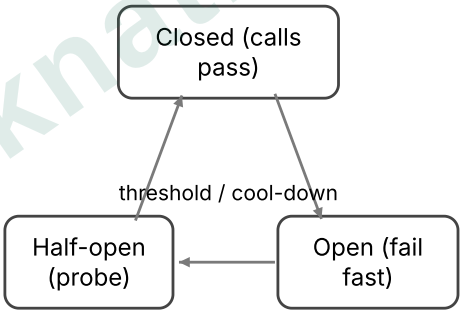
🎙 **Spoken answer · ~70 s**

A circuit breaker protects a caller and a failing dependency. It wraps calls and tracks failures. In the closed state calls go through normally. When failures exceed a threshold within a window, it opens, and calls fail immediately without contacting the dependency, which saves resources, avoids long waits and gives the dependency room to recover. After a cool-down it moves to half-open and lets a few trial calls through: if they succeed, it closes again, and if they fail it reopens. Together with timeouts, retries and bulkheads, it prevents cascading failures where threads pile up waiting on a dead service. In Python, libraries such as pybreaker or purgatory implement it, or it can be built with a small class. When open, the code needs a fallback: return cached data, a default, or a clear error, and report state changes in metrics and logs. Thresholds and cool-down times must be tuned to the dependency. For service-to-service traffic, a service mesh or proxy can also provide the pattern outside application code.

Likely follow-ups

- What happens in the half-open state?
- What should the caller do when the circuit is open?

🗺 **Circuit breaker**



A circuit breaker opens after repeated failures so calls fail fast, then half-opens after a cool-down to test recovery. Use pybreaker, provide fallbacks, tune thresholds, emit state metrics and pair with timeouts and retries.

```
import pybreaker
breaker = pybreaker.CircuitBreaker(fail_max=5, reset_timeout=30)
@breaker
def call_api(): ...
```

Interview tip: Say fail fast, then probe in half-open.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Addressing Cascading Failures
- Microsoft: Cloud design pattern – Circuit Breaker
- Python Documentation

Q93 How do you make resource-creating scripts idempotent?

Patterns & scenarios

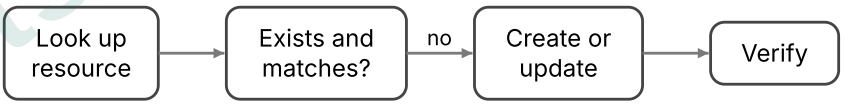
🎙 **Spoken answer · ~73 s**

An idempotent script can run repeatedly and reach the same end state, which matters because jobs are retried, interrupted and run twice. The pattern is to describe the desired state and compare it with the actual state: check whether the resource exists before creating it, update it only if it differs, and do nothing if it already matches. For cloud APIs, use natural identifiers or client tokens, such as an idempotency token on create calls, or deterministic names, and handle already-exists errors by treating them as success after verifying the settings. Use upsert operations where the API offers them, and prefer declarative tools such as Terraform or Ansible for this reason. Record progress in durable state so a partially completed run can resume, and design steps so each is safe to repeat. Deletions treat not-found as success. Avoid appending blindly to files or counters; instead write the whole intended content or use markers. Tests run the script twice and check the second run makes no changes, and a plan or dry-run mode shows intended actions.

Likely follow-ups

- How do client tokens help?
- How do you test idempotency?

🗺 **Idempotent create**



Compare desired and actual state, create only when missing, update only on difference, use deterministic names or client tokens, treat already-exists and not-found as success after verification, record progress and test by running twice.

```
try:
    s3.create_bucket(Bucket=name)
except s3.exceptions.BucketAlreadyOwnedByYou:
    pass
```

Interview tip: Say second run changes nothing.

certs.eknathalabs.com

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)

Q94 How do you handle locking and concurrency control in automation scripts?

Patterns & scenarios

🎙 **Spoken answer · ~71 s**

Two concurrent runs of the same script can corrupt state or duplicate actions, so I use an explicit lock suited to the scope. On one host, fcntl.flock on a lock file is simple and is released automatically if the process dies, unlike a pid file that goes stale. The filelock library offers a portable wrapper. Across hosts, I use a lock in a shared system: a database advisory lock, a Redis key with set-if-not-exists and an expiry, a lease in etcd or Consul, or a cloud service such as DynamoDB conditional writes. Locks need a time-to-live and renewal so a crashed holder does not block forever, and the work should still be idempotent because a lock can expire or be lost during a pause. Fencing tokens let the protected resource reject stale writers. Lock scope should be as small as possible, held for the shortest time, with a timeout for acquisition and clear logging of who holds it. Where possible, I design work queues or leader election so locking is unnecessary.

Likely follow-ups

- Why is flock better than a pid file?
- What problem do fencing tokens solve?

Use flock or filelock on one host and a TTL-based distributed lock such as Redis, etcd or a database across hosts. Keep scope small, set acquisition timeouts, make work idempotent and use fencing tokens against stale holders.

```
from filelock import FileLock, Timeout
try:
    with FileLock('/var/lock/job.lock', timeout=1):
        run()
except Timeout:
    sys.exit('another run is active')
```

Interview tip: Say locks expire, so keep work idempotent.

📚 **Official docs**

- GitHub Actions: Workflow syntax
- Python: concurrent.futures

certs.eknathalabs.com

Q95 How do you manage configuration in Python tools across environments?

Patterns & scenarios

🎙 Spoken answer · ~65 s

I separate code from configuration, following the twelve-factor approach. Defaults live in code, environment-specific values come from environment variables or a config file, and secrets come from a secrets manager. A typed settings object, using pydantic-settings or dataclasses, loads and validates everything at startup, so a missing or malformed value fails immediately with a clear message, not halfway through a run. Precedence is clear and documented: command-line flags override environment variables, which override files, which override defaults. Config files use YAML, TOML or JSON, parsed with safe loaders such as yaml.safe_load, never executing code. Different environments such as development, staging and production use separate files or overlays, with no production values committed to the repository. I avoid reading configuration deep inside functions and instead pass a settings object in, which makes testing easy. Log the effective non-secret configuration at startup to help troubleshooting, and provide a validate or show-config command. Changes to configuration are reviewed like code.

Likely follow-ups

- What is the config precedence order?
- Why validate at startup?

Use typed, validated settings loaded at startup from defaults, files, environment and flags with documented precedence, safe parsers, separate environment files, secrets from a manager, injected settings objects for testing and a show-config command.

```
from pydantic_settings import BaseSettings
class Settings(BaseSettings):
    api_url: str
    timeout: int = 10
s = Settings()
```

Interview tip: Say validate at startup, secrets elsewhere.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q96 How should Python scripts handle secrets safely?

Patterns & scenarios

🎙 Spoken answer · ~70 s

Secrets never belong in source code, command-line arguments, which are visible in process lists, or logs. Preferred sources are workload identity, such as IAM roles, Kubernetes service accounts with cloud federation or OIDC from CI, which give short-lived credentials with no stored secret at all. Where a secret is needed, fetch it at runtime from a manager such as Vault, AWS Secrets Manager or Kubernetes secrets, or read it from environment variables or a mounted file with tight permissions, keeping lifetime in memory short. Use the secrets module, not random, for generating tokens. Rotate credentials regularly and support rotation by re-reading them, and scope them to least privilege. Redact secrets in logs, exceptions and tracebacks, and avoid printing request headers. Use getpass for interactive entry. Compare secrets with hmac.compare_digest to avoid timing leaks. Pre-commit hooks and scanners such as gitleaks catch accidental commits, and a leaked secret is revoked and rotated at once, not just removed from history. Dependencies and containers are checked so images contain no baked-in secrets.

Likely follow-ups

- Why avoid secrets in command-line arguments?
- What is better than a long-lived key?

Prefer workload identity and short-lived credentials; otherwise fetch from a secrets manager or restricted file at runtime, never from code, arguments or logs. Rotate, scope tightly, redact in logs, use the secrets module and compare_digest, scan commits and revoke leaks immediately.

```
import os
token = os.environ['API_TOKEN']
```

```
gitleaks detect --source .
```

Interview tip: Say short-lived credentials beat stored secrets.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q97 How do you implement graceful shutdown in a Python service or worker?

Patterns & scenarios

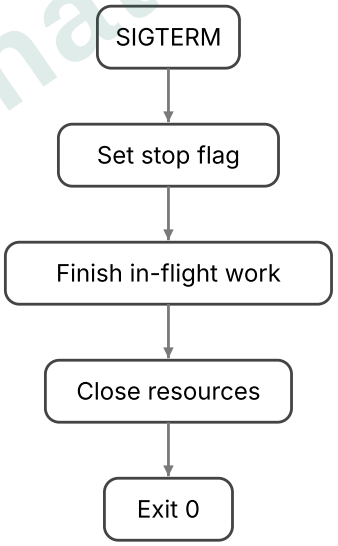
🎙 Spoken answer · ~71 s

When an orchestrator or systemd stops a process, it sends SIGTERM and later SIGKILL after a grace period, so the program should use that window to finish cleanly. I register a signal handler with the signal module, or the event loop's add_signal_handler in asyncio, that sets a shutdown flag rather than doing heavy work in the handler. The main loop checks the flag, stops accepting new work, finishes or checkpoints in-flight items, closes connections, flushes buffers and logs, and exits with a proper code. Workers consuming from a queue stop fetching, complete current messages and acknowledge them, or return unfinished ones for redelivery. Web services fail readiness first so the load balancer drains traffic, then wait for open requests. Thread and process pools are shut down with wait semantics, and timeouts cap the total so the process exits before SIGKILL, which should be configured via terminationGracePeriodSeconds or TimeoutStopSec. Work should be idempotent, since a hard kill can still happen. Testing sends SIGTERM during processing and checks no data is lost.

Likely follow-ups

- Why set a flag instead of working in the signal handler?
- What happens if the grace period expires?

🗺 SIGTERM handling



Handle SIGTERM by setting a flag, stop taking new work, finish or checkpoint in-flight items, acknowledge or return queue messages, close resources and exit within the grace period. Fail readiness first for web services and keep work idempotent in case of SIGKILL.

certs.eknathalabs.com

```
import signal
stop = False
signal.signal(signal.SIGTERM, lambda *a: globals().update(stop=True))
while not stop:
    process_next()
```

Interview tip: Say flag, drain, finish, exit before SIGKILL.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Python Documentation

Q98 How do you work reliably with flaky or rate-limited third-party APIs?

Patterns & scenarios

🎙 **Spoken answer · ~70 s**

I assume failure and design for it. Every call has connect and read timeouts. Transient errors are retried with backoff and jitter, honouring Retry-After for 429 responses, and a circuit breaker stops hammering a dependency that is down. To respect rate limits, I throttle on the client side with a token bucket or a limiter library, spread work over time and use bounded concurrency, instead of bursting and being blocked. Pagination is handled completely with cursors, and large listings use filters and incremental sync with a last-modified marker. Responses are validated against a schema, since APIs change, and unknown fields are tolerated. Results can be cached where freshness allows, using ETags and conditional requests. For writes, idempotency keys prevent duplicates on retry. I log request ids for support tickets, record metrics for latency, errors and throttling, and put API calls behind a thin client class, so behaviour is centralised and easily mocked in tests. For long jobs, checkpoints let me resume rather than restart after a failure.

Likely follow-ups

- How do you respect a rate limit proactively?
- Why wrap the API in a client class?

Use timeouts, retries with backoff and jitter, Retry-After handling and circuit breakers, throttle with a token bucket and bounded concurrency, paginate fully, validate responses, cache with ETags, use idempotency keys, record metrics and checkpoint long jobs behind a thin client class.

```
s = requests.Session()
s.mount('https://', HTTPAdapter(max_retries=Retry(total=5, backoff_factor=1,
status_forcelist=[429,500,502,503,504])))
r = s.get(url, timeout=(3, 10))
```

Interview tip: Say throttle yourself before the server does.

certs.eknathalabs.com

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling
- Requests documentation

Q99 How do you process large batches of items safely, with chunking, progress and failure handling?

Patterns & scenarios

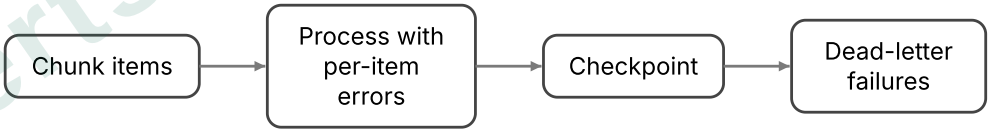
🎙 **Spoken answer · ~69 s**

Large jobs fail partway, so the design must handle that. I process items in bounded chunks rather than loading everything into memory, using generators and itertools batching, and apply concurrency within limits. Each item is processed independently, with errors caught per item and recorded, with the item id and reason, so one bad record does not abort a million good ones, though a failure-rate threshold stops the run if something is systematically wrong. Progress is logged and exposed as metrics, such as processed, failed and remaining counts. Checkpointing stores the last completed item or batch in durable storage so a rerun resumes, and idempotent processing makes reprocessing safe. Failed items go to a dead-letter file or queue for inspection and later retry. Bulk APIs reduce call counts, and rate limits are respected. At the end, the job prints a summary with totals and exits non-zero if failures exceed tolerance. A dry-run option and a limit option help test on a small sample before a full run.

Likely follow-ups

- Why record failed items individually?
- What is a failure-rate threshold for?

🗺 **Batch processing**



Process in bounded chunks with per-item error capture, a failure-rate circuit to stop systematic problems, progress metrics, durable checkpoints, idempotent processing, a dead-letter store for failures, a summary at the end and dry-run and limit options.

```
from itertools import islice
def batches(it, n):
    it = iter(it)
    while chunk := list(islice(it, n)):
        yield chunk
```

```
python migrate.py --dry-run --limit 100
```

certs.eknathalabs.com

Interview tip: Say checkpoints and a dead-letter list.

📚 **Start here (official docs)**

- Python documentation

Q100 What common anti-patterns do you see in Python operations scripts, and how do you fix them?

Patterns & scenarios

🎙 **Spoken answer · ~68 s**

Frequent problems include shell=True with string-built commands, which invites injection and quoting bugs; ignoring return codes and swallowing exceptions with a bare except, which hides failures; no timeouts on network calls, so scripts hang; hard-coded credentials, hosts and paths; mutable default arguments; parsing text output of tools where a library or JSON output exists; global state and everything in one huge file with no functions or tests; printing instead of logging; no dry-run for destructive actions; assuming the happy path with no pagination, so only the first page of results is processed; time handling in local time instead of UTC; installing into the system Python; and no idempotency or locking. Fixes follow the practices discussed: argument lists with check=True and timeouts, specific exceptions, configuration and secrets outside code, structured logging, a main function with argparse, type hints and tests, virtual environments with pinned dependencies, dry-run and confirmation, and code review. A linter and formatter in CI, such as ruff, catch many of these automatically.

Likely follow-ups

- Why avoid shell=True?
- What does a bare except hide?

Common problems are shell=True, ignored errors and bare except, missing timeouts, hard-coded config and secrets, no pagination, local time, no dry-run, no tests and system-wide installs. Fix them with argument lists, specific exceptions, config outside code, logging, pagination, UTC, tests, virtual environments and ruff in CI.

```
subprocess.run(['systemctl', 'restart', svc], check=True, timeout=60)
```

```
ruff check . && ruff format --check .
```

Interview tip: Say lint in CI catches most of them.

📚 **Official docs**

- Microsoft: Anti-patterns (performance antipatterns)
- Python Documentation

certs.eknathalabs.com

Q101 What is the Python data model and how do dunder methods make your own classes behave like built-ins?

Core language

🎙 Spoken answer · ~71 s

The data model is the set of special methods, written with double underscores, that Python calls to implement language features. Defining them lets your classes integrate with built-in syntax. The repr and str methods control how an object prints, which is vital for useful logs and debugging; len, getitem and iter make a class sized, indexable and iterable; eq and hash control equality and use in sets and dictionaries; enter and exit make it a context manager; call makes an instance callable; and lt and related methods enable sorting. For operations tooling, a small class representing a host, a resource or a time window becomes much easier to use when it prints clearly and compares naturally. Dataclasses generate many of these for you. Care is needed with hash and equality: objects that compare equal must have equal hashes, and mutable objects used as dictionary keys cause bugs. Implementing the protocol methods, rather than inventing custom method names, makes code idiomatic and compatible with standard library functions like sorted, sum and any.

Likely follow-ups

- Why must equal objects have equal hashes?
- Which methods make an object iterable?

Dunder methods such as repr, len, iter, eq, hash, enter and exit let custom classes work with built-in syntax and functions. Use clear repr for logs, keep eq and hash consistent, and let dataclasses generate the boilerplate.

```
class Host:
    def __init__(self, name): self.name = name
    def __repr__(self): return f'Host({self.name!r})'
    def __eq__(self, o): return self.name == o.name
    def __hash__(self): return hash(self.name)
```

Interview tip: Say implement the protocols instead of custom method names.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q102 How do itertools and functools help in everyday operations scripts?

Core language

🎙 Spoken answer · ~68 s

These standard modules provide efficient building blocks that replace hand-written loops. From itertools, chain joins several iterables lazily, islice takes a slice from any iterator, which suits reading the first lines of a huge file, batched or a small helper splits work into chunks for API calls, groupby groups consecutive items after sorting, product and combinations generate test matrices, and takewhile and dropwhile process streams until a condition changes. Because they are lazy, they handle data larger than memory. From functools, lru_cache memoises expensive pure functions such as repeated DNS or metadata lookups, partial pre-fills arguments to create specialised callbacks, reduce folds a sequence, wraps preserves metadata when writing decorators, and cached_property computes an attribute once per instance. Singledispatch provides function overloading by type. Using them makes code shorter and often faster, and communicates intent. The caveats are that groupby needs sorted input, lru_cache needs hashable arguments and can retain memory, and cached results must not hide data that changes over time.

Likely follow-ups

- Why does groupby need sorted input?
- What is the risk of lru_cache for changing data?

Use itertools chain, islice, batched and groupby for lazy stream processing, and functools lru_cache, partial, wraps and cached_property to cache and specialise functions. Remember groupby needs sorted data and caches can serve stale results.

```
from itertools import islice
first = list(islice(open('big.log'), 10))
```

```
from functools import lru_cache
@lru_cache(maxsize=1024)
def resolve(host): ...
```

Interview tip: Say lazy iteration keeps memory flat.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q103 How do references, shallow copies and deep copies work in Python, and what bugs do they cause?

Core language

🎙 Spoken answer · ~73 s

Variables in Python are names bound to objects, so assignment copies the reference, not the object. Two names can point to the same list, and changing it through one changes what the other sees. Passing a list or dictionary to a function lets that function modify the caller's data, which surprises people. A shallow copy, made with the copy method, slicing or the copy module, creates a new container but shares the nested objects, so modifying an inner list still affects both. A deep copy recursively duplicates everything, at a cost in time and memory, and can fail or misbehave for objects holding connections or locks. Common bugs include mutating shared default arguments, building a list of lists with multiplication, which repeats the same inner list, modifying a configuration dictionary meant to be a template, and accidentally changing data while iterating over it. Safer patterns are to treat inputs as read-only, return new objects, use tuples or frozen dataclasses for immutable data, and copy deliberately at the boundary where a function needs its own version.

Likely follow-ups

- Why does a list of lists made with multiplication misbehave?
- When is a deep copy a problem?

Names bind to objects, so assignment shares them. Shallow copies share nested objects, deep copies duplicate everything at a cost. Avoid mutating inputs and shared defaults, prefer immutable data and copy deliberately at boundaries.

```
import copy
b = copy.deepcopy(config)
```

```
grid = [[0]*3 for _ in range(3)]
```

Interview tip: Say assignment copies the reference, not the object.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q104 Which collections module types do you use in operations code, such as Counter, defaultdict, deque and OrderedDict?

Core language

🎙 Spoken answer · ~66 s

The collections module offers containers that remove a lot of boilerplate. Counter counts hashable items and provides most_common, which is ideal for top error codes, busiest clients or most frequent log messages from a stream. Defaultdict creates missing entries automatically with a factory, so grouping items by key needs no existence check, such as collecting instances by region. Deque is a double-ended queue with fast appends and pops at both ends, and a maxlen argument makes a fixed-size sliding window, useful for the last N log lines or a moving average. Namedtuple creates lightweight immutable records, though dataclasses are often clearer. ChainMap layers several dictionaries without copying, which models configuration precedence of defaults, file and environment. OrderedDict matters less since normal dictionaries preserve insertion order, though it still offers move_to_end for LRU-style structures. Choosing the right container makes code shorter and faster, because operations like pop from the front of a list are slow compared with a deque.

Likely follow-ups

- Why use deque instead of a list for a queue?
- How does ChainMap model config precedence?

Counter counts items, defaultdict groups without checks, deque gives fast ends and fixed-size windows with maxlen, ChainMap layers config sources, and namedtuple or dataclasses model records. The right container beats manual loops.

```
from collections import Counter, defaultdict, deque
top = Counter(codes).most_common(5)
by_region = defaultdict(list)
last = deque(maxlen=100)
```

Interview tip: Say Counter for top-N, deque for sliding windows.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q105 How should you handle dates, times and time zones correctly in Python?

Core language

🎙 Spoken answer · ~70 s

Time bugs are common in operations code, so I follow firm rules. Store and compute in UTC, convert to a local zone only for display. Use timezone-aware datetime objects: a naive datetime has no zone and is ambiguous, and mixing naive and aware values raises errors or silently gives wrong results. The zoneinfo module in the standard library provides IANA zones, and datetime.now with a UTC zone replaces the deprecated utcnow that returns a naive value. Parse ISO 8601 strings with fromisoformat, which handles offsets, and format with isoformat. For durations use timedelta, not manual arithmetic on seconds, and for monotonic measurement of elapsed time, such as timeouts and benchmarks, use time.monotonic because the wall clock can jump with adjustments. Daylight saving changes make local-time schedules tricky, with skipped and repeated hours, so scheduling should be in UTC or handled by tools that understand zones. APIs and logs should carry timestamps with explicit offsets, and tests should freeze the clock rather than depend on the real time.

Likely follow-ups

- Why use monotonic for timeouts?
- What is wrong with a naive datetime?

Work in UTC with timezone-aware datetimes, use zoneinfo for display, parse and emit ISO 8601, use timedelta for durations and time.monotonic for elapsed time, and beware daylight saving in local schedules. Freeze time in tests.

```
from datetime import datetime, timezone
now = datetime.now(timezone.utc)
print(now.isoformat())
```

```
start = time.monotonic(); do(); print(time.monotonic()-start)
```

Interview tip: Say UTC inside, local only at the edges.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q106 What are entry points and console_scripts, and how do they differ from running python script.py?

Packaging & envs

🎙 **Spoken answer · ~69 s**

Entry points are metadata declared in a package's configuration that other tools can discover. The console_scripts group tells the installer to generate an executable wrapper in the environment's bin directory, so a function in your package becomes a normal command such as mytool. The wrapper imports the module and calls the function, using the right interpreter and environment, which avoids path problems, shebang issues and the need to copy scripts around. Running python script.py depends on the current directory and the interpreter on the path, and the script cannot easily import its own package without path hacks. With entry points, the code is properly installed, versioned and testable, dependencies are declared, and the same command works in every environment, including pipx installs. Other groups allow plugins: a package can register an extension under a named group, and the host application loads it dynamically through importlib.metadata. In pyproject.toml, the scripts table defines console commands. This is how well-behaved tools, such as pytest plugins, are wired together.

Likely follow-ups

- Where are entry points declared?
- How do plugins use entry point groups?

Entry points declared in pyproject.toml create installed command wrappers with the correct interpreter and dependencies, and register plugins under named groups loaded via importlib.metadata. This replaces fragile python script.py usage.

```
[project.scripts]
mytool = "mytool.cli:main"
```

```
pipx install .
```

```
python -c "from importlib.metadata import entry_points;
print(entry_points(group='console_scripts'))"
```

Interview tip: Say installable commands beat loose scripts.

📚 **Official docs**

- Python Documentation

certs.eknathalabs.com

Q107 How do you use mypy or pyright in CI and adopt typing gradually in an existing automation codebase?

Packaging & envs

🎙 Spoken answer · ~72 s

Static type checkers find a class of bugs before runtime, but a legacy codebase cannot be fully typed overnight, so adoption is incremental. I start by adding a checker to CI with a lenient configuration that ignores missing imports and untyped functions, so it passes on day one. Then I tighten step by step: type new code fully, enable stricter options per module through configuration overrides, and annotate the most important boundaries first, such as public functions, data structures and API clients. Dataclasses, TypedDict and Protocol give structure to dictionaries and duck-typed objects. Third-party libraries without annotations use stub packages. The any type is a temporary escape and I track and reduce it. The checker runs in pre-commit and CI as a required check, with a baseline so existing errors do not block while new ones do. Benefits include safer refactors, better editor completion and living documentation. I balance value against effort, avoiding contortions for dynamic code, and use runtime validation with pydantic for external input, since types are not enforced at runtime.

Likely follow-ups

- Why is incremental adoption better?
- What does a type checker not guarantee?

Add mypy or pyright to CI with a lenient config, type new code and key boundaries first, tighten per module, use TypedDict, Protocol and stubs, track Any, and require the check in pre-commit and CI. Types are not enforced at runtime, so validate external input.

```
mypy src --ignore-missing-imports
```

```
[tool.mypy]
strict = false
[[tool.mypy.overrides]]
module = "mytool.core.*"
disallow_untyped_defs = true
```

Interview tip: Say gradual typing, strict where it counts.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q108 How do you handle Python packages with C extensions or system dependencies, such as psycopg2, cryptography or lxml?

Packaging & envs

🎙 Spoken answer · ~71 s

Packages with native code need compiled artefacts. The easiest route is binary wheels: modern versions of cryptography, lxml, numpy and psycopg publish prebuilt wheels for major platforms, so pip installs them with no compiler. The problems arise on unusual platforms, such as Alpine Linux with musl, new architectures like ARM, or very new Python versions where wheels are not yet published, and then pip builds from source and needs headers and a compiler, with failures like missing libpq or openssl development files. Solutions include choosing a base image with wheel support, installing the required system packages and build tools in the image build stage and discarding them with a multi-stage build, using the binary variants of packages such as psycopg binary for convenience, and pinning versions. For production images, I prefer to build wheels once in CI, store them in an internal index and install from there, giving reproducible and fast installs. Documentation lists system requirements, and tests run on the same base image as production to catch missing libraries early.

Likely follow-ups

- Why does Alpine cause build issues?
- How does a multi-stage build help?

Prefer prebuilt binary wheels; when building from source install headers and compilers in a builder stage, use a multi-stage image, build wheels in CI into an internal index, pin versions and test on the production base image. Alpine and new platforms are common pain points.

```
pip install --only-binary=:all: cryptography
```

```
pip wheel -r requirements.txt -w wheelhouse/ && pip install --no-index --find-links
wheelhouse -r requirements.txt
```

Interview tip: Say build wheels once, install everywhere.

📚 Official docs

- Python: venv
- Python Documentation
- Python Packaging User Guide

certs.eknathalabs.com

Q109 How do you diagnose and resolve dependency conflicts in Python projects?

Packaging & envs

🎙 Spoken answer · ~71 s

Conflicts happen when two packages require incompatible versions of a third. Pip reports the resolution failure or installs a combination with warnings. To diagnose, I use pip check to list broken requirements, pip show and pipdeptree to display who depends on what, and the resolver's verbose output to see which constraints clash. Fixes depend on the cause. I can relax an overly strict pin in my own requirements, upgrade the packages involved to versions that agree, replace a library that is unmaintained and holds back others, or isolate incompatible tools in separate environments, for example installing command-line tools with pipx so they do not share dependencies with the application. Constraints files keep transitive versions consistent across several requirement sets. Lock files from pip-tools, Poetry or uv record a tested, conflict-free set, and updates are done deliberately by a bot, with CI testing the result. Upper bounds on dependencies are avoided in libraries but used for applications through locks. Keeping the dependency tree small prevents many conflicts in the first place.

Likely follow-ups

- Which tools show the dependency tree?
- Why isolate tools with pipx?

Use pip check, pipdeptree and verbose resolver output to find the clash, then relax pins, upgrade, replace unmaintained packages or isolate tools in separate environments. Lock files and constraints files keep a tested set; keep the dependency tree small.

```
pip check
```

```
pipdeptree -r -p urllib3
```

```
uv pip compile requirements.in -o requirements.txt
```

Interview tip: Say read the tree, then relax, upgrade or isolate.

📚 Official docs

- Python: venv
- Python Documentation

certs.eknathalabs.com

Q110 How do you share common code across several internal Python tools: monorepo, shared library or copy?

Packaging & envs

🎙 Spoken answer · ~70 s

Copying code leads to divergence and fixes that never reach every tool, so I prefer a shared library with a clear boundary. In a monorepo, a common package lives beside the tools and is installed in editable mode during development, so changes are visible immediately, with one set of tests and a single lint configuration. Workspaces in tools like uv or Poetry manage the relationships, and CI builds only what changed. In separate repositories, the shared library is published to an internal package index with semantic versions, and tools pin compatible ranges, giving independent release cycles at the cost of coordination and upgrade work. I keep the library small and stable, with logging, configuration, API clients and retry helpers, and avoid turning it into a dumping ground. Public interfaces are documented and tested, deprecations come with warnings and a transition period, and breaking changes need a major version. A changelog and automated dependency updates help consumers keep current. The choice depends on team size and release independence.

Likely follow-ups

- What belongs in a shared library?
- How do you handle breaking changes?

Prefer a small, well-tested shared library over copying: in a monorepo use editable installs and workspaces; across repos publish semantic versions to an internal index. Keep the interface stable, deprecate gracefully and automate dependency updates.

```
pip install -e ./libs/opscommon
```

```
uv sync --all-packages
```

Interview tip: Say small stable library, semantic versions.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q111 When and how do you use SQLite from Python for local state in automation tools?

Files & data

🎙 Spoken answer · ~69 s

SQLite is a file-based database in the standard library, ideal for small tools that need structured local state without running a server. Examples are tracking which items a batch job has processed, caching API results with timestamps, storing inventory snapshots for querying and recording run history. It supports SQL, transactions and indexes, and handles concurrent readers with a single writer. Good practice is using parameterised queries to avoid injection, wrapping writes in transactions via a context manager, enabling write-ahead logging for better concurrency, setting a busy timeout so writers wait rather than fail, and keeping the database file on local disk, because network filesystems can corrupt it. Schema changes need a simple migration approach, such as a version table. A connection should not be shared across threads without care. Compared with JSON files, SQLite is safer for partial writes and crashes, and queries scale better. It is not suitable for many concurrent writers or multi-host access, where PostgreSQL or a managed store is the right choice.

Likely follow-ups

- Why avoid SQLite on network filesystems?
- What does write-ahead logging improve?

SQLite gives transactional, queryable local state for checkpoints, caches and run history. Use parameterised queries, transaction context managers, WAL mode and busy timeouts, keep it on local disk and move to a server database for many writers or hosts.

```
import sqlite3
con = sqlite3.connect('state.db')
con.execute('PRAGMA journal_mode=WAL')
with con:
    con.execute('INSERT OR REPLACE INTO done(id) VALUES (?)', (item,))
```

Interview tip: Say transactions and parameters, local disk only.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q112 How do you write files atomically and avoid corruption when a script crashes mid-write?

Files & data

🎙 Spoken answer · ~70 s

Writing directly to a target file risks leaving it half written if the process is killed, the disk fills or the machine loses power, and readers may see partial content. The safe pattern is to write the new content to a temporary file in the same directory, flush and fsync it so the data reaches the disk, and then rename it over the target using os.replace, which is atomic on the same filesystem. Readers see either the complete old file or the complete new one. The temporary file must be on the same filesystem as the target, otherwise the rename becomes a copy. Permissions and ownership should be set on the temporary file before the rename. For critical data, fsync the containing directory as well. The tempfile module with the delete option off and a named temporary file in the target directory makes this easy, and libraries such as atomicwrites package the pattern. For logs and databases, append-only writes or transactional stores are more appropriate than rewriting files.

Likely follow-ups

- Why must the temp file be in the same directory?
- What does os.replace guarantee?

🗺 Atomic write



Write to a temp file in the same directory, flush and fsync, set permissions, then os.replace over the target for an atomic swap; readers see old or new, never partial. Use transactional stores for frequent updates.

```
import os, tempfile
fd, tmp = tempfile.mkstemp(dir=os.path.dirname(path))
with os.fdopen(fd, 'w') as f:
    f.write(data); f.flush(); os.fsync(f.fileno())
os.replace(tmp, path)
```

Interview tip: Say temp file, fsync, atomic rename.

📚 Official docs

- Python: pathlib

certs.eknathalabs.com

Q113 How do you generate configuration files from templates using Jinja2 in Python?

Files & data

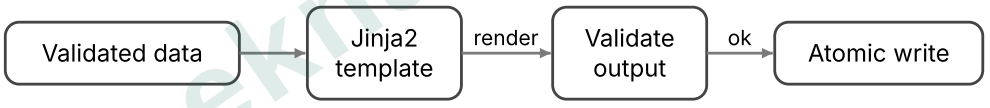
🎙 **Spoken answer · ~73 s**

Jinja2 renders text templates with variables, loops and conditions, and is a good fit for generating configuration for services like Nginx, systemd units or Prometheus rules. I load templates from a directory with a file system loader and create an environment with strict undefined behaviour, so a missing variable raises an error instead of producing an empty string and a broken file. Autoescape is off for non-HTML files but on for HTML. Whitespace control options such as trim_blocks and lstrip_blocks keep the output tidy. Data comes from validated structures, ideally typed through pydantic or dataclasses, so templates stay simple and logic remains in Python. Rendered output is validated before use, for example by running the target program's config test command, and written atomically. Custom filters add reusable formatting. A dry-run mode prints a diff against the existing file. Templates are version-controlled with the code and tested by rendering sample data and comparing to expected output. User-supplied template content should not be rendered, because template injection is a risk, so sandboxed environments are used if needed.

Likely follow-ups

- Why use strict undefined?
- How do you validate rendered configs?

🗺 **Config generation**



Load templates with a Jinja2 environment using StrictUndefined and whitespace trimming, feed validated data, keep logic in Python, validate output with the target's config test, write atomically, show diffs in dry-run and use a sandbox for untrusted templates.

```
from jinja2 import Environment, FileSystemLoader, StrictUndefined
env = Environment(loader=FileSystemLoader('templates'), undefined=StrictUndefined,
trim_blocks=True, lstrip_blocks=True)
out = env.get_template('nginx.conf.j2').render(vhosts=vhosts)
```

Interview tip: Say strict undefined so mistakes fail loudly.

📚 **Official docs**

- Python: pathlib
- Python Documentation

certs.eknathalabs.com

Q114 How do you watch files or directories for changes in Python, for example to reload configuration?

Files & data

🎙 Spoken answer · ~72 s

There are two approaches: polling and event notifications. Polling periodically checks modification times or content hashes, which is simple and portable but has latency and cost, and is fine for a handful of files checked every few seconds. Event-based watching uses operating system facilities such as inotify on Linux through libraries like watchdog or watchfiles, delivering notifications when files are created, modified, moved or deleted. Events can be noisy: editors write temporary files and rename, one save may trigger several events, and events can arrive before a write finishes, so I debounce them and re-read the file after a short quiet period. For configuration reload, I parse and validate the new content first, and only swap it in if valid, keeping the old configuration and logging an error otherwise. Signals such as SIGHUP are a conventional alternative, letting an operator trigger a reload deliberately. On Kubernetes, mounted config maps update through symlink swaps, so watching the directory is more reliable than watching a single file path. Watchers must be stopped cleanly on shutdown.

Likely follow-ups

- Why debounce file events?
- Why validate before swapping config?

Poll mtimes or hashes for simplicity, or use inotify through watchdog or watchfiles; debounce noisy events, validate new content before swapping, consider SIGHUP, and watch the directory for Kubernetes config map symlink swaps.

```
from watchfiles import watch
for changes in watch('/etc/app'):
    reload_config()
```

Interview tip: Say debounce, validate, then swap.

📚 Official docs

- Python: pathlib
- py-spy
- Python Documentation

certs.eknathalabs.com

Q115 How do you read and write Parquet, Excel and other binary data formats from Python scripts?

Files & data

🎙 Spoken answer · ~70 s

For analytics-style data, binary formats are more compact and faster than CSV and preserve types. Parquet is a columnar format that compresses well and supports efficient column selection and predicate pushdown; pandas, Polars, PyArrow and DuckDB read and write it, and it suits cost exports, inventory snapshots and log summaries. For spreadsheets, openpyxl reads and writes xlsx files with formatting, formulas and multiple sheets, and pandas wraps it for simple dataframe export, while xlsxwriter is a fast write-only alternative with charts. These are useful when the audience is non-technical and wants a file they can open. Good practice includes specifying schemas and dtypes, using compression, partitioning large datasets by date, avoiding loading everything into memory by reading column subsets or batches, and never trusting macros or formulas from untrusted spreadsheets. Excel files from users can contain surprises such as merged cells, dates stored as numbers and hidden sheets, so validation is important. Sensitive data should be protected at rest, and exports should record generation time and source.

Likely follow-ups

- Why is Parquet better than CSV for analytics?
- What problems appear in user-supplied spreadsheets?

Use Parquet with pandas, Polars, PyArrow or DuckDB for compact typed columnar data, and openpyxl or xlsxwriter for spreadsheets with formatting. Specify dtypes, compress, partition, read subsets, validate user files and protect sensitive exports.

```
import pandas as pd
df.to_parquet('inventory.parquet', compression='zstd')
pd.read_parquet('inventory.parquet', columns=['id', 'region'])
```

```
df.to_excel('report.xlsx', index=False, sheet_name='Summary')
```

Interview tip: Say Parquet for machines, Excel for humans.

📚 Official docs

- Python Documentation
- Ruff documentation

certs.eknathalabs.com

Q116 How do you produce readable terminal output with rich, tables, progress bars and colours?

CLI & scripting

🎙 Spoken answer · ~69 s

The rich library makes command-line output clear without much code. It prints coloured, styled text, renders tables with aligned columns, shows progress bars with elapsed and remaining time, displays syntax-highlighted code and formatted tracebacks, and can render Markdown. For operations tools, a table of resources with status colours is far easier to scan than raw lines, and a progress bar assures users that a long job is alive. Good practice is to respect the environment: detect whether output is a terminal and disable colours and animation when it is redirected to a file or pipe, honour the NO_COLOR convention, and provide a machine-readable mode, for example JSON output, so scripts can consume results without parsing decorated text. Human output goes to standard output when it is the result, while progress and diagnostics go to standard error. Logs use the logging module, optionally with a rich handler for development. In continuous integration, plain output avoids control characters polluting logs. Keeping presentation separate from logic makes testing easier.

Likely follow-ups

- Why offer JSON output as well?
- What should happen when output is piped?

Use rich for tables, colours and progress bars, but detect non-terminals and NO_COLOR, send progress to stderr, offer JSON output for scripts and keep presentation separate from logic.

```
from rich.console import Console
from rich.table import Table
t = Table('Name', 'Status')
t.add_row('web01', '[green]ok[/green]')
Console().print(t)
```

Interview tip: Say pretty for humans, JSON for machines.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q117 How do you handle interactive prompts and secret input safely in command-line tools?

CLI & scripting

🎙 Spoken answer · ~68 s

Interactive input should be used sparingly and never in a way that breaks automation. For secrets, the getpass module reads input without echoing it, and click and typer offer a prompt with hide_input. Secrets should not be accepted as command-line arguments, because they appear in shell history and in the process list for all users; instead accept them from environment variables, files with restricted permissions, standard input or a secrets manager. Confirmation prompts protect destructive actions, but a tool must also support non-interactive use with an explicit yes flag, and should detect when standard input is not a terminal and fail with a clear message rather than hang waiting. Prompts should offer sensible defaults and validate answers, repeating on bad input. Sensitive values are never logged, and are cleared from memory where practical, although Python offers limited guarantees. For operations tools run by humans, a typed confirmation, such as entering the environment name before a production change, reduces mistakes more than a simple yes.

Likely follow-ups

- Why not pass secrets as arguments?
- How do you support both interactive and CI use?

Use getpass or hide_input prompts, never accept secrets as arguments, offer env vars, files, stdin or secrets managers, add --yes for non-interactive runs, detect missing terminals and require typed confirmation for production.

```
import getpass, sys
if not sys.stdin.isatty(): sys.exit('no terminal; use --token-file')
token = getpass.getpass('API token: ')
```

Interview tip: Say secrets never go in argv.

📚 Official docs

- Python: argparse
- Python: subprocess

certs.eknathalabs.com

Q118 How do you add shell completion and good help text to a Python command-line tool?

CLI & scripting

🎙 Spoken answer · ~68 s

Completion and clear help make a tool pleasant and discoverable. Frameworks such as click and typer generate completion scripts for bash, zsh and fish, and argcomplete adds completion to argparse programs; users install the generated script into their shell configuration, and dynamic completions can query live data, such as environment or cluster names, though they must be fast and fail quietly. Help text is written for operators: a one-line summary for each command, descriptions for each option with defaults and allowed values, examples in the epilog, and consistent naming of flags, with long and short forms. Subcommands group related actions, and the help output shows them clearly. Error messages say what went wrong and how to fix it, and exit codes are documented. Version output includes the tool version and, optionally, dependency information for bug reports. A man page or markdown reference can be generated from the command definitions so documentation never drifts from the code. These details reduce support questions and mistakes.

Likely follow-ups

- How does argcomplete work with argparse?
- What belongs in the epilog?

click and typer generate completion for bash, zsh and fish, and argcomplete covers argparse. Write clear one-line summaries, option descriptions with defaults, examples, helpful errors and documented exit codes, and generate reference docs from the definitions.

```
eval "$(_MYTOOL_COMPLETE=bash_source mytool)"
```

```
parser = argparse.ArgumentParser(epilog='Example: mytool deploy --env staging')
```

Interview tip: Say help and completion are part of the product.

📚 Official docs

- Python: argparse
- Python: subprocess
- Python Documentation

certs.eknathalabs.com

Q119 How do you structure subcommands and a plugin architecture in a larger CLI tool?

CLI & scripting

🎙 Spoken answer · ~68 s

A growing tool needs structure so it stays maintainable. Subcommands group actions under one executable, like git does, with click groups or typer apps, argparse subparsers, each command in its own module and a thin main that registers them. Shared options such as verbosity, configuration file and output format are defined once and passed through a context object holding the loaded configuration and clients, which also makes testing easier. Business logic lives in separate functions or classes with no dependency on the CLI framework, so it can be called from tests, other tools and a web interface. For extensibility, a plugin mechanism lets teams add commands without changing the core: packages register commands under an entry point group, and the core discovers them at start-up through importlib.metadata. Lazy loading keeps start-up fast by importing a command's module only when it runs. Consistent conventions for naming, flags, exit codes and output formats across all commands give a uniform experience, enforced by tests and code review.

Likely follow-ups

- Why keep business logic out of command functions?
- How do plugins register commands?

Use click groups or typer apps with one module per command, a shared context for config and clients, framework-independent business logic, entry-point plugins discovered with importlib.metadata, lazy imports for speed and consistent conventions across commands.

```
@click.group()
@click.option('--env', default='staging')
@click.pass_context
def cli(ctx, env): ctx.obj = Context(env)
@cli.command()
@click.pass_obj
def status(ctx): ...
```

Interview tip: Say thin commands, fat library.

📚 Official docs

- Python: argparse
- Python: subprocess

certs.eknathalabs.com

Q120 How do you manage child processes and signals so a Python tool cleans up after itself?

CLI & scripting

🎙 Spoken answer · ~68 s

A tool that starts subprocesses must make sure they do not outlive it or leak resources. With subprocess, I keep a reference to each Popen object, use context managers where possible, and set timeouts on waits. When the tool receives SIGINT or SIGTERM, a handler sets a flag or raises an exception so the main flow reaches its cleanup code in finally blocks, which terminates children, first politely with terminate and then with kill after a grace period. Starting children in their own process group, with start_new_session, allows signalling the whole group so grandchildren die too. Without that, a shell wrapper may leave orphans running. Temporary files and locks are removed in finally or through context managers and atexit handlers, though a hard kill bypasses them, so stale lock detection is needed. Exit codes reflect the cause, for example 130 for interrupt, so callers can react. Avoid shell=True and pass argument lists. Testing sends signals to a running instance and verifies no processes remain.

Likely follow-ups

- Why start children in a new process group?
- What can atexit not handle?

Track Popen objects with timeouts, handle SIGINT and SIGTERM by setting a flag and cleaning up in finally, terminate then kill children, use process groups to reach grandchildren, remove temp files and locks and use meaningful exit codes. atexit cannot survive SIGKILL.

```
p = subprocess.Popen(cmd, start_new_session=True)
try:
    p.wait(timeout=600)
finally:
    if p.poll() is None:
        os.killpg(p.pid, signal.SIGTERM)
```

Interview tip: Say terminate, then kill, the whole process group.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q121 How do you implement OAuth2 client credentials and token refresh in a Python API client?

APIs & cloud SDKs

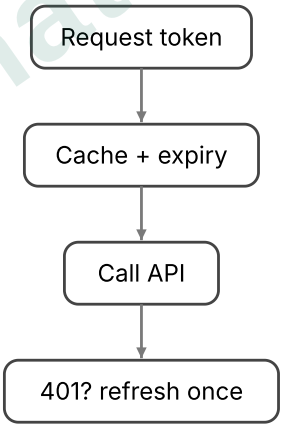
🎙 Spoken answer · ~70 s

Service-to-service integrations commonly use the OAuth2 client credentials flow: the client posts its id and secret to the token endpoint and receives a short-lived access token, which it sends as a bearer token on API calls. A robust client caches the token in memory together with its expiry, refreshes it shortly before it expires, instead of requesting one per call, and handles a 401 response by fetching a new token once and retrying the request. The refresh logic must be thread-safe, using a lock so concurrent workers do not all request tokens at once. Libraries such as requests-oauthlib and authlib implement the flow, and httpx supports custom auth classes that attach tokens automatically. Client secrets come from a secrets manager or environment, never source code, and are never logged. Where the platform offers workload identity, such as cloud metadata or federated tokens, that is preferable because no long-lived secret exists. Scopes are requested minimally. Tests use a fake token endpoint to cover expiry, refresh failure and revoked credentials.

Likely follow-ups

- Why cache and pre-refresh the token?
- What do you do on a 401?

🗺 Token handling



Use client credentials: post id and secret for a short-lived token, cache it with expiry, refresh before it lapses under a lock, retry once on 401, keep secrets in a manager, prefer workload identity and request minimal scopes. Libraries like authlib or httpx auth classes help.

```
class TokenAuth(httpx.Auth):
    def auth_flow(self, request):
        request.headers['Authorization'] = f'Bearer {self.token()}'
        yield request
```

Interview tip: Say cache, pre-refresh and lock the token.

certs.eknathalabs.com

📚 **Official docs**

- OAuth 2.0 / OIDC overview (OpenID)
- Python: argparse
- Python Documentation

Q122 How do you use httpx and async clients to call many endpoints efficiently?

APIs & cloud SDKs

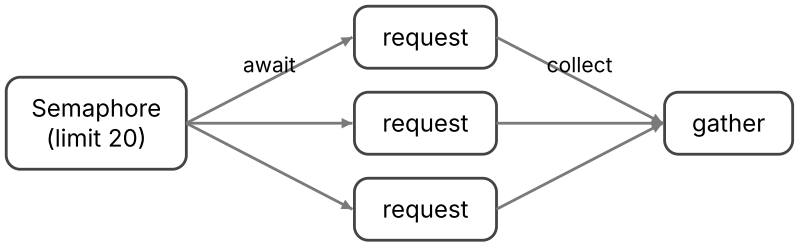
🎙 **Spoken answer · ~70 s**

Httpx offers a requests-like interface with both synchronous and asynchronous clients, HTTP/2 support and strict timeouts by default. For many calls, I create one AsyncClient and reuse it, since it keeps a connection pool and avoids repeated TCP and TLS handshakes. Requests are launched concurrently with asyncio gather or a task group, but bounded with a semaphore so I do not overload the server or exhaust file descriptors, and the client limits settings control the pool size. Timeouts are configured for connect, read, write and pool acquisition separately. Retries use the transport retries option for connection failures, plus backoff logic for status codes. Results are collected with exceptions captured per request, so one failure does not cancel the rest unless I want it to. The client is closed with a context manager. Compared with threads, async handles thousands of waiting I/O tasks with low overhead, but blocking libraries inside async code stall everything, so those calls go to an executor. For small jobs, a synchronous client is simpler.

Likely follow-ups

- Why reuse one client?
- Why bound concurrency with a semaphore?

🗺 **Bounded async calls**



Reuse one httpx AsyncClient for connection pooling, run requests concurrently with gather or task groups bounded by a semaphore, set separate timeouts, retry transient failures, capture per-request errors and keep blocking code out of the event loop.

certs.eknathalabs.com

```
sem = asyncio.Semaphore(20)
async with httpx.AsyncClient(timeout=10) as c:
    async def get(u):
        async with sem:
            return await c.get(u)
    res = await asyncio.gather(*(get(u) for u in urls), return_exceptions=True)
```

Interview tip: Say pool the client and bound the concurrency.

📚 **Official docs**

- Python: argparse
- Python: asyncio
- Requests documentation

Q123 How do you write Python for AWS Lambda or other serverless functions in an operations context?

APIs & cloud SDKs

🎙 **Spoken answer · ~73 s**

Serverless functions suit event-driven operations tasks such as reacting to an alarm, tagging new resources, processing a queue message or running a scheduled check. The handler receives an event and context, does a bounded amount of work and returns. Good practice: keep functions small and single-purpose; create SDK clients outside the handler so warm invocations reuse connections; load configuration from environment variables and secrets from a secrets manager with caching; make the work idempotent because events can be delivered more than once; handle partial failures for batch events by reporting failed items; and set sensible timeouts, memory and concurrency limits. Dependencies are packaged in the deployment archive, a layer or a container image, with attention to size and cold start time. Structured logging with request identifiers, metrics and tracing aid debugging. Permissions follow least privilege on the function's role. Dead-letter queues capture failures for inspection. Local testing uses unit tests with mocked clients and tools like the SAM CLI, and infrastructure is defined as code. Long-running or heavy jobs belong in containers or batch services.

Likely follow-ups

- Why create clients outside the handler?
- Why must handlers be idempotent?

Keep handlers small, create SDK clients outside the handler, read config from environment and secrets with caching, make processing idempotent, report partial batch failures, set timeouts and least-privilege roles, add structured logs and DLQs, and use containers for heavy work.

certs.eknathalabs.com

```
import boto3, os
s3 = boto3.client('s3')
def handler(event, context):
    for r in event['Records']:
        process(r)
    return {'ok': True}
```

Interview tip: Say small, idempotent, least privilege.

📚 **Official docs**

- AWS Lambda Developer Guide
- Python Documentation
- boto3 documentation

Q124 How do you authenticate to Azure and Google Cloud from Python without storing keys?

APIs & cloud SDKs

🎙 **Spoken answer · ~70 s**

Both clouds provide libraries that implement a chain of credential sources, so code works unchanged from a laptop to production. For Azure, DefaultAzureCredential from the azure-identity package tries environment settings, workload identity, managed identity, the Azure CLI login and more, in order, so on a virtual machine or container with a managed identity no secret is needed. For Google Cloud, application default credentials are discovered from the environment, attached service accounts on compute services, workload identity on Kubernetes, or the gcloud login during development, and the client libraries pick them up automatically. In CI, OIDC federation lets the pipeline exchange its own signed token for short-lived cloud credentials, with no stored secret. Permissions are granted to the identity with least privilege roles, scoped to the resource group or project. Service account key files are avoided, and if one is unavoidable it is stored in a secrets manager, rotated and monitored. Code should accept a credential object as a parameter for testability, and logs must never include tokens.

Likely follow-ups

- What is DefaultAzureCredential?
- Why avoid service account key files?

Use DefaultAzureCredential and Google application default credentials so code picks up managed identity, workload identity or developer logins, use OIDC federation in CI, grant least-privilege roles and avoid downloadable key files.

certs.eknathalabs.com

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.compute import ComputeManagementClient
client = ComputeManagementClient(DefaultAzureCredential(), sub_id)
```

```
import google.auth
creds, project = google.auth.default()
```

Interview tip: Say identities, not keys.

📚 **Official docs**

- Python Documentation

Q125 How do you build a clean generator-based paginator for any API, and test it?

APIs & cloud SDKs

🎙 **Spoken answer · ~71 s**

Many APIs return results in pages with a cursor, a next link or page numbers. Rather than scattering loops through the code, I wrap pagination in a generator function that yields items one at a time, hiding the paging mechanics. It requests the first page, yields each item, follows the next token until none remains, and respects rate limits and retries inside the request helper. Callers simply iterate, and because it is lazy, memory use stays low and processing can start before all pages arrive; they can stop early with islice. Edge cases include empty pages in the middle, duplicate items across pages when data changes during iteration, a maximum page size, and tokens that expire. Where possible, I use stable ordering and a since-timestamp filter for incremental syncs. Tests feed the paginator a fake client returning a scripted sequence of pages, including errors, and check the yielded items and number of calls. Many SDKs already provide paginators, such as boto3, which I use in preference to writing my own.

Likely follow-ups

- Why is a generator a good fit?
- What edge cases should tests cover?

Wrap paging in a generator that yields items and follows next tokens, with retries and rate limits in the request helper. It keeps memory low and lets callers stop early. Test with scripted fake pages including empty pages, duplicates and errors; use SDK paginators when available.

```
def items(client):
    token = None
    while True:
        page = client.list(page_token=token)
        yield from page['items']
        token = page.get('next')
        if not token: break
```

certs.eknathalabs.com

Interview tip: Say hide paging behind an iterator.

📚 Official docs

- REST API pagination (Google AIP-158)
- Boto3 documentation
- pytest documentation

Q126 How do asyncio TaskGroup, gather, timeouts and cancellation work together?

Concurrency & speed

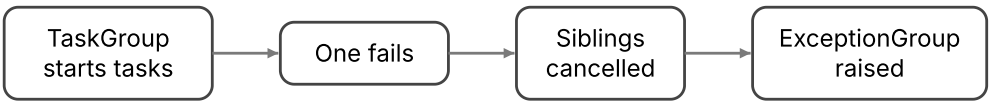
🎙 Spoken answer · ~70 s

Structured concurrency in modern asyncio makes concurrent code easier to reason about. A TaskGroup starts several tasks inside an async context manager and waits for all of them at exit; if one fails, the others are cancelled and the errors are raised together as an exception group, so nothing is left running unnoticed. Gather also runs awaitables concurrently, and with return_exceptions it collects failures as results instead of raising, which suits independent best-effort operations, but a plain gather does not cancel siblings reliably on error. Timeouts are applied with asyncio.timeout as a context manager or wait_for, which cancels the inner work when the deadline passes and raises a timeout error. Cancellation is delivered as a CancelledError exception inside the task, at an await point; code must let it propagate, or clean up in finally blocks and re-raise, never swallow it. Shielding protects critical sections. Blocking calls inside tasks prevent cancellation from working, so they go to executors. Understanding these rules prevents leaks, hung programs and lost errors.

Likely follow-ups

- What does a TaskGroup do on failure?
- Why must CancelledError propagate?

🗺 Structured concurrency



TaskGroup runs tasks together, cancels siblings on failure and raises an exception group; gather with return_exceptions collects results; asyncio.timeout bounds work and cancels it; CancelledError must propagate after cleanup in finally. Keep blocking calls out of the loop.

```
async with asyncio.timeout(30):
    async with asyncio.TaskGroup() as tg:
        for h in hosts:
            tg.create_task(check(h))
```

certs.eknathalabs.com

Interview tip: Say structured concurrency: nothing outlives its block.

📚 **Official docs**

- Python: asyncio

Q127 How do you run many subprocesses in parallel from Python and collect their results?

Concurrency & speed

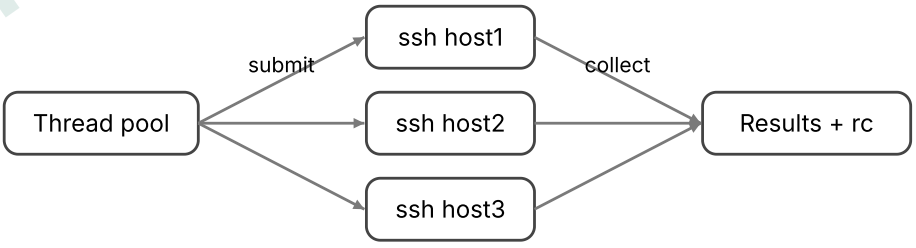
🎙 **Spoken answer · ~71 s**

For running commands on many items, I choose between threads, asyncio subprocesses and process pools depending on the workload. Because subprocesses do their work outside Python, a ThreadPoolExecutor calling subprocess.run is simple and effective: each thread waits for its child, and the GIL is not a problem. Futures return results and exceptions, and as_completed processes them as they finish. For a very large number of concurrent commands, asyncio.create_subprocess_exec avoids thread overhead and integrates with other async code, with a semaphore to limit concurrency. In both models I set a maximum number of workers appropriate to the machine and the target systems, pass arguments as lists, capture output with limits to avoid memory blow-ups, apply timeouts and kill commands that exceed them. Results are collected into a structure with the item, return code, output and duration, so failures are reported clearly at the end, and the exit code of the tool reflects whether any failed. Output from parallel commands should be tagged or buffered per item so lines are not interleaved.

Likely follow-ups

- Why do threads work fine for subprocesses?
- How do you avoid interleaved output?

🗺 **Parallel commands**



Use ThreadPoolExecutor with subprocess.run for simplicity, or asyncio subprocesses with a semaphore at scale; cap workers, pass argument lists, enforce timeouts, buffer output per item, collect return codes and durations and set the tool's exit code from the results.

```
with ThreadPoolExecutor(max_workers=8) as ex:
    futs = {ex.submit(subprocess.run, ['ssh', h, 'uptime'], capture_output=True, text=True,
timeout=30): h for h in hosts}
    for f in as_completed(futs): print(futs[f], f.result().returncode)
```

certs.eknathalabs.com

Interview tip: Say threads wait on children without GIL trouble.

📚 **Official docs**

- Python: subprocess
- Python Documentation

Q128 What makes code thread-safe in Python, and how do locks, queues and thread-local data help?

Concurrency & speed

🎙 **Spoken answer · ~73 s**

Even with the GIL, thread-unsafe code can break because operations that look simple, such as incrementing a counter or check-then-act on a dictionary, are made of several bytecode steps and threads can interleave between them. Thread-safe designs avoid shared mutable state, or protect it. A Lock or RLock guards a critical section, used as a context manager so it is always released; keep sections short and acquire multiple locks in a consistent order to avoid deadlock. A Queue is the best way to pass work and results between threads, since it handles locking internally and supports bounded sizes for backpressure. Thread-local storage gives each thread its own copy of data, such as a database connection or an HTTP session, where sharing is unsafe. Immutable data and message passing remove most hazards. Events and conditions coordinate threads, and daemon threads are avoided for important work because they are killed abruptly at exit. Libraries are checked for thread safety, as many clients are not. Tests with stress loops and tools such as thread sanitiser equivalents help find races.

Likely follow-ups

- Why is incrementing a counter unsafe?
- When use thread-local storage?

Avoid shared mutable state; guard it with locks used as context managers in consistent order, pass work through Queue, use thread-local storage for per-thread clients, prefer immutable data and check library thread-safety. Even with the GIL, check-then-act is not atomic.

```
lock = threading.Lock()
with lock:
    counters[key] += 1
```

```
local = threading.local()
def session():
    if not hasattr(local, 's'): local.s = requests.Session()
    return local.s
```

Interview tip: Say the GIL doesn't make your code atomic.

certs.eknathalabs.com

📚 **Official docs**

- Python: concurrent.futures
- Python Documentation

Q129 What are free-threaded Python and sub-interpreters, and how might they change automation code?

Concurrency & speed

🎙 **Spoken answer · ~71 s**

The GIL has limited CPU parallelism in threads for decades. Recent Python versions introduce an optional free-threaded build that disables the GIL, letting threads run Python code truly in parallel on multiple cores, and per-interpreter GILs with a standard library interface for sub-interpreters, which run isolated interpreters in one process with less overhead than separate processes. For operations scripts that are mostly I/O-bound, nothing changes: threads and asyncio already work well. The new options matter for CPU-heavy tasks such as parsing huge logs, compression or data crunching, where today multiprocessing with its serialisation costs is needed. Caveats are important: the free-threaded build is still maturing, some C extensions are not yet compatible and may re-enable the GIL or fail, single-thread performance can be slightly lower, and code that relied on the GIL for accidental safety needs proper locking. I would test the workload and dependencies on the new build, measure the gain, and keep production on the standard build until my libraries support it. Multiprocessing remains the reliable choice today.

Likely follow-ups

- Which workloads benefit most?
- What risks come with removing the GIL?

Free-threaded builds and sub-interpreters enable parallel CPU work without multiprocessing overhead, but I/O-bound scripts see little change. Extensions may be incompatible and code that relied on the GIL needs real locking, so measure and keep multiprocessing as the dependable option for now.

```
python3.13t -c "import sys; print(sys._is_gil_enabled())"
```

Interview tip: Say I/O-bound code doesn't need it; CPU-bound might.

📚 **Official docs**

- Python: concurrent.futures
- py-spy
- Python Documentation

certs.eknathalabs.com

Q130 When would you use a task queue such as Celery, RQ or Dramatiq for operations automation?

Concurrency & speed

🎙 Spoken answer · ~71 s

A task queue decouples submitting work from executing it. A producer enqueues a job, a broker such as Redis or RabbitMQ stores it, and worker processes pick it up, run it and store the result. It fits operations needs like long-running jobs triggered from a web interface or ChatOps command, scheduled and periodic jobs, retries with backoff, fan-out of work across many workers, rate limiting and processing events from alerts. Compared with simply starting a thread or subprocess, queues give durability, so jobs survive restarts, visibility into status, retries, concurrency control and horizontal scaling. Celery is feature-rich with scheduling, workflows and many integrations but heavier to run; RQ is simple and Redis-based; Dramatiq focuses on reliability with sensible defaults. Design points: tasks must be idempotent because delivery is at least once; arguments are small and serialisable; results are stored briefly; timeouts and dead-letter handling are set; and workers have limited privileges. For small needs, a systemd timer or cron may be enough, and cloud queues with functions can replace self-run workers.

Likely follow-ups

- Why must tasks be idempotent?
- When is cron enough?

Task queues provide durable, retried, scalable background execution for long jobs, schedules and fan-out. Choose Celery for features, RQ for simplicity, Dramatiq for reliability; make tasks idempotent with small arguments and timeouts, and use cron or timers when needs are small.

```
from rq import Queue
from redis import Redis
q = Queue(connection=Redis())
q.enqueue(run_backup, db='orders', job_timeout=3600)
```

Interview tip: Say at-least-once delivery means idempotent tasks.

📚 Start here (official docs)

- Python documentation

certs.eknathalabs.com

Q131 How do you test asynchronous code with pytest-asyncio or anyio?

Testing & quality

🎙 Spoken answer · ~70 s

Async functions must run inside an event loop, so a plain test cannot await them. Plugins such as pytest-asyncio or anyio provide markers and fixtures: a test declared as async def with the appropriate marker runs inside a loop managed by the plugin, and async fixtures can set up clients or servers and tear them down. I test behaviours rather than implementation: that concurrent calls produce the right results, that timeouts cancel work, that errors propagate from task groups and that cleanup runs. External calls are replaced with async mocks, using AsyncMock from unittest.mock or fake clients, so tests are fast and deterministic. Time-dependent logic can use a controllable clock or short real timeouts, because long sleeps slow the suite; some plugins offer fake time. Common pitfalls are forgetting to await, leaking tasks past the end of a test, sharing an event loop between tests with state, and blocking calls that hide problems. Tests should assert no pending tasks remain. Running the suite with warnings enabled surfaces unawaited coroutines.

Likely follow-ups

- What does AsyncMock provide?
- How do you avoid slow sleeps in tests?

Use pytest-asyncio or anyio markers and async fixtures, replace I/O with AsyncMock or fakes, test concurrency, timeouts, error propagation and cleanup, avoid long sleeps, ensure no tasks leak and enable warnings to catch unawaited coroutines.

```
@pytest.mark.asyncio
async def test_fetch(monkeypatch):
    client = AsyncMock(); client.get.return_value = {'ok': True}
    assert await fetch(client) == {'ok': True}
```

Interview tip: Say fake the I/O, assert on behaviour.

📚 Official docs

- Python: asyncio
- pytest documentation
- pytest documentation

certs.eknathalabs.com

Q132 How do you record and replay HTTP interactions in tests with VCR.py or responses?

Testing & quality

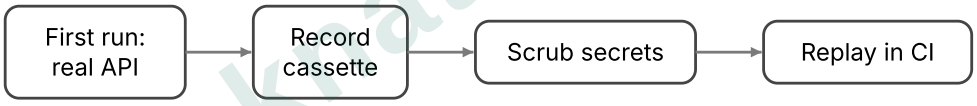
🎙 Spoken answer · ~68 s

Tests that call real APIs are slow, flaky and may cost money or change state, but hand-written mocks can drift from reality. Two approaches help. The responses library, for requests, and respx, for httpx, register fake endpoints with specific status codes and bodies, which is ideal for error and edge cases like rate limiting, malformed payloads and timeouts. VCR.py records real interactions into cassette files the first time a test runs and replays them afterwards, giving realistic payloads without network access. Recorded data must be scrubbed: filter authorisation headers, tokens, account identifiers and personal information before committing cassettes. Matching rules decide how requests are compared, such as method, URI and body. Cassettes are refreshed periodically, perhaps in a scheduled job against a sandbox account, to detect API changes. The test suite runs in a mode that forbids new recordings in CI, so unexpected calls fail. Contract tests against a sandbox complement them, because replayed data cannot prove the real service still behaves the same.

Likely follow-ups

- Why scrub cassettes?
- What do responses and respx offer that VCR does not?

🗺 Cassettes



Use responses or respx to script exact edge-case replies, and VCR.py cassettes for realistic recorded payloads. Scrub secrets from recordings, set matching rules, forbid new recordings in CI, refresh cassettes periodically and back them with sandbox contract tests.

```
@vcr.use_cassette('tests/cassettes/list.yaml', filter_headers=['authorization'])
def test_list(): ...
```

```
responses.add(responses.GET, 'https://api.example.com/items', status=429)
```

Interview tip: Say record real shapes, script the failures.

📚 Official docs

- pytest documentation
- Requests documentation

certs.eknathalabs.com

Q133 What is snapshot or golden-file testing, and where is it useful for infrastructure code?

Testing & quality

🎙 Spoken answer · ~71 s

Snapshot testing stores the expected output of a function in a file, and later runs compare current output with it. If it differs, the test fails and a developer reviews the change, accepting it by updating the snapshot when it is intended. It is valuable where outputs are large and structured, such as rendered configuration templates, generated Kubernetes manifests, report text, CLI help output and API responses after transformation. Instead of writing dozens of assertions, one comparison covers the whole result, and the diff in review shows exactly how behaviour changed. Libraries such as syrupy, pytest-snapshot or approvaltests manage the files. The risks are blindly accepting changes, huge snapshots nobody reads, and brittleness from volatile fields like timestamps, random identifiers and ordering, which should be normalised or sorted before comparison. Snapshots complement, not replace, targeted assertions of key properties. I keep them small, committed to the repository, reviewed in pull requests, and generated from deterministic inputs, so a change in output is always the result of a deliberate change in code.

Likely follow-ups

- What makes snapshots brittle?
- How should changes be reviewed?

Snapshot tests compare output with a stored golden file, useful for rendered configs, manifests and reports. Normalise volatile fields, keep them small and deterministic, review diffs rather than blindly updating, and pair them with targeted assertions.

```
def test_render(snapshot):
    assert render(vhosts=SAMPLE) == snapshot
```

```
pytest --snapshot-update
```

Interview tip: Say review the snapshot diff like code.

📚 Official docs

- Python: pathlib
- pytest documentation

certs.eknathalabs.com

Q134 What is mutation testing and does it tell you more than code coverage?

Testing & quality

🎙 Spoken answer · ~72 s

Coverage only shows which lines ran during tests, not whether the tests would notice if the code were wrong. A test that calls a function without asserting anything still counts as coverage. Mutation testing measures test strength: a tool such as mutmut or cosmic-ray makes small deliberate changes to the code, called mutants, such as flipping a comparison, changing a constant or removing a statement, and re-runs the tests for each. If the tests still pass, the mutant survived, meaning the suite does not detect that kind of fault. The mutation score, the proportion of mutants killed, is a better indicator of effectiveness than line coverage. The costs are long run times, since the suite runs many times, and equivalent mutants that do not change behaviour. So I run it selectively on critical modules, on changed files in a scheduled job, and use the surviving mutants as a to-do list for missing assertions rather than chasing a perfect score. It helps most for logic-heavy code such as parsers, retry policies and calculation functions.

Likely follow-ups

- What is a surviving mutant?
- Why is it expensive?

Mutation testing changes the code in small ways and checks that tests fail; surviving mutants reveal weak assertions that line coverage misses. It is slow, so run it on critical or changed modules and use survivors as a to-do list.

```
mutmut run --paths-to-mutate src/retry.py
```

```
mutmut results
```

Interview tip: Say coverage shows what ran, mutation shows what is checked.

📚 Official docs

- pytest documentation

certs.eknathalabs.com

Q135 How do you benchmark and guard against performance regressions in Python code?

Testing & quality

🎙 Spoken answer · ~68 s

Performance claims need measurement. For micro-benchmarks, the timeit module repeats a small snippet and reports the best time, while pytest-benchmark integrates benchmarks into the test suite, runs each many times, reports statistics such as mean, minimum and standard deviation, and can save results and compare runs, failing a build when a function becomes slower than a threshold. For larger programs, cProfile and tools like py-spy find the hot spots first, because optimising the wrong part wastes effort. Benchmarks use realistic data sizes, run on stable hardware, warm up caches and avoid noise from other processes; shared CI runners are noisy, so tolerances are generous or dedicated runners are used. Tracking results over time in a dashboard reveals gradual degradation. Memory benchmarks use tracemalloc or memray. I optimise algorithms and data structures before micro-tuning, and only after a measurable need, keeping readability. A regression test with a loose bound on an important code path, such as parsing a standard sample file, catches accidental quadratic behaviour.

Likely follow-ups

- Why profile before optimising?
- How do you handle noisy CI runners?

Use timeit for snippets and pytest-benchmark for tracked comparisons and thresholds, profile with cProfile or py-spy before optimising, benchmark realistic data, allow tolerance on noisy runners and track trends over time.

```
def test_parse(benchmark):
    benchmark(parse_log, SAMPLE)
```

```
pytest --benchmark-compare --benchmark-compare-fail=mean:20%
```

Interview tip: Say measure first, optimise the hot spot.

📚 Official docs

- Python Documentation

certs.eknathalabs.com

Q136 How would you automate offboarding or user lifecycle tasks across several systems with Python?

Ops automation

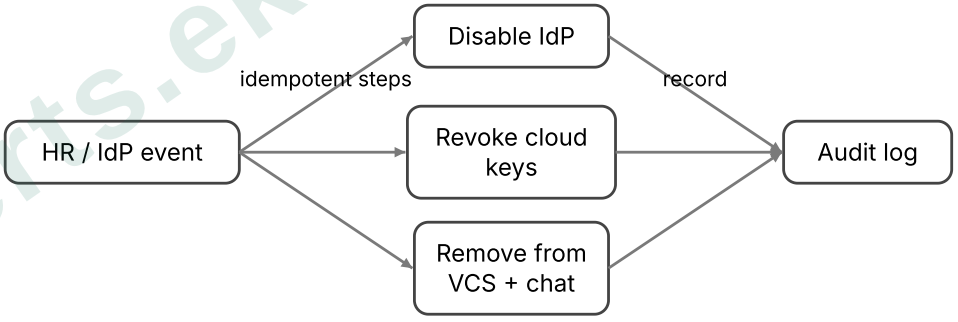
🎙 Spoken answer · ~77 s

Lifecycle automation touches many systems, such as the identity provider, cloud accounts, source control, chat and ticketing, so reliability and auditability matter more than speed. The trigger comes from an authoritative source, usually the HR system or the identity provider, delivered by webhook or polled. The workflow is a list of idempotent steps per system: disable the identity provider account, revoke sessions and tokens, remove access keys and group memberships, transfer ownership of resources and shared documents, reassign on-call schedules and open tickets, and archive mailboxes according to policy. Each step is retried safely, records its outcome and continues despite failures in unrelated systems, and failures raise a ticket for manual follow-up. Destructive actions such as deleting data are delayed, with a retention window and approval. A dry-run mode and a report of what would change allow review. API credentials for the automation are narrowly scoped, stored in a secrets manager, and every action is logged with who or what triggered it. Joiners and movers use the same framework, driven by role definitions rather than copied permissions. Regular reconciliation finds accounts that escaped the workflow.

Likely follow-ups

- Why delay destructive steps?
- How do you detect accounts that were missed?

🗺 Offboarding workflow



Trigger from the HR or identity source and run idempotent per-system steps with logged outcomes, delay destructive actions, support dry-run, use scoped secrets-managed credentials and audit logs, handle joiners and movers by role, and reconcile regularly to catch missed accounts.

```
python offboard.py --user jdoe --dry-run
```

```
python offboard.py --user jdoe --apply --ticket HR-1432
```

Interview tip: Say idempotent steps, audit trail, reconcile later.

certs.eknathalabs.com

📚 **Official docs**

- Python Documentation

Q137 How do you build a diagnostics bundle collector that gathers evidence during an incident?

Ops automation

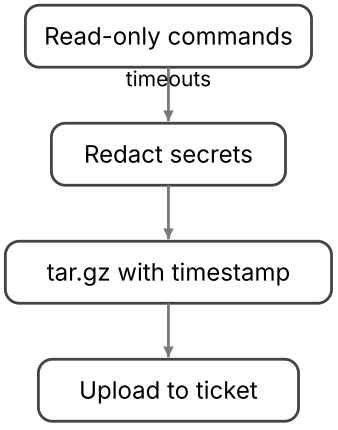
🎙 **Spoken answer · ~77 s**

During incidents, engineers waste time running the same commands on different hosts. A collector script automates evidence gathering consistently and safely. It runs on a host or against a list of hosts and collects a defined set of read-only data: system load, memory and disk usage, process lists, network connections and listening ports, recent kernel and service logs, container and orchestrator state, configuration files with secrets redacted, and application-specific health output. Each command runs with a timeout and its output is captured with the timestamp and exit code, so failures are visible and a hung command cannot block the rest. The results are packaged into a compressed archive named with the host and time, written with restrictive permissions, and uploaded to a controlled location, with a size limit to avoid filling the disk. Sensitive data is filtered by patterns, and the collection is logged. It must have minimal impact, avoiding heavy commands on struggling systems, and be safe to run repeatedly. Kubernetes versions gather pod descriptions, events and logs. A runbook links to it, and the bundle attaches to the incident ticket for later analysis.

Likely follow-ups

- Why use per-command timeouts?
- How do you avoid collecting secrets?

🗺 **Evidence bundle**



Run a fixed set of read-only commands with timeouts, record output, exit codes and timestamps, redact secrets, package and upload with restrictive permissions and size limits, minimise load on the host, and link the bundle to the incident ticket.

```
tar czf /tmp/diag-$(hostname)-$(date -u +%Y%m%dT%H%M%SZ).tgz -C /tmp/diag .
```

certs.eknathalabs.com

```
subprocess.run(['journalctl','-u','app','--since','-30min'], timeout=20,
capture_output=True)
```

Interview tip: Say consistent, read-only, redacted, time-boxed.

📚 **Start here (official docs)**

- Python documentation

Q138 How do you produce cloud cost reports and find savings with Python and billing exports? Ops automation

🎙 **Spoken answer · ~72 s**

Cost visibility starts with the billing export: AWS Cost and Usage Reports delivered as Parquet to object storage, Azure cost exports, or the Google billing export in BigQuery. A Python job loads the data with pandas, DuckDB or the query client, normalises it, and groups by tags such as team, environment and application, plus service, region and usage type. Reports show trends over time, month-on-month changes, the top spenders and anomalies, such as a sudden daily jump. Untagged spend is flagged with owners to chase. Finding savings combines billing data with utilisation metrics: instances with low CPU for weeks, unattached volumes, old snapshots, idle load balancers, oversized databases, missing savings plans or reserved capacity coverage, and data transfer hot spots. Recommendations include estimated monthly savings and an owner. Output goes to dashboards, spreadsheets or chat summaries, with weekly scheduling. The job uses read-only roles, and cost allocation tags must be activated for data to appear. Showback to teams drives behaviour, and automation can tag resources or stop non-production resources outside working hours.

Likely follow-ups

- Why do tags matter for cost reports?
- How do you detect cost anomalies?

Load billing exports (CUR Parquet, Azure exports, BigQuery) with pandas or DuckDB, group by tags, service and region, trend and flag anomalies and untagged spend, combine with utilisation to find idle or oversized resources, and deliver weekly showback with owners.

```
import duckdb
duckdb.sql("select tag_team, sum(cost) c from 'cur/*.parquet' group by 1 order by c
desc").show()
```

Interview tip: Say tags make spend attributable; utilisation makes savings real.

📚 **Official docs**

- Python Documentation

certs.eknathalabs.com

Q139 How do you use the ipaddress module and Python to plan and validate IP address ranges and CIDR allocations?

Ops automation

🎙 Spoken answer · ~73 s

The standard ipaddress module represents addresses and networks as objects with correct arithmetic, replacing fragile string manipulation. IPv4Network and IPv6Network parse CIDR blocks, expose the netmask, number of addresses and host iterators, test membership with the in operator, and check whether networks overlap, are subnets of one another or are private. The subnets method splits a block into smaller ones of a given prefix length, and supernet or collapse_addresses merge ranges. For planning, a script can read existing allocations from a file or IPAM system, find overlaps between VPC, on-premises and partner ranges that would block peering, calculate free blocks, and propose the next available subnet of a requested size. Validation in CI can reject proposals that overlap or fall outside approved ranges. The module handles network and broadcast addresses and edge cases such as point-to-point links. Cloud reserved addresses per subnet are accounted for separately. Output is a clear table, and the single source of truth should be an IPAM tool or version-controlled data, with the script enforcing consistency before Terraform applies changes.

Likely follow-ups

- How do you detect overlapping ranges?
- What does collapse_addresses do?

Use ipaddress to parse CIDRs, test overlaps and membership, split with subnets, merge with collapse_addresses, find free blocks and propose allocations, then validate proposals in CI against the source of truth before infrastructure changes.

```
import ipaddress as ip
a = ip.ip_network('10.0.0.0/16')
print(a.overlaps(ip.ip_network('10.0.4.0/22')))
list(a.subnets(new_prefix=20))[:3]
```

Interview tip: Say never parse CIDRs with string splitting.

📚 Official docs

- RFC 4632: CIDR
- Python Documentation

certs.eknathalabs.com

Q140 How would you build a simple capacity forecasting script from metrics history?

Ops automation

🎙 Spoken answer · ~69 s

Capacity problems are cheaper to prevent than to fix at 3 a.m., so I forecast from history. The script pulls a series from the metrics backend, for example daily peak disk usage, memory or request rate for the last several months, through the Prometheus HTTP API with range queries. It cleans the data, handles gaps, and aggregates to a sensible resolution. For steadily growing resources, a linear regression with numpy or a library such as statsmodels projects when usage will cross a threshold, producing a days-until-full estimate. For seasonal workloads, I use models that capture weekly patterns, or compare year-on-year, and always show the uncertainty. Prometheus itself offers predict_linear for alerting. The output is a ranked report of resources predicted to hit limits within a horizon, with owners and growth rates, feeding tickets and planning. I validate forecasts against what actually happened, keep models simple and explainable, and avoid false precision, because step changes from launches or migrations break trends. Alerts for time-to-full complement the report.

Likely follow-ups

- What does predict_linear do?
- Why keep forecasting models simple?

Pull history via the Prometheus range API, clean and aggregate it, fit a linear or seasonal trend to estimate days until a threshold, rank resources by urgency with owners, validate against actuals and keep models simple; predict_linear can alert on time-to-full.

```
predict_linear(node_filesystem_avail_bytes[7d], 14*24*3600) < 0
```

```
import numpy as np
slope, intercept = np.polyfit(days, usage, 1)
```

Interview tip: Say forecast, then verify against reality.

📚 Official docs

- prometheus_client

certs.eknathalabs.com

Q141 How do you manage Grafana dashboards and alerts as code using Python or the API?

Observability & tooling

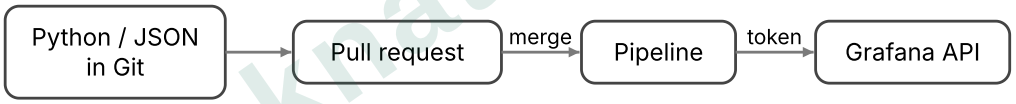
🎙 Spoken answer · ~74 s

Dashboards edited by hand in the interface drift, vanish and differ between environments, so I treat them as code. The Grafana HTTP API lets a script create, update and export dashboards, folders, data sources and alert rules. Dashboards are described in JSON or generated with a library such as grafanalib or the Grafana Foundation SDK in Python, which produces consistent panels from small functions, so a standard service dashboard can be created for every application from a template with the service name as a parameter. The definitions are stored in Git, reviewed in pull requests and deployed by a pipeline with an API token of limited scope, or provisioned from files on disk or through a Kubernetes sidecar. Terraform's Grafana provider is another option. Variables, consistent naming and tags help navigation. Alert rules are defined alongside dashboards with clear thresholds and runbook links. Exported JSON should be cleaned of volatile fields like ids and versions to give small diffs. Changes can be validated by loading them in a staging instance, and drift detection compares live dashboards with Git.

Likely follow-ups

- Why generate dashboards from code?
- How do you keep diffs small?

🗺 Dashboards as code



Keep dashboards and alert rules in Git, generated with grafanalib or SDKs from templates parameterised by service, deployed by pipeline with scoped API tokens or provisioning, cleaned of volatile fields, validated in staging and checked for drift.

```
requests.post(f'{GRAFANA}/api/dashboards/db', headers={'Authorization': f'Bearer {TOKEN}'},
json={'dashboard': dash, 'folderUid': folder, 'overwrite': True})
```

Interview tip: Say dashboards are code; generate the standard ones.

📚 Official docs

- Grafana documentation
- Python Documentation
- Requests documentation

certs.eknathalabs.com

Q142 How do you ship logs to Elasticsearch, OpenSearch or Loki from Python, and when should you not?

Observability & tooling

🎙 Spoken answer · ~75 s

Applications should normally write logs to standard output or files and let an agent such as Fluent Bit, Vector or Promtail collect and ship them, because that decouples the application from the logging backend, handles buffering and retries, and keeps working when the backend is down. Direct shipping from Python is justified for scripts and jobs with no agent, serverless functions and structured audit events. Clients exist: the opensearch-py and elasticsearch libraries provide bulk helpers that send many documents in one request, and Loki accepts pushes over HTTP, with libraries or logging handlers available. Good practice is batching, using the bulk API, with timeouts, retries and backoff; running the shipping in a background thread or queue handler so application latency is unaffected; bounding memory with a queue limit and a drop policy so a slow backend cannot take down the application; and using structured JSON with consistent field names and index templates. Credentials are scoped and rotated. Logging failures must not crash the program, and a local fallback file keeps data when the backend is unreachable. Avoid sending sensitive data.

Likely follow-ups

- Why prefer an agent for services?
- How do you avoid blocking the application?

Prefer writing to stdout and letting agents like Fluent Bit or Vector ship logs. For scripts and functions, ship directly using bulk APIs with batching, timeouts, backoff, background queues with bounded memory and a local fallback, structured fields and scoped credentials.

```
from opensearchpy import helpers
helpers.bulk(client, ({'_index': 'ops-logs', '_source': d} for d in docs),
request_timeout=30)
```

Interview tip: Say agents for services, bulk APIs for scripts.

📚 Official docs

- Grafana Loki documentation
- Elasticsearch reference
- Python Documentation

certs.eknathalabs.com

Q143 How do you write synthetic probes or blackbox checks in Python for user journeys?

Observability & tooling

🎙 Spoken answer · ~75 s

Internal metrics can look healthy while users cannot log in, so synthetic probes exercise the service from the outside like a user. A probe is a script that performs a realistic request sequence, such as loading a page, logging in with a test account, searching, adding an item and checking out, and verifies content, status codes and timing at each step. Libraries range from requests for API journeys to Playwright for real browser flows including JavaScript. Probes run on a schedule from multiple locations, including outside the cluster and different regions, to catch DNS, certificate, CDN and routing issues. Results become metrics, success and duration per step, scraped by Prometheus or pushed to a monitoring service, with alerts on consecutive failures and slow responses, and failure screenshots or response bodies stored for diagnosis. Test accounts and data must be isolated, safe and clearly labelled so that probes do not pollute analytics or trigger fraud rules, and secrets are managed properly. Probes should be stable to avoid false alarms, with retries only for network flukes, and their own health is monitored.

Likely follow-ups

- Why run probes from outside the cluster?
- How do you avoid flaky probes?

Script realistic user journeys with requests or Playwright, run on a schedule from several locations, export per-step success and duration as metrics, alert on consecutive failures, keep screenshots on failure, isolate test accounts and keep probes stable and monitored.

```
r = requests.post(f'{BASE}/login', json=creds, timeout=5)
assert r.status_code == 200 and 'token' in r.json()
PROBE_OK.labels('login').set(1)
```

Interview tip: Say probe the journey users take, not just /health.

📚 Official docs

- Google SRE Book: Distributed tracing & logs (Monitoring chapter)
- Prometheus Blackbox exporter
- Python Documentation

certs.eknathalabs.com

Q144 How do you detect anomalies in metrics with simple statistics before reaching for machine learning?

Observability & tooling

🎙 Spoken answer · ~70 s

Simple methods catch most operational anomalies and are easier to explain and trust. A rolling mean and standard deviation give a z-score, flagging points several deviations away from recent behaviour. Median and median absolute deviation resist outliers better. Comparing with the same time last week handles daily and weekly seasonality, and a percentage-change threshold detects sudden jumps. Exponential smoothing or a seasonal decomposition separates trend and cycles from the residual. For rates, comparing against a forecast band works well. Implementation uses pandas or numpy on data from the Prometheus API, and Prometheus can do some of this in queries with functions such as avg_over_time, stddev_over_time and offset. Alerting on anomalies needs care: require sustained deviation, set minimum absolute thresholds to avoid noise on tiny numbers, and tune with historical data to measure false positives and misses. Machine-learning approaches are considered when seasonality is complex or there are thousands of series, but they add maintenance and opacity. Whichever method, alerts must be actionable and link to a dashboard and runbook.

Likely follow-ups

- Why compare to the same time last week?
- Why add minimum thresholds?

Start with rolling z-scores, median absolute deviation, week-over-week comparison and seasonal decomposition using pandas or Prometheus functions. Require sustained deviation and minimum absolute thresholds, tune on history for false positives, and use ML only when simple methods fall short.

```
z = (s - s.rolling(288).mean()) / s.rolling(288).std()
alerts = s[z.abs() > 4]
```

```
stddev_over_time(rate(http_requests_total[5m])[1h:])
```

Interview tip: Say simple, explainable statistics first.

📚 Official docs

- prometheus_client

certs.eknathalabs.com

Q145 How do you profile CPU and memory of a Python program with cProfile, scalene or memray, and interpret the results?

Observability & tooling

🎙 **Spoken answer · ~73 s**

Profiling starts from a question: is the program slow because of CPU, I/O waiting or memory pressure? Cprofile, built in, records function call counts and cumulative times; sorting by cumulative or total time shows which functions dominate, and the output can be viewed with visualisation tools like snakeviz. Its overhead is moderate and it does not see inside C extensions. Sampling profilers such as py-spy attach to running processes with little overhead, ideal in production, and produce flame graphs where wide frames show where time goes. Scalene separates Python time from native time and I/O wait, and shows line-level CPU and memory use, even for GPU. For memory, tracemalloc compares snapshots to find allocation sites, and memray records every allocation with flame graphs and can find leaks and peak usage. Interpretation focuses on the biggest contributors: a hot loop, repeated parsing, an accidental quadratic algorithm, large intermediate lists or an unbounded cache. After changing code, I re-profile to confirm the gain. Profiles should use realistic inputs, because toy data hides the real hot spots.

Likely follow-ups

- Why use a sampling profiler in production?
- What does memray show?

Use cProfile for call-level timing, py-spy for low-overhead sampling and flame graphs in production, scalene for line-level CPU, native and memory split, and tracemalloc or memray for allocation hot spots. Profile with realistic data, fix the biggest contributor and re-measure.

```
python -m cProfile -o out.prof job.py && snakeviz out.prof
```

```
py-spy record -o flame.svg -- python job.py
```

```
memray run job.py && memray flamegraph memray-job.bin
```

Interview tip: Say question first, tool second, re-measure after.

📚 **Official docs**

- Python: pathlib
- py-spy
- Python Documentation

certs.eknathalabs.com

Q146 How do you design testable operations code using dependency injection and thin wrappers around cloud SDKs?

Patterns & scenarios

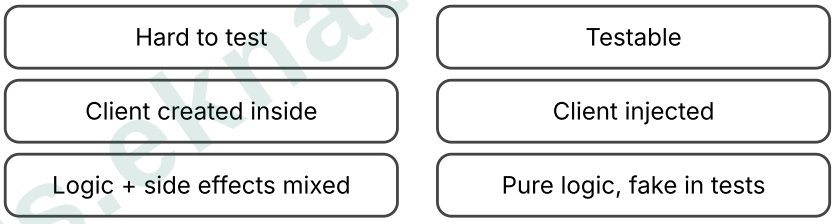
🎙 Spoken answer · ~73 s

Scripts that create clients inside functions and call them directly are hard to test without mocking globals. A better structure passes dependencies in: functions or classes receive the client, configuration, clock and logger as arguments, so tests supply fakes. I wrap the parts of an SDK I actually use in a small interface, for example a class with methods to list volumes and delete a volume, hiding pagination and error mapping. Business logic, such as deciding which volumes are old and unattached, works on plain data and is a pure function that is trivial to test with tables of cases. The real wrapper is tested in a few integration tests, and everything else uses an in-memory fake implementing the same interface, which is often more reliable than patching with mocks. This also makes changing providers or API versions easier. Protocols or abstract base classes document the contract. Composition happens at the edge, in main, where real clients are created. The result is smaller functions, faster tests and clearer separation between decisions and side effects.

Likely follow-ups

- Why separate decisions from side effects?
- Why prefer fakes over heavy mocking?

🗺 Testable design



Inject clients, config, clock and logger; wrap the SDK parts you use in a small interface; keep decision logic as pure functions on plain data; use in-memory fakes in tests, with few integration tests; wire real dependencies in main.

```
class VolumeAPI(Protocol):
    def unattached(self) -> list[Volume]: ...
    def delete(self, vid: str) -> None: ...
def plan(vols, now, max_age): return [v for v in vols if now - v.created > max_age]
```

Interview tip: Say pure decisions, thin side-effect edges.

📚 Official docs

- OWASP Top Ten
- Python: venv
- pytest documentation

certs.eknathalabs.com

Q147 How do you model long-running workflows as state machines in Python so they can resume after failure?

Patterns & scenarios

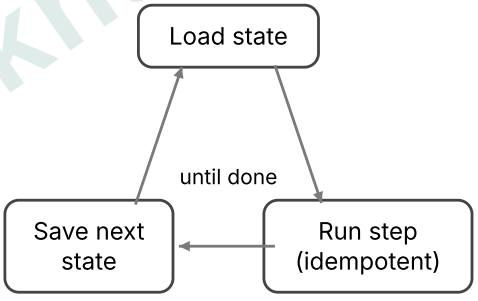
🎙 Spoken answer · ~76 s

A multi-step operation like provisioning a cluster or migrating data can fail anywhere, so it should be resumable. I model it as a state machine: a defined set of states, such as requested, network created, nodes ready and verified, with allowed transitions, and each step as a function that moves the workflow to the next state. The current state and relevant data are persisted after every step in a database or a state file, so on restart the program reads the last state and continues from there. Steps are idempotent, since a crash after performing an action but before saving the state means the step runs again. Failures record the error and either retry with backoff or move to a failed state that waits for human action, with compensating steps for rollback where needed. Timeouts and a maximum number of attempts prevent endless loops. The state history provides an audit trail and visibility, and a command can show the status of all workflows. Libraries like transitions help, and workflow engines such as Temporal, Airflow or Step Functions provide this durably for complex cases.

Likely follow-ups

- Why must steps be idempotent?
- When do you adopt a workflow engine?

🗺 Resumable workflow



Define explicit states and transitions, persist state after each step, make steps idempotent so a crash can replay them, handle failures with retries, failed states and compensation, add timeouts and history, and use engines like Temporal for complex or critical flows.

```
STATES = ['requested', 'network', 'nodes', 'verified']
while (s := load_state(job)) != 'verified':
    STEPS[s](job); save_state(job, NEXT[s])
```

Interview tip: Say persist state, make steps replayable.

certs.eknathalabs.com

📚 **Official docs**

- Python Documentation

Q148 How do you implement a plugin or registry pattern for checks, collectors or exporters in Python?

Patterns & scenarios

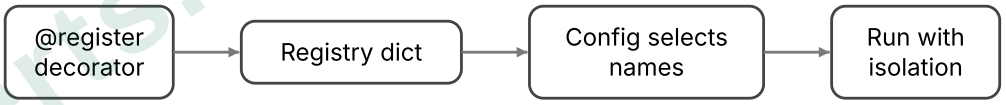
🎙 **Spoken answer · ~72 s**

When a tool needs many similar components, such as health checks, inventory collectors or report generators, a registry pattern avoids long if-else chains and lets teams add components without editing core code. A base class or protocol defines the interface, for example a check with a name and a run method returning a result. A registry maps names to implementations, filled by a decorator that registers each class or function on import, or by discovering plugins through entry points, so installed packages can contribute. The core iterates over the registry, applying configuration to select which components run, with per-component options and error isolation, so one failing plugin is reported but does not stop the others. Each component has a timeout, and results use a common structure with status, message and metrics. Configuration validation ensures unknown names are rejected early. Tests cover each component in isolation plus the runner. Documentation explains how to write a new one in a few lines. This design keeps the tool extensible, makes behaviour uniform and encourages reuse.

Likely follow-ups

- How does a decorator register components?
- Why isolate plugin errors?

🗺 **Registry**



Define a common interface, register implementations with a decorator or entry-point discovery, select them from configuration, run each with a timeout and error isolation, and return results in a shared structure. Validate config, test components individually and document the extension steps.

```
CHECKS = {}
def check(name):
    def deco(fn): CHECKS[name] = fn; return fn
    return deco
@check('disk')
def disk(cfg): ...
```

Interview tip: Say register by decorator, isolate failures.

certs.eknathalabs.com

📚 **Official docs**

- Prometheus: Instrumentation best practices
- prometheus_client
- Python Documentation

Q149 How do you implement a token bucket or sliding-window rate limiter in a Python client or service?

Patterns & scenarios

🎙 **Spoken answer · ~75 s**

Rate limiting protects both your own service and the APIs you call. In a token bucket, a bucket holds up to a maximum number of tokens that refill at a steady rate; each request consumes a token, and when the bucket is empty the request waits or is rejected. This allows bursts up to the bucket size while enforcing an average rate. A sliding window counts requests in the last interval and rejects when the count exceeds the limit, which is more precise about boundaries but stores more data. For a client, I implement the bucket with a monotonic clock, a lock for thread safety, and a method that either sleeps until a token is available or reports the wait time; async code uses an asyncio lock and sleep. Libraries such as limiter packages or aiolimiter provide this. Honouring the server's rate-limit headers and Retry-After is better than guessing, and I combine limiting with retries and backoff. For limits shared across many processes or hosts, state lives in Redis, using atomic scripts. Metrics on waits and rejections show when limits bind.

Likely follow-ups

- How does a token bucket allow bursts?
- How do you share a limit across processes?

🗺 **Token bucket**



A token bucket refills at a steady rate and allows bursts up to its capacity; sliding windows count recent requests. Implement with a monotonic clock and lock, honour server headers, combine with backoff, and use Redis with atomic scripts for shared limits.

```
class Bucket:
    def __init__(self, rate, cap): self.rate, self.cap, self.t, self.n = rate, cap,
time.monotonic(), cap
    def take(self):
        now = time.monotonic(); self.n = min(self.cap, self.n + (now-self.t)*self.rate);
self.t = now
        if self.n >= 1: self.n -= 1; return 0
        return (1-self.n)/self.rate
```

Interview tip: Say bursts allowed, average rate enforced.

certs.eknathalabs.com

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling
- Python: argparse

Q150 How do you add reloadable configuration and feature flags to a long-running Python service without redeploying?

Patterns & scenarios

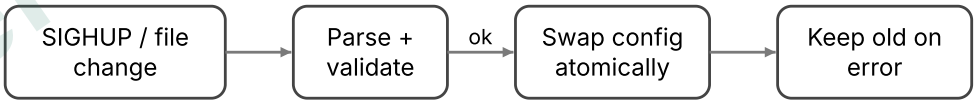
🎙 **Spoken answer · ~74 s**

Some behaviour needs to change while the service runs: log level, thresholds, allow lists or the enabling of a new code path. For configuration, the service loads settings into an immutable object and replaces the whole object atomically on reload, triggered by a SIGHUP, a file change or a periodic poll of a config store, so no request sees a half-updated state. New configuration is parsed and validated first, and if invalid the old one stays active and an error is logged and counted. Code reads the current configuration through an accessor, not by caching values at import time. Feature flags add a layer: a flag service or file defines flags, possibly with percentage rollouts or targeting by user or environment, and code checks a flag at the decision point. Flags have owners and expiry dates and are removed after rollout, because stale flags create untested combinations. Defaults are safe if the flag service is unreachable. Changes are audited, and critical settings avoid dynamic changes when a restart is safer. Tests cover both flag states and reload failure.

Likely follow-ups

- Why swap the config object atomically?
- What is the risk of old flags?

🗺 **Hot reload**



Load settings into an immutable object and swap atomically on SIGHUP, file change or poll after validating; keep the old config on error. Use flags with safe defaults, owners and expiry, audit changes, test both states and remove flags after rollout.

```
signal.signal(signal.SIGHUP, lambda *a: reload())
def reload():
    global CFG
    new = load_and_validate(path)
    if new: CFG = new
```

Interview tip: Say validate first, swap atomically, retire flags.

certs.eknathalabs.com

📚 **Official docs**

- Martin Fowler: Feature Toggles
- Python Documentation

certs.eknathalabs.com